

Vanilla Web

Building apps without frameworks



Maximiliano Firtman



**MEAP Edition
Manning Early Access Program**

Vanilla Web
Building apps without frameworks
Version 4

Copyright 2026 Manning Publications

For more information on this and other Manning titles go to manning.com.

Welcome

Dear reader,

Thanks for picking up the MEAP edition of *Vanilla Web*. This book is for developers who want to build modern web experiences by leaning on the platform itself: HTML, CSS, JavaScript, browser APIs, and HTTP, without defaulting to heavyweight frameworks, libraries, or complex stacks.

To get the most out of the book, you should be comfortable writing and debugging everyday web code: fundamentals in HTML, CSS, and JavaScript, plus basic confidence using browser DevTools. If you haven't used them in several years, you are still learning, or your experience is only coding with libraries and frameworks, I've got you covered in this book.

A quick note as you read: the skills here are not "anti-framework." They're foundational. Even if you spend your day in React/Angular/Vue, you still benefit from understanding the underlying platform and its APIs, because that's what everything else is built on.

As a web developer myself, starting my first website 30 years ago, I've seen patterns, libraries, and frameworks come and go, while the fundamentals stay and make you a better developer. We will focus on: simplicity, performance, stability, usability, and accessibility. And it doesn't mean we will take a prehistoric web development lesson or create ugly or outdated apps. Vanilla web is the present, and it's a modern platform.

What you can expect to learn is how to build web apps that stay understandable, portable, and fast by design. We'll focus on:

- Writing semantic HTML that improves usability, accessibility, and even how machine agents interpret your UI.
- Using modern CSS techniques (including "Web UI" patterns) to create web components and polished interfaces without bolting on big CSS frameworks.
- Updating and managing the UI directly with the DOM APIs and creating reusable web components and client-side routers, all with data binding support.

The web runtime is also changing: people won't always consume your work as pixels in a tab. AI-driven, agentic browsers increasingly parse structure, metadata, and accessibility signals to summarize and act on content, raising the bar for clean semantics and predictable behavior.

MEAP is a conversation, not a monologue. Please post questions, corrections, and “this part was confusing” feedback in the [liveBook Discussion Forum](#); those comments directly shape the final book.

—Maximiliano Firtman

brief contents

CHAPTERS

- 1 Hello Vanilla Web*
- 2 Understanding the Web Platform*
- 3 The User Interface*
- 4 Vanilla CSS*
- 5 Vanilla JavaScript*
- 6 The Document Object Model API*
- 7 Web Components*
- 8 Working with Templates and Slots*
- 9 Advanced web components*
- 10 Client-side routing*
- 11 Working with data*
- 12 Micro-libraries*
- 13 Beyond basics*
- 14 Building an app*

1 Hello Vanilla Web

This chapter covers

- What's Vanilla Web
- Analyzing the architecture and technologies used
- Learning what's different compared with using frameworks
- Growing your developer skills when learning Vanilla Web
- Working with the main challenges and risks

Let's say you have a new project in mind, and you're in that first moment when you want to decide on the stack you'll be using before even having full details of the problem to solve. Choosing a technology stack without first understanding the requirements is a mistake, often described by the adage: 'If all you have is a hammer, everything looks like a nail.'

Now, replace "hammer" with "React": "When you only know React, everything looks like something React can solve." You can replace React with any other frontend library or framework stack: Angular, Vue, Next.js, you name it. In the past few years, many developers - particularly new ones - have been heavily invested in one library or framework, often overlooking the underlying technology or the basics of creating a basic web app without it.

Libraries and frameworks are just tools in your toolbox. There will be many projects in which using a library can be a good idea; we will not deny that in this book. However, many projects don't require a library, and adding one can create more problems in terms of security, performance, and maintainability.

With Vanilla you work directly with the platform, you understand exactly what's happening in your app: no hidden abstractions, no magic conventions, no background processes you didn't ask for. The browser becomes your runtime, not a black box you build on top of. That transparency often leads to simpler architectures, smaller bundles, and fewer surprises during maintenance. More importantly, it gives you technical autonomy: you're no longer tied to a release cycle, a breaking-change announcement, or a dependency ecosystem to keep your project alive. Sometimes the biggest advantage of Vanilla Web isn't what you remove, but what you regain, control.

Frameworks can be remarkable, but they come with a price when overused. They introduce runtime and tooling overhead that is not always needed: virtual DOM diffing, hydration layers, complex build chains, a high cost to keep your app updated and secure and dependency churn. Studies show that Vanilla JavaScript still outperforms most frameworks in core UI operations. That's why some companies opted for Web Components and server-rendered HTML; not to be trendy, but to be fast, stable, and in control. In a world of LLMs writing code and users demanding speed, the ability to understand and own your codebase is more valuable than ever.

In this book, we won't be advocating against using libraries and frameworks; instead, we will advocate for learning web technologies with a modern approach. Understanding the tools available lets you choose intentionally, not automatically follow trends. We will explore APIs and design patterns for creating websites and web applications without libraries and frameworks. Additionally, if you come from a framework or library as your primary development platform, we will discuss how to retain the good ideas from there in the Vanilla Web world.

This book will push you on the right path if:

- You are a developer starting with web development and already have a basic knowledge of HTML, CSS, and JavaScript syntax.
- You are an experienced web developer who mainly uses a library or framework.
- You have been developing websites from 1990s-2015, and you've lost track of what's possible today on web languages.

NOTE While there are no strict definitions, in frontend web development, a *framework* is generally a structured set of tools and conventions that define how to build and organize web applications, handling tasks such as rendering, state management, and routing to streamline development. *Libraries*, on the other hand, solve one problem at a time and don't enforce any particular structure. However, community practices can evolve around a library, turning it into an ecosystem that resembles a framework, as is often the case with React.

1.1 What Vanilla Web is not

Before we explore what it is, let's start by clarifying what it's not and debunk some common myths in the meantime.

Vanilla Web is not a library or framework. It's also not an old, deprecated, or low-level programming paradigm. It's built on the most modern features of the web platform (HTML, CSS, and JavaScript) as they exist today, not as they were in 1997. Vanilla Web is not against using tools. It's not "anti-framework" or a cult of purists yelling, "No React allowed!". It simply means that you decide when and why to bring tools in, not because a template or tutorial said so, but because it solves a specific problem in your project.

Vanilla Web is not incompatible with modern practices. You can write modular code, adopt design patterns, create reusable components, and even test your app with modern tooling. The difference is that you are in control of the architecture, not the other way around.

Vanilla Web is not always slower to develop. For simple to mid-sized projects, it can be faster because there's less overhead. You're not avoiding tools out of dogma, you're avoiding unnecessary abstractions that delay shipping and complicate debugging.

Vanilla Web is not just for hobbyists. Large-scale platforms like GitHub, GOV.UK and Frontend Masters have adopted Vanilla Web principles for serious production apps. This approach scales, if you architect it with intention.

NOTE You don't need a framework to start, the browser provides the runtime, APIs, and consistency modern apps depend on.

1.2 What Vanilla Web is

Vanilla Web is an expression that defines the use of core web technologies, APIs, and languages to create websites and web applications without the need for frameworks or large libraries to define the architecture and development flow.

Vanilla Web involves thinking of a web solution without considering a particular framework or library beforehand, thereby reducing the complexity and abstraction layers on top of Web technologies. This approach eschews popular frameworks like React, Angular, Vue, or Next.js in favor of "plain vanilla" techniques, often resulting in simpler, faster, and more maintainable codebases in the long run. Rather than abstracting web technologies away, developers work directly with the DOM (Document Object Model), events, and browser features.

WARNING Vanilla Web does not mean doing everything manually, as if it were 1995 again. We're not writing spaghetti code inside `<script>` tags. We're not avoiding tooling because we're afraid of it. We're simply avoiding unnecessary abstraction layers that get in the way of understanding, performance, and control.

We can divide the Vanilla Web field into separate building blocks using Web Standards:

- *Semantic HTML*: using the full potential of HTML to define the semantic structure, content, and data of your website or web application. That means we won't be using a lot of `<div>` and random class names, as it's common to deploy when using specific frameworks and libraries. It will also include the usage of Custom Elements and Web Components.
- *Vanilla CSS*: using modern CSS (Cascade Style Sheets) techniques without pre-compilers, such as SASS or LESS, and core libraries, such as Bootstrap or Tailwind, for the layout and design of your web apps. We will set the layout, design system, theme, and animations using this language. We will also utilize new techniques, grouped today under the name "Web UI," to design custom components, such as Popups, Carousels, and custom dropdown select elements, without relying on libraries or designing with JavaScript.
- *Vanilla JavaScript* (or just Vanilla JS): using JavaScript and browser APIs to code your frontend models, logic, and UI management. That means we won't be using React, Vue, Angular, or any other addition to the stack that defines how you write and design your solution. Micro-libraries are welcome, as well as some pre-processors in case you have specific needs.
- *HTTP*, This means interacting directly with HTTP endpoints via conventions rather than relying on specific client-server contracts like GraphQL or the complex hydration strategies found in frameworks like Next.js.

A BRIEF HISTORY OF VANILLA

The word "vanilla" originates from the ice cream industry, where vanilla is the base flavor of ice cream, unadulterated by any additional flavoring. It began being used in software development in the early 80s to describe software and tools used without extensions. It was popularized as an adjective during the 90s, for example, within the Unix ecosystem, where a "vanilla kernel" signified an unmodified kernel, and within the gaming world, where a "vanilla game" was defined as one running without any user-created mods or add-ons.

With Vanilla Web, you can create any web experience, no matter if it's a static website, a web application, a game, or a VR experience. Libraries, frameworks, and entire web ecosystems are merely abstraction layers built on top of the core web technologies available in web browsers and the HTTP protocol.

After a decade of frameworks and architectural libraries, we've entered a period where many companies and websites are returning to their roots, and Vanilla Web technologies are seeing a rise in usage.

1.3 How it works

When you learn a new approach to application development, it is a good idea to understand the entire stack, including all the technologies, practices, and languages involved. A Vanilla Web project will typically have the stack seen in Figure [1.1](#):

- *Frontend development*: the process that builds the client-side interface users see and interact with.
 - Semantic HTML, including the usage of Custom Elements and Web Components
 - CSS, including web user interface modern patterns
 - JavaScript
 - Browser APIs
 - WebAssembly (optional)
- *Network protocols*: they will let us communicate the client (the browser) with the server.
 - HTTPS
 - WebRTC
 - WebSockets
- *A Web Server*: it serves the website or web application and connects the backend when needed.
- *Backend development*: the process that builds the server-side logic, databases, and APIs that power an application's functionality behind the scenes.
 - Any solution, such as Node.js, Python, Java, .NET, PHP, Ruby and more.
 - Cloud-based providers
 - Server-less solutions
- Additional Services consumed by the backend
 - Database Servers
 - Cloud-based AI models
 - Authentication servers

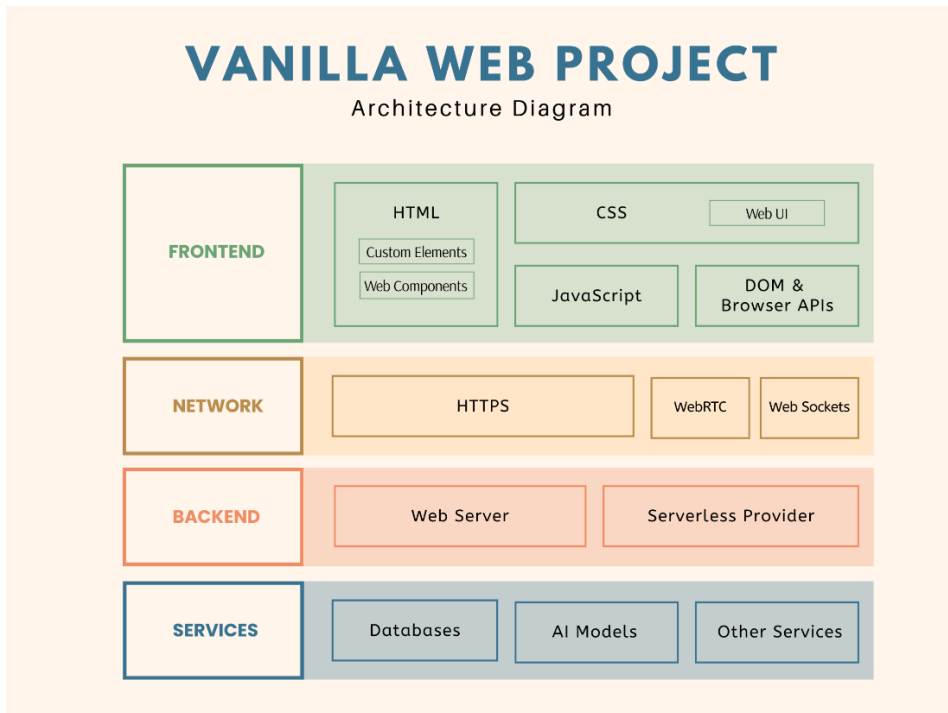


Figure 1.1 The technology stack we will be using while developing a Vanilla Web project is divided into different layers that may or may not be developed by the same developer or team. In this book, we will focus more on the top layers.

In this book, we will focus most of our effort on the frontend layer, using HTTP as the network protocol to communicate with any backend suite of technologies you choose.

1.4 Remembering our goals

You don't always need the latest fancy tech to achieve your goals. Using the latest fancy tech is not a goal. Tools are not goals. Nor is using Vanilla Web.

When building a website or web application, there are likely many goals at different levels; however, the primary ones are typically to solve a problem for a user in the most effective way possible, which in turn increases conversion rates. Users don't care about the tools you use to create the experience.

Your goals will likely focus on increasing conversion rates (whatever that means for the project) and, indirectly, improving usability, accessibility, high performance, scalability, and maintainability for the future.

As a professional, you will pick the best tool set you've found based on the resources you have and the goals you need to achieve.

While Developer Experience (DX) is good to have and it's part of your resources, but if you need to pick between a five stars DX but with a two stars performant solution and a two stars DX with a five stars performance solution, you should always pick the one more performant over the one with better DX. Performance, accessibility, usability, and other product-focused goals are the ones that actually move the needle on conversion and help you meet your business objectives.

In the past few years, the fever of shipping more in less time made us forget about the real goals, the problems we are trying to solve, and the right priorities. The Web platform has evolved significantly, and it's no longer true that choosing a fully or partially Vanilla Web approach leads to a poorer user experience compared with using a library or framework.

We are now living in a new era for developers, with AI coding assistants, AI coding agents, AI engineers, and vibe coding; most of the code is written these days and will be partially or defined by an AI model in the future.

This era means we need to rethink what's the right Developer Experience, as the coding task is shifting dramatically.

TIP The most common Generative IA models, *LLMs* (Large Language Models), work better with a shorter context. When vibe-coding or assisting a developer in coding, testing, or debugging, they will work faster and more accurately with simple project structures that do not require hundreds of external resources. Therefore, Vanilla Web is an optimal solution for the AI coding era.

For a future of code written mainly through AI, it makes more sense to request solutions that are faster to execute, have a better lifespan, require less maintenance in the future, and have a simpler project structure rather than continuing to request the use of frameworks and libraries that target human developer experience.

1.5 Using Vanilla Web

To start, it's essential to understand that every web application and website can be implemented with Vanilla Web because frameworks and libraries are just abstraction layers on top of the Web platform. However, it's also true, as with any toolbox, that not every tool is best suited for every project.

Good candidates for using Vanilla Web are:

- Small to medium static or content-oriented websites: blogs, wikis, and similar content sites.

- Landing pages: Pages designed for marketing, campaigns, or product promotion benefit greatly from Vanilla Web because they demand fast load times and maximum reliability. With no complex client-side architecture to maintain, you get instant rendering, excellent Core Web Vitals, and simpler deployment, all of which directly improve engagement and conversion.
- Performance-critical applications: If the user experience demands maximum speed or if the app performs numerous rapid DOM updates (such as animations or real-time data), a handcrafted vanilla solution can be optimized to be highly efficient.
- Apps with long lifespans and low change rates: corporate websites or apps that are heavily based on data coming from APIs or with hardcoded logic with very low behavior change rates over time.
- Dynamic web apps: When performance is your top priority, any web application that renders dynamic data can be developed without frameworks.

There is subtle category of "vanilla-ish" apps that utilize micro-libraries rather than large frameworks. For example, some projects use Alpine.js or htmx, which aren't exactly frameworks but rather small helpers, alongside primarily server-rendered or static pages. While not "zero libraries," these apps maintain the spirit of Vanilla Web by keeping things minimal and close to HTML.

1.5.1 Mixing Vanilla with libraries

When working with web projects, it's perfectly possible to mix and match a Vanilla web solution with a framework solution, such as a Vanilla JavaScript client-side app that interacts with a widget written in React or a Static Web-Generated blog with a dynamic section that renders an Angular application.

TIP You can interchange data between a framework-based app and a Vanilla Web app that runs as one app using different mechanisms.

The other option is also possible, as you can extend or exit a library ecosystem and write Vanilla Web code at any time within most frameworks because we are consistently executing web apps in the same engine: the browser.

Mixing a framework with Vanilla Web technologies happens at many levels, including short snippets. For example, Adam Rackis, a software engineer, has posted that he spent hours trying to solve a simple video player problem using React contexts, and he realized he could solve the problem with just a few lines of Vanilla JavaScript outside of React, as shown in Figure [1.2](#).



Figure 1.2 Sometimes, thinking out of the framework or library for a minute will save you hours of trying to adjust the solution to the mental model of it.

It's also worth noticing that if you need to create a plugin or add-on for a library or framework, you may also need to understand how to deal with the browser APIs directly, which means the skills that we will learn in this book will help you even when you are working with frameworks and large libraries.

THE EXAMPLE OF GITHUB

GitHub.com is a massive, dynamic web application, yet it utilizes Vanilla Web technologies for many places. GitHub has adopted web standards by using custom elements (web components) and a limited application of libraries, including React, to enhance behavior on some parts. They found this approach beneficial for keeping the app fast and avoiding a heavy client framework. As their engineers noted, "Web Components offered better encapsulation... and we were happy to experiment since it doesn't incur any upfront cost or buy-in to a specific framework."

Over the years, they've created numerous elements like `<relative-time>` for timestamps and `<details-dialog>` for modal dialogs, which operate as plain HTML elements and show that even at scale (hundreds of engineers, huge codebase), a mostly vanilla approach can work with careful tooling and conventions. GitHub's success here is a strong testament: they kept their frontend "as lightweight, fast, and accessible as possible" by avoiding large frameworks. Instead, they lean on the platform (and augment it with small tools for productivity).

1.6 Enhancing your skills

It's often recommended that beginners grasp vanilla JavaScript before moving to frameworks, as it provides a solid foundation. Many report a sense of empowerment and "joy" rediscovering what can be done with just the basics once liberated from framework conventions.

As the developer, Ben Holmes wrote in his X account at <https://x.com/BHolmesDev/status/1653026700404445186>:

"Genuinely think every frontend dev should build their own framework at some point (...) Building Slinkity (an Eleventy template project) taught me more JavaScript and Web APIs in a month than years of React ever did. — Ben Holmes

In the end, Vanilla Web development underscores a broader lesson: sometimes, the most straightforward solution, directly using the platform, is the most elegant and sufficient. It reminds us that frameworks are tools, not prerequisites and that choosing not to use one can be a deliberate, strategic decision to achieve specific goals.

As the Web continues to evolve, the "vanilla" approach remains a relevant and robust approach in the modern web ecosystem, not as a rejection of progress but as a different path that coexists with frameworks, each learning from the other to make the Web better for everyone.

NOTE Any web developer with solid fundamentals in HTML, CSS, and JavaScript can understand the project without additional training or prior experience.

A Vanilla Web project can be a point of pride; it's like handcrafted code in an age of boilerplate, which isn't to romanticize it unnecessarily. Still, the ethos appeals to engineers who value understanding over abstraction. It's not about saying "never use frameworks"; sometimes, understanding the terrain is more potent than importing a map someone else drew.

Learning how to use Vanilla Web technologies makes you a better web developer. It is a skill that is even tested at many companies in recruiting tests, even if you are applying to a job that will involve coding in a framework or library such as Angular or React.

For junior engineers, understanding Vanilla Web development is an invaluable foundation; it's the baseline upon which all frameworks are built. Knowing how to manipulate the DOM, handle events, and manage state in pure JS will make you far more effective when you use any framework, and you'll be able to judge when a framework is truly needed versus when something can be done with a few lines of vanilla code.

For senior engineers, having the capability to go frameworkless offers an additional architectural option to meet performance budgets or reduce complexity. It also helps in debugging problems in framework-based apps since, ultimately, the framework outputs vanilla constructs.

Using Vanilla JS/CSS gives developers complete control over every aspect of the application. There is no "magic" happening under the hood; you wrote all the code, so you can reason about precisely what it does, which can lead to a deeper understanding of web fundamentals and, ultimately, better problem-solving skills. Many engineers find that writing vanilla code forces them to learn how the browser works (e.g., event propagation, DOM reflows, CSS cascading), which can make them more effective even when they later use frameworks. In a vanilla project, if something isn't performant or is causing a bug, you can dig in and optimize it directly without being constrained by framework abstractions.

You're free to choose algorithms and data structures best suited to your use case. This control also extends to markup and styling: you emit exactly the HTML you want (useful for SEO and accessibility), and you can avoid the extraneous wrapper `<div>` or structures that some frameworks impose. In short, Vanilla Web development can be more transparent; developers have a mental model closely aligned with what's happening in the browser. As a result, debugging can be more straightforward: the call stack is your code, not a framework's internals. You don't need specialized dev tools to inspect a virtual DOM or component hierarchy. You can use regular browser inspectors to examine the real DOM and state.

1.7 Characteristics

Large libraries and frameworks often sacrifice specific characteristics in favor of Developer Experience (DX).

The main characteristics of Vanilla Web compared with using large libraries and frameworks are:

- Web standards as building blocks

- Simple and more secure project organization
- High performance
- Interoperability
- Cost reduction
- Minimal tooling

We will get deeper into these topics in the next chapter.

1.8 Challenges

Vanilla Web development trades convenience for control. It's a balance: you give up some "short-term comforts," like instant routing or state libraries, in exchange for long-term simplicity and performance. But if a project objectively needs those comforts at scale, a framework might be the pragmatic choice.

Some of the challenges are:

- Lack of structured architecture
- Projects with very short timelines and large teams: the lack of an imposed structure in Vanilla can slow things down or lead to inconsistency unless carefully managed.
- Applications requiring complex DOM diffing or optimized rendering
- Need for server-side rendering out of the box
- Avoid the need to reinvent the wheel
- Projects requiring strong typing or enterprise standards
- Developer familiarity
- Knowledge requirement
- Handling state and complexity the right way
- Avoid DOM manipulation overhead

WARNING Vanilla Web might be simpler overall, but not always easier during initial development due to the lack of pre-built solutions.

Web technologies have undergone significant evolution over the past few decades. In the rest of the book, we will see how powerful and simple they can be today without the addition of any abstraction layer on top.

1.9 Summary

- Vanilla Web is an expression that defines the use of core web technologies, APIs, and languages to create websites and web applications without the need for frameworks or large libraries to define the architecture and development flow.

- Vanilla Web is not about nostalgia or doing things the hard way; it's about using modern web standards with purpose and control.
- Using Vanilla Web involves the usage of web standards as building blocks, including HTML, CSS, and JavaScript, with micro-libraries and additional tools when needed for the front end.
- A Vanilla Web project is compatible with any web server and backend stack using HTTP and other standard protocols for communication.
- The point of learning Vanilla Web is to understand you don't always need a framework because the browser is already your most powerful, battle-tested platform where you can run your apps.
- In the AI-assisted coding era, simpler codebases with fewer dependencies are easier to generate, test, and maintain.
- Vanilla Web approach gives you full ownership of your architecture, making your project more straightforward, more secure, and easier and cheaper to maintain.
- Good candidates for using Vanilla Web are small to medium static or content-oriented websites, performance-critical applications, dynamic web apps, and apps with long lifespans and low Change rates.
- Vanilla Web architecture scales, but it needs discipline, and real-world platforms like GitHub prove that "vanilla" doesn't mean small.
- Learning Vanilla Web makes you a better web developer: even if you use a framework, the skills make you more effective.
- Vanilla Web projects leverage simpler project organization, faster performance, cost reduction, and minimal tooling.
- There are challenges when using Vanilla Web compared with framework-based stacks, mainly coming from the lack of strict rules, as we are in charge of enforcing our guidelines and design patterns.

2 Understanding the Web Platform

This chapter covers

- The core building blocks of the web platform: rendering engines, browsers, protocols, and APIs
- Choosing an application architecture for navigation, rendering, and UI models
- Mixing HTML, CSS, JavaScript, and other standard specs together in Vanilla projects
- Implementing key design patterns and compatibility guidance
- Performance, accessibility, distribution, and modern web apps with Vanilla Web

The web platform is powerful by default, but with more than a decade of focusing on libraries, frameworks, and large stacks, we often overlook the most valuable features. Let's cover the most essential advantages of using Vanilla Web in our projects.

2.1 Building blocks

We will create software using web standards as building blocks. We can see a diagram of all the pieces in [Figure 2.1](#) that we will explain in this chapter. Web standards evolve slowly and with consensus from different providers, which leads to a more robust platform that makes our code less likely to need re-testing or re-coding when the platform grows, as sometimes happens with frameworks.

These standards don't appear by accident. They're shaped through open, collaborative processes led by organizations such as the W3C, the WHATWG, the IETF, and various browser vendor working groups. Proposal drafts are debated, tested, and refined in the open, with input from browser engineers, developers, accessibility experts, and the broader community. Because every major browser vendor participates in these discussions, the resulting specifications prioritize interoperability and long-term stability.

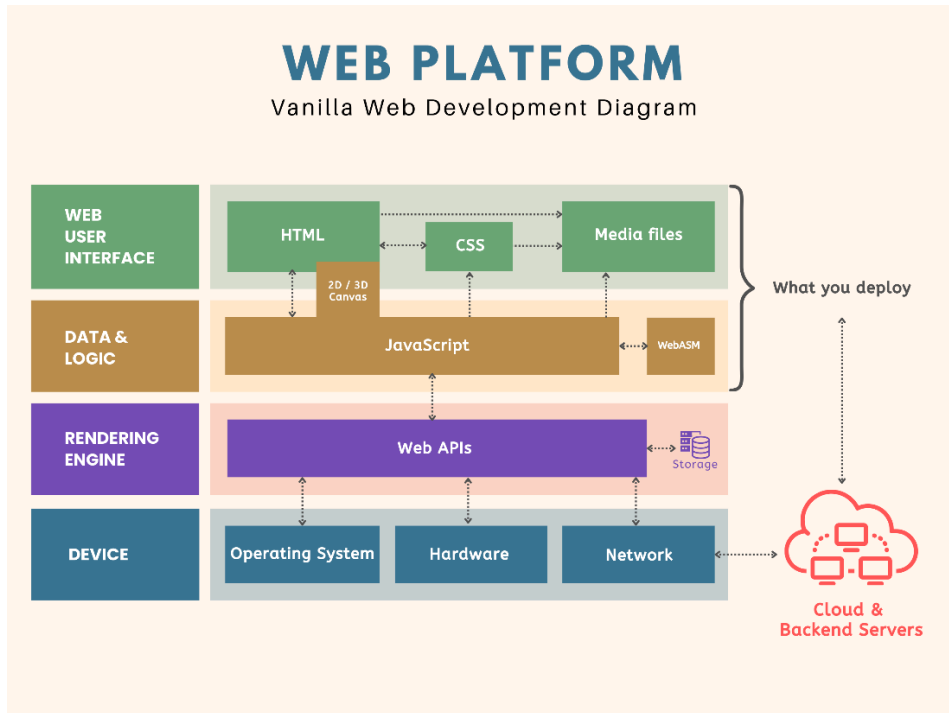


Figure 2.1 A diagram to understand the pieces that make a Vanilla Web application, which parts are our responsibility, and which parts are delivered by the web rendering engine, most typically a browser.

Many frameworks, such as Angular or Vue, ship yearly major versions. While you are not forced to upgrade your project to every new major version when using those frameworks, staying too far behind the latest stable version can lead to security issues and buggy implementations. Frequent project upgrades require continuous time and developer efforts to keep your app tested and working correctly, adding a high maintenance cost to every project.

Building on standards gives you a predictable foundation: the platform evolves carefully, intentionally, and with a strong bias toward backward compatibility. You don't incur the constant upgrade costs that come with versioned frameworks, and the need to update your project for new APIs or platform changes is minimal.

FUTURE-PROOF BY DESIGN

If you open a website created in 1997, it will be appropriately rendered in today's devices and browsers, including mobile phones, smartwatches, or even in a VR environment. The long lifespan is an excellent feature of using Vanilla Web technologies for your solution. When using a framework or extensive library instead of web standards, building, testing, and deploying the solution 20 years later will become more problematic. Even if deployed, it may have security issues or larger problems than a Vanilla solution.

2.1.1 Rendering engines

Rendering engines are the silent workhorses of the Web. They take our HTML, CSS, and JavaScript and transform them into the interactive experiences we see in browsers, apps, and even AI chat interfaces. Today, the landscape is consolidated around three major families: Blink (used by Chrome, Edge, Opera, Brave, and most Chromium-based browsers), WebKit (Safari on all Apple platforms), and Gecko (Firefox). Alternative engines occasionally emerge as well, such as the Ladybird project (<https://ladybird.org>), which demonstrates that the browser ecosystem is still evolving and that independent implementations can influence future standards and ideas.

Each engine aims to follow the same web standards, but implementation details, performance optimizations, and release cadences differ. That's why standards bodies and efforts like the Baseline project (web.dev/baseline) are critical: they help ensure that the experience of "Vanilla" development is consistent and future-proof across engines. We will cover the Baseline project later in this chapter.

The JavaScript code that will run in your web app is compiled on the fly (JIT - Just in Time) and executed by a browser's engine. Although it's separate from the rendering engine, the execution engine is typically grouped with it. For example, as execution engines, Chrome uses V8 and Firefox uses SpiderMonkey. The important part is that we, as developers, can't select the engine, and we must provide the source JavaScript code to them.

In practice, you rarely need to care which engine is running your code, but subtle differences still surface. Safari's release cycle is tied to iOS versions, meaning some APIs arrive later. Chromium's faster cadence means experimental features can land quickly, but also shift under your feet. Firefox continues to be a pioneer in privacy innovation. For a Vanilla developer, the goal is not to memorize quirks but to lean on feature detection and progressive enhancement so your app adapts gracefully to whichever engine executes it.

Every rendering engine instance (such as a browser tab or window) will have a page navigation with a URL and a main document loaded. That document will be our website or application. Our content can also be rendered in inner documents, such as within an *iframe* which is a subdocument that is loaded within another document. In every case, our web solution will have access to a set of abilities and APIs.

THE BROWSER

The *browser* is no longer just "the thing you open websites with." It is now the universal runtime for apps, services, and even AI assistants. Modern browsers integrate password managers, payment APIs, GPU acceleration, sandboxing, and privacy controls. For web developers, the browser is the framework: it brings the rendering engine, the JavaScript runtime, and hundreds of standardized APIs out of the box. Choosing Vanilla means accepting that this runtime is powerful enough to be the foundation of nearly every app.

It's important to remember that browsers today are multi-surface. A user might open your web app in a desktop window, a mobile browser tab, a picture-in-picture floating window, or even as part of a voice or AI-driven session. This flexibility makes the browser the most versatile execution environment in computing today. You aren't coding for "Chrome" or "Safari", you're coding for the web layer that spans across all of them.

AGENTIC BROWSERS

A new category of browsers is emerging: *agentic browsers*. Unlike classic browsers that wait for user navigation, agentic browsers are driven by AI models or assistants that parse, summarize, and sometimes act on web content automatically. Think of ChatGPT's agent mode, Perplexity's Comet, ChatGPT Atlas or the AI agents embedded into Chrome, Edge or Arc. They still rely on a rendering engine under the hood (usually Chromium), but they change how content is consumed: not just pixels on screen, but structured data streams fed into an AI.

For Vanilla developers, this shift matters because your code may no longer be read by humans only. Markup, metadata, and accessibility attributes become critical signals for machine agents. A well-structured Vanilla app (semantic HTML, progressive enhancement, ARIA roles) doesn't just improve accessibility for humans, it also enhances "machine accessibility." Building not only for screens and clicks, but also for AI agents that navigate on behalf of users, is part of the future.

INSTALLED WEBAPPS (PWAS)

On most browsers and operating systems, including desktop, mobile, and other devices, you can install a URL as a standalone web application. While this is not a standard spec, it's more of a user experience design pattern available on most platforms. The name of this pattern differs per platform, but it's typically known as a *Progressive Web App*, or *PWA*. It can also be referred to as *Homescreen Web App* (used on iOS and iPadOS), or just installed Web Apps (the name used lately on macOS and Chrome desktop).

Progressive Web Apps (PWAs) allow the same browser engine to run in an "installed" mode, outside the tab. Once installed, a PWA looks like a native app: it has its own icon, can launch offline, and may integrate with the OS through notifications, file handlers, and background sync. For the user, the experience is seamless, the line between a "web page" and an "app" disappears. For developers, the great news is that you are still coding with HTML, CSS, and JavaScript. The browser simply packages your app with extra OS integration.

For example, there is an open source PWA named Squoosh that lets you convert and resize images in different formats. It's available at <https://squoosh.app>, and it's an installable web app, or PWA. That means it works offline and can be installed on every operating system - desktop and mobile- using the "Install", "Add to Homescreen", or "Add to Dock" menu items or buttons, as shown in Figure 2.2.

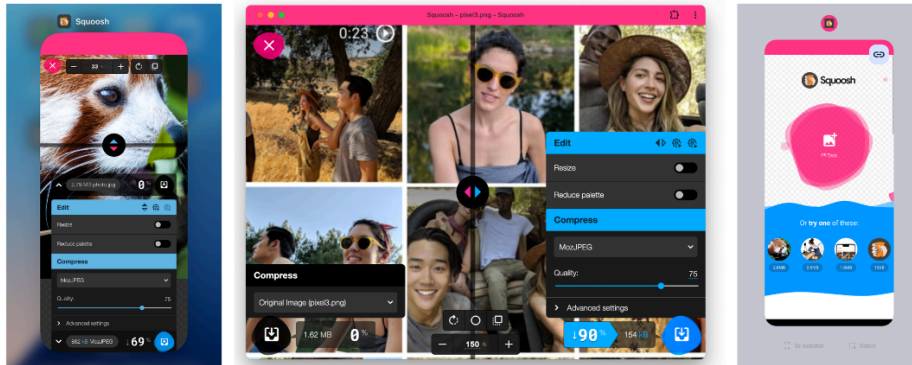


Figure 2.2 This is the open-source PWA image converter Squoosh installed on iOS, macOS, and Android with different browsers and web engines rendering its content in a standalone window.

The key takeaway is that PWAs don't require special frameworks or proprietary SDKs (Software Developer Kits). The installability comes from a manifest file, a standard API that we will cover in Chapter 3. Your Vanilla Web app can be installed on Android, iOS, Windows, macOS, iPadOS, ChromeOS, and even some Linux distributions with no extra build step.

Each browser and platform handles the creation of a PWA native launcher for us automatically on installation. For instance, under certain conditions on Android, users receive a native Android package known as WebAPK, which is compiled and signed by Google Play, Samsung, or Huawei. We don't need to compile anything, just ship the content using the web specs.

NOTE If you don't provide the manifest file, some browsers, such as Safari on all Apple platforms or Chrome on desktop, let you install the Web app as a standalone anyway with default settings.

App Stores

Most app stores support publishing PWAs as apps or games. In this case, we create a PWA Launcher native app for that specific platform (for example, using Swift for Apple App Store, .NET for Microsoft, or using Kotlin for Google Play Store). That launcher will insert a full-screen rendering engine on the operating system, loading our web content inside as if it were a headless browser window.

NOTE Android supports Trusted Web Activities (TWA), a technology that lets the default installed browser on that device render standalone or full-screen web apps (without the browser user interface). When you publish a PWA on Android app stores, including Google Play Store, Amazon App Store, and Huawei AppGallery, you typically use a TWA for your PWA.

NATIVE IN-APP BROWSERS

A typical pattern over the past few years has been to render websites within in-app browsers on apps that are not browsers but work with links. For example, if you are in the Gmail application on your phone and the email has a link, or if you are swiping in a social network and you click on a link to an external website, an in-app browser will appear instead of the standard default browser. This pattern allows the app to encapsulate external navigation within its app, rather than redirecting you to the browser. This feature increases the time you spend using the app and makes it easier to return to it.

NOTE When the user installs your website as a PWA and your content has a link to an external website, it will be rendered in a native In-App browser by default.

Apple operating systems and Android offer a native way to offer in-app browsing experiences without the native app developer implementing their own rendering engine. The In-App browser is managed by the default browser installed on the operating system. From our perspective, it functions like an additional tab within the browser, lacking an editable URL bar and a complete browser user interface, as illustrated in Figure [2.3](#).

On Safari, the mechanism is known as Safari View Controller, and it's available on every Apple platform; on Android, it's called Custom Tabs, and most browsers, including Chrome, Edge, Samsung Internet, and Firefox, implement it. These in-app browsers share storage, session logins and plugins with the web browser, which increases the user experience on that device.

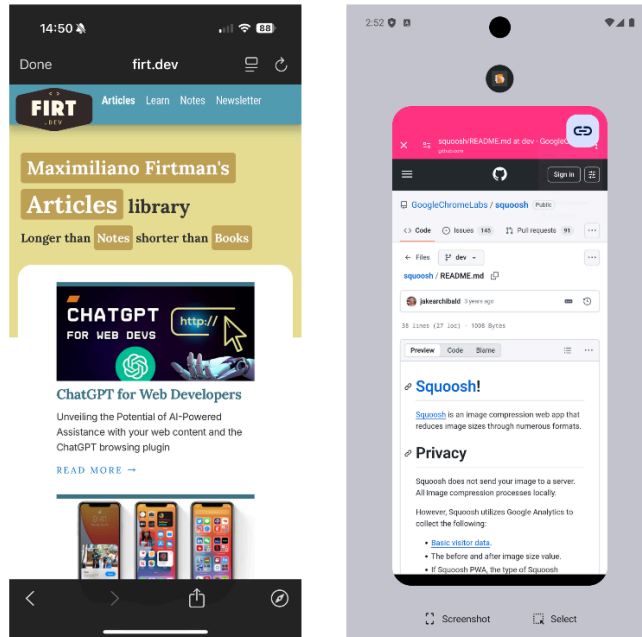


Figure 2.3 Native in-app browsers utilize the installed browser within another app in a special window. For instance, we observe at the left a Safari View Controller browsing a website on the social network X (Twitter) and at the right a Custom Tab in Android accessing an external website from the Squoosh installed PWA.

WEB VIEWS AND CUSTOM IN-APP BROWSERS

Web views are lightweight browser containers embedded inside other applications. They use rendering engines similar to those in the browser, but they appear as part of another app's UI. These rendering engines are updated with the OS or as separate components from the default browser. For example, you may be using Chrome 150 on your Android phone, but the Chromium WebView on that device may be on Chrome 140. API availability may differ as well between the browser and the webview on that particular device.

Some in-app browsers, such as those on TikTok, Facebook, X (formerly Twitter) or Instagram, use web views when you click on a specific link. In these cases, these apps are creating their own custom internal web browsers using the web view as a tool instead of using the native in-app browsing platforms, as we can see on Figure 2.4. Using a custom web-view implementation lets the app developer customize the UI (such as X's sliding post overlay) and track how users interact with the website.

WARNING It's important to highlight that storage, sessions, and other settings inside a web view are completely isolated from the user's default browser. As a result, a user may be logged in when using the browser, but not when accessing the same website through a social network's in-app browser.

We should always test our Vanilla project on these in-app browsers, as API compatibility may differ from the default browser on that same device.

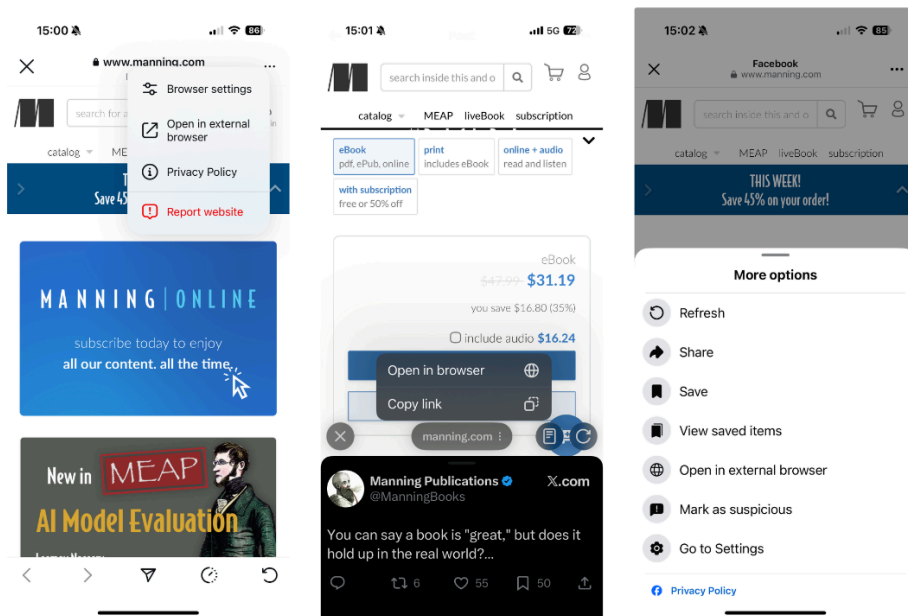


Figure 2.4 Meta social network apps, such as Instagram (left image), X (middle image) and Facebook (right image), use web views for their internal in-app browsers when users click on external links within posts and profiles.

If you've ever logged into a payment provider or opened a help screen inside a native app, you've used a web view. If you have used Visual Studio Code or other derived IDE you've already used a web view.

Web views also power hybrid frameworks like Cordova, Electron, and Capacitor, but you don't need those abstractions to target them. A well-written Vanilla app can drop directly into a web view with no changes. As AI browsers and super apps evolve, many of them are effectively wrappers around web views, which makes standards-based development even more valuable.

For developers, this means the same Vanilla codebase can live in multiple contexts: a browser tab, a PWA, or a web view inside another app.

SUPER APPS

Super apps such as platforms like WeChat, Grab, World (formerly, Worldcoin), or ChatGPT are applications you install typically on a mobile device that can install other apps inside, called *mini apps* (or just apps, as with ChatGPT), and they act as operating systems within an operating system. Super apps provide an app store, messaging, payments, shopping, ride-hailing, and more, all bundled in a single environment. Under the hood, most of them rely heavily on embedded web views to deliver mini apps.

The same HTML/CSS/JS stack you use for the open Web is also powering mini apps inside closed ecosystems. You need to review the super app's documentation to understand what additional information is required beyond the Vanilla Web app, which typically involves a configuration manifest file and metadata such as the app's icon and permissions.

NOTE For Vanilla developers, the lesson is clear: the web platform is not just for "websites." Your code may end up running inside a super app, as part of a mini app ecosystem, or wrapped by an AI assistant. Designing with web standards maximizes your reach. Even if the host platform imposes limitations, a standards-compliant app will be easier to port, test, and maintain across all these contexts.

2.1.2 Network protocols

The web platform is more than rendering engines, it's a network-first environment. Every Vanilla project interacts with the network in some way, even if it's just serving static assets. Today, HTTP 3 over SSL (HTTPS) is the default, but developers also have access to low-latency and peer-to-peer protocols like WebSockets and WebRTC. Choosing the proper protocol is part of your architecture: a blog doesn't need WebRTC, but a collaborative whiteboard app might. The good news is that you don't need extra libraries to use any of them, the APIs are native.

The most important design principle of Web architecture is that protocols are additive, not exclusive. A Vanilla project can serve static HTML over HTTPS, fetch live updates over WebSockets, and establish peer-to-peer calls over WebRTC all within the same app. These are not "advanced" or "framework" features. They are baseline APIs that browsers expose directly, and you can reach them with just a few lines of JavaScript.

DEFINITIONS

When working later with Web APIs, it's important to clarify some network-based concepts that may later rule security or restrictions over a *URL (Unique Resource Location)*, the standard way we use on the Web to identify resources and page navigations:

- *Protocol*: The communication scheme used in the URL, such as https, http, or ws.
- *Host*: The domain name or IP address identifying the server, e.g., example.com.
- *Port*: The numeric endpoint on the host used for communication (default 80 for HTTP, 443 for HTTPS).
- *Origin*: Combination of protocol+host+port that defines a unique security boundary.
- *Cross-origin*: Any request or resource access that goes to a different origin than the current one.
- *Site*: The registrable domain plus its public suffix ignoring subdomains, e.g., example.com and www.example.com belong to the same site.
- *Cross-site*: Any navigation or request that goes to a different site, which is relevant for modern browser security policies.
- *Path*: The hierarchical part of the URL that points to a resource, e.g., /products/item1.
- *Hash*: The fragment identifier in a URL after #, used for in-page navigation or state.
- *Query*: The set of key/value parameters in a URL after ?, e.g. ?id=1&sort=asc.
- *Text Fragment*: A URL fragment starting with #::~text= that highlights specific text in a page.
- *Scope*: Combination of origin+path defining the boundaries where a web app (like a PWA) is valid.

Let's take two examples of URL decomposition:

For the sample URL <https://firt.dev/notes/pwa#android>

- Protocol: HTTPS
- Host: firt.dev
- Port: 443 (implicit)
- Origin: <https://firt.dev:443>
- Path: /notes/pwa
- Hash: Android
- Query: (empty)
- Text Fragment: (empty)
- Scope: <https://firt.dev:443/notes/pwa>

For the sample URL <http://localhost:8080/webapp?section-3#::~text=:price>

- Protocol: HTTP
- Host: localhost
- Port: 8080
- Origin: <http://localhost:8080>
- Path: /webapp

- Hash: (empty)
- Query: section-3
- Text Fragment: price
- Scope: <http://localhost:8080/webapp>

This information will be helpful later to understand Web APIs' abilities and other specifications.

HTTP

HTTP is the backbone of the Web. Every resource (HTML, CSS, JS, images, fonts, APIs) travels over it. When you fetch data with `fetch()` or submit a form, you are speaking HTTP. Modern browsers enforce HTTP over SSL (HTTPS) as a baseline: most APIs (like geolocation, camera, push notifications) only work on secure origins. That means your Vanilla project should always run under HTTPS. There is one exception: for development purposes you can use localhost on HTTP with no SSL, and the browser will treat it as a secure origin from a security and API availability point of view.

Every request your website makes, from the initial HTML document load to all the resources, will use HTTP (from version 1.0 to 3, based on your server's capabilities).

You can easily access HTTP through the fetch API in JavaScript, as in:

Listing 2.1 A simple fetch API sample in JavaScript

```

async function sendData() {
  const data = { name: "Alice", email: "alice@example.com" };

  try {
    const res = await *fetch*('/api/submit', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(data)
    });

    if (!res.ok) {
      throw new Error(`HTTP error! ${res.status}`);
    }

    const result = await res.json();
    console.log("Server response:", result);

  } catch (err) {
    console.error("Request failed:", err);
  }
}

```

Beyond `GET` and `POST`, HTTP gives you headers, status codes, caching, content negotiation, and compression. You don't need Axios or another client library to use it. Vanilla JavaScript with the Fetch API covers most use cases. If you understand how to structure requests, work with JSON, and handle status codes, you are already fluent in the Web's native protocol.

WEBSOCKETS

WebSockets enable persistent, full-duplex communication between client and server. Unlike HTTP, which is request/response, a WebSocket connection stays open and allows both sides to send messages at any time. WebSockets are the backbone of real-time apps, including chat systems, stock tickers, and multiplayer games. In Vanilla JS, the API is as simple as.

```

const ws = new WebSocket("wss://echo.websocket.org");

ws.onopen = () => ws.send("Hello WebSocket!");
ws.onmessage = e => console.log("Received:", e.data);

```

For a developer coming from frameworks, it's worth emphasizing: you don't need Socket.io or another wrapper unless you want cross-protocol fallbacks for legacy browsers. Native WebSockets are fast, secure, and available in every modern browser. If your project requires live updates, start with the native API first, only add layers if you absolutely need them.

WARNING WebSockets may not be suitable for applications with a large number of users, as they require significant server resources per connected user. For a large number of users, you have to check other solutions.

WEBRTC

WebRTC (Web Real-Time Communication) is the standard way to establish direct peer-to-peer connections between browsers or apps. It powers video calls, file transfers, and even IoT scenarios without needing an intermediary server for the actual data stream (though signaling servers are required to set up the connection). WebRTC is not a "framework feature", it's part of the browser.

The API can look intimidating at first, but at its core, you create `RTCPeerConnection`, add media or data channels, and exchange session descriptions. Once the connection is set up, the data flows directly. For Vanilla developers, the key is that you can build a Zoom-like experience with just HTML, CSS, and JavaScript, no extra SDKs required. Frameworks only abstract the setup; the power is already in the platform.

NOTE While WebRTC is a peer-to-peer connection, it needs a signaling channel over a centralized server.

OTHER PROTOCOLS AND STANDARDS

Beyond HTTP, WebSockets, and WebRTC, the web platform supports a growing set of protocols. WebTransport is an experimental API that provides reliable and unreliable transport over HTTP/3. WebBluetooth, WebUSB, and WebSerial allow direct communication with hardware. WebShare and WebShareTarget connect your app with the OS-level share sheet. Even background synchronization and push notifications are standardized.

In the new AI era, new protocols have arisen that we may also want to implement, such as *MCP (Model Context Protocol)*, which could allow an AI tool to access our data and interact with our web solution.

For a Vanilla developer, the key lesson is: the browser is not "just a document viewer." It's a universal client. Many tasks that used to require native SDKs can now be done with direct APIs. You don't need plugins, polyfills, or frameworks. Just use feature detection, progressive enhancement, and the courage to use what's already built in.

2.1.3 Web security

Security on the Web is not optional; every app, no matter how small, is exposed to threats. The good news is that the platform itself gives us strong security foundations, defined by standards and enforced by browsers. Unlike native apps, which have direct access to the operating system, web apps always run inside a sandbox that isolates them from the device and other websites. This design is one of the main reasons the Web has scaled safely to billions of users.

The most important rule is the *Same-Origin Policy (SOP)*: a page can only freely access resources from the exact origin. This spec prevents malicious pages from reading sensitive data from other sites.

When cross-origin communication is needed, modern standards define safe mechanisms:

- CORS (Cross-Origin Resource Sharing): a server declares which other origins can request its data.
- CSP (Content Security Policy): headers control which scripts, styles, or connections can be loaded, protecting against code injection and XSS.
- Secure Contexts: many APIs, such as geolocation, camera, service workers, or Web Bluetooth, only work over HTTPS, ensuring encrypted communication.
- Trusted Types, COOP/COEP: newer specifications to mitigate DOM-based injections and cross-document leaks by enforcing stricter defaults.

NOTE You can read more about these security-related specs at MDN on <https://developer.mozilla.org/docs/Web/Security>.

As a summary, the Table 2.1 gives you an idea of which security specification to use for each case.

Table 2.1 Security specification overview

Mechanism	What it enforces / solves	When to use	How it's set	Gotchas / notes
SOP	Isolates origins so one site can't freely read/modify another site's data.	Always, it's the browser's default isolation model.	Built-in (no config).	Core to web security; use <code>postMessage</code> or server-mediated APIs for safe cross-origin comms.
CORS	Allows controlled cross-origin reads/writes from JS (<code>fetch/XHR</code>).	Exposing APIs/assets to other origins.	HTTP response headers: <code>Access-Control-Allow-Origin</code> , <code>-Methods</code> , <code>-Headers</code> , <code>-Credentials</code> .	Must be set on the server. Wildcard <code>*</code> + credentials is not allowed.
CSP	Restricts where scripts/styles/images/etc. can load from; mitigates XSS/code injection.	Any app loading third-party content or handling user input.	HTTP header <code>Content-Security-Policy</code> (or <code><meta http-equiv></code>).	Prefer nonces/hashes over host allowlists; test in report-only mode first.
HTTPS (TLS)	Encrypts transport; required for powerful APIs and secure cookies.	Always in production (and most dev flows).	Server certificate + redirect HTTP→HTTPS; use HSTS where appropriate (a spec that forces your domain into HTTPS for every request).	Needed for Service Workers, Geolocation, etc.; localhost is treated as secure for dev.
Permissions	User-mediated access to camera, mic, location, notifications, etc.	Any feature gated by user consent.	JS APIs trigger prompts; query via <code>navigator.permissions</code> .	Request lazily with context; handle denial gracefully; users can revoke anytime.

PERMISSIONS

A *permission model* always mediates access to sensitive features in the browser. For example, if your app wants to use the camera, microphone, geolocation, or notifications, the browser will ask the user explicitly. These prompts are not under your direct control: the API request triggers the UI, and the user decides. Once granted, the permission can persist or be session-based, depending on the feature and the browser. Users can also revoke permissions at any time from their browser settings.

As a developer, you should:

- Request permissions lazily, only when the feature is needed, not on first page load.
- Provide a clear context in your user interface, so users understand why you are requesting access.
- Implement graceful fallbacks when permissions are denied.

Some key principles to apply:

- Always serve your site over HTTPS; many APIs won't work otherwise.
- Treat user input as untrusted; validate and sanitize before using it.
- Use CSP to restrict which scripts and styles can run.
- Apply progressive enhancement: design so that if a permission is denied, the app still works in a reduced mode.
- Rely on browser defaults instead of reinventing security logic in JavaScript.

NOTE Libraries or frameworks cannot make your site inherently more secure. They may add helpers, but the real security model is enforced by the browser and defined by specs. A Vanilla approach means you configure those protections directly, without depending on abstractions that may become outdated.

2.1.4 Application architecture

When we start a Vanilla project, we have to make some initial decisions, including the navigation model, the rendering model, and the user interface engines we will use.

NAVIGATION MODEL

A Vanilla Web project begins with defining the navigation type based on the kind of web solution you need to create.

- *Multi-page application (MPA) or content-based website*: Different sections of the website or web application are rendered by different URLs on the server, allowing for new browser navigation with each interaction. These URLs can be served by static HTML files or dynamic content generated on the fly, based on different rendering models that we will see in the next section.
- *Single-page application (SPA)*: There is a single main navigation point to the application's entry, and all navigation occurs client-side using a custom routing system. In this case, we typically have one initial browser navigation (usually served statically from one HTML file) and, after it is loaded, all the following sections and data are rendered dynamically using client-side code and optional HTTP data or partial code requests. With an SPA, the browser will not make new page navigations, reloading the whole page context.

You can also combine both techniques, such as serving many static sections as an MPA, while managing a particular part of your app as a SPA.

RENDERING MODELS

After the definition of the navigation type, we need to define the rendering model architecture by choosing from the following options:

- *Static Site (SS)*: Typically applicable for simple websites with limited content or infrequent updates, you create one or more HTML files with all the necessary resources without any special architecture. What you code is what is shipped to the server and then to the browser, and the amount of JavaScript involved is minimal. Any web server can host this type of website, as there is no need for any server-side execution.
- *Static Site Generator (SSG)*: it's an evolution of a static site, but for sites with large pieces of content, such as blogs, e-commerce websites, or newspapers. In this case, you can develop your app using templates with any technology and store your content in a database or other information source. At specific periods (regularly or when an update is made), you build the site and obtain a set of HTML files with resource files as output, which are then deployed to any web server.
- *Server-Side Rendering (SSR)*: Most of the logic and content merged with the template occurs server-side in the backend on the user's request. Therefore, you can have a database of articles or products and then use scripts to make queries and fill the content into HTML templates. SSR is the more classic approach on the Web, originating from the 1990s, which is still used today in the majority of websites - mainly on the long tail - thanks to tools such as WordPress.

- *Client-Side Rendering* (CSR) Most of the logic occurs on the client side, in JavaScript, and is optionally enhanced with WebAssembly modules. JavaScript code will make requests to RESTful APIs on the same server or another server for interactivity, content management, authentication, and other purposes. Most communication occurs over HTTPS, but WebRTC and WebSockets are also available for specific purposes. CSR makes heavy usage of the DOM APIs as it's rendering the UI in the client. Most SPAs will fit in this architecture.
- *Incremental Static Regeneration* (ISR): sometimes known as Universal Rendering, Hybrid Rendering, or Rendering with Hydration, it mixes SSR with CSR using a technology that works both client- and server-side, such as JavaScript. Some frameworks, such as Next.js or Angular Universal, use this approach. In this case, you need specific backend technologies on your server that are tightly integrated with the client. ISR's complexity almost necessitates a framework to manage the build and serving process, making it a natural fit for frameworks but a less common pattern for a pure Vanilla approach.

You can see all these options in Table 2.2.

Table 2.2 Rendering models cheat sheet

Rendering model	What you code	What you build and ship	What the server does on request	What the client does
SS	You code raw HTML, CSS, and JS files directly.	None – you ship the files as-is.	Server serves static files (no logic).	browser renders HTML, applies CSS, executes minimal JS.
SSG	You code templates + content sources (Markdown, DB, CMS).	A build step generates static HTML/JS/CSS files.	The server only serves the pre-generated files, similar to SS.	browser renders static pages, with optional JS for interactivity.
SSR	You code templates + server logic (PHP, Node, Python, etc.).	Optional pre-compilation of templates or code. Ship source code or compiled binaries.	On each request, the server queries the DB, merges data with templates, and outputs HTML.	The browser receives ready HTML, CSS, and optional JS, which enhances interactivity.
CSR	You code mostly JS + HTML shell + API endpoints.	Optional tooling. Ship source code.	The server serves static files, similar to SS, and also provides JSON endpoints.	browser runs JS and manages the DOM, fetches data via APIs.
ISR / Hybrid	You code with a framework that can target both client and server.	Build step pre-generates some pages; setup logic for dynamic regeneration.	Server can prerender on demand and cache results.	browser renders pre-generated HTML, then hydrates or updates with JS logic.

You can also create hybrid solutions to deploy web applications by combining techniques, such as integrating SSG with a small CSR solution.

USER INTERFACE ENGINE

When rendering the user interface of your website or web application, the Web platform offers several options:

- HTML, CSS, and JavaScript: this is the most common scenario, and it's ideal for content-based websites and applications. This book focuses on this engine.
- 2D Canvas: When choosing this option, we don't use HTML or CSS for the interface, and we render the user interface in JavaScript in a low-level 2D API. Canvas is commonly used for web games, advanced visualization and charting apps, and for rendering apps created by other platforms, such as DOS or Windows 95 emulators, pure client-side Linux runtimes, or high-performance app engines like Flutter.
- 3D Canvas (WebGL) and eXtended Reality (XR): as with 2D canvas, we don't use HTML and CSS, and we focus on calling a native API that lets us render 3D interfaces directly. WebGL is helpful for 3D games and experiences, as well as for XR experiences, such as VR (Virtual Reality) or AR (Augmented Reality).

2.1.5 HTML, CSS, and JavaScript

Every Vanilla project begins with semantic HTML code, defining the structure, accessibility metadata, and initial data. As we will see in Chapter 6, this layer is not limited to the available HTML elements in the spec, as we can utilize HTML Custom Elements and Web Components, which enable us to encapsulate functionality, design, and behavior in a single, cohesive unit; in short, our own HTML tags.

We will add a CSS layer over the HTML that creates a responsive layout, design, and animation using modern techniques such as Container Queries, CSS Variables, and more. We will utilize a Web UI layer with modern techniques that enable us to customize native UI controls, such as scrollbars, dropdown select boxes, popups, and more, without the need for external JavaScript libraries. We will focus on this layer in Chapters 3 and 4.

A JavaScript initial code that starts an app will typically be inserted using DOM and Browser APIs and then split the logic across multiple files using ECMAScript modules. Based on the project's architecture, the JavaScript code can be used to add basic interactivity, such as validating forms, filling content based on APIs, sending data to the server, or controlling the app's entire logic. We will use JSON and HTTP requests to communicate with the server. This book will focus on the basics of modern JavaScript in chapters 5 and 6.

WEBASSEMBLY

A Vanilla web project can take advantage of native code execution shipped as a WebAssembly module.

WebAssembly (WASM) is a binary format that allows you to run code written in languages like C, Rust, or Go directly in the browser, at near-native performance. It doesn't replace JavaScript, it complements it. You can think of JavaScript as the "glue" and WebAssembly as the "engine." Heavy computation, such as image processing, video encoding, cryptography, or simulations, can run in WASM, while the UI and business logic remain in JS.

Let's say we have a simple code in C that we compile to WebAssembly with tools such as Emscripten or clang.

```
int add(int a, int b) {  
    return a + b;  
}
```

We can call that native module from JavaScript using:

```
async function execute() {  
    const res = await fetch("add.wasm");  
    const { instance } = await WebAssembly.instantiateStreaming(res);  
  
    // Call the exported function  
    console.log("2 + 3 =", instance.exports.add(2, 3));  
}
```

The key point for a Vanilla developer: you don't need a framework to use WebAssembly. The browser gives you the WebAssembly API directly. You can load modules, call exported functions, and pass data back and forth. WASM is especially powerful when paired with Web Workers, moving heavy tasks off the main thread. While you may not use it in every project, it's reassuring to know that if your app needs more performance, the web platform already has a native escape hatch.

WebAssembly is out of the scope of this book, but it's worth noting that it's an option available for every Vanilla project.

TIP Have in mind that JavaScript is always necessary when using WebAssembly, as you don't have access to the DOM or any Web API from native code.

2.2 Design Patterns

As with any piece of software or digital product, it's a good idea to embrace best practices and design patterns that were already discussed and defined in the industry. For a Vanilla project, we need to push this even further, as we won't have a compiler from a framework or a library forcing us into any practice, good or bad.

A *design pattern* is essentially a recipe of steps and ideas to solve a specific problem; it's not an API or a library, and there is no one-size-fits-all approach to implementing it. In our projects, we will be fully in charge. During the rest of this book, we will analyze different design patterns, but let's discuss the first most important ones:

- *Responsive Web Design (RWD)*: it solves the problem of optimizing the layout and design of our solution to different contexts, including media, screen sizes, and form factors. While we won't focus on the RWD theory in this book, we will use it with a modern approach in Chapter 4.
- *Progressive Enhancement (PE)*: it let us design web solutions compatible with a wide range of devices starting with basic compatibility and honoring the idea that not ever device and user will have the same browser, version, device, permissions enabled and API support, adding support of abilities by using feature detection instead of making your solution not compatible. We will see PE in action in many places around this book.
- *Web Components*: it lets us organize small pieces of content of our project in reusable elements with their own encapsulated behaviour and styles, with some communication to and from the outside. We will focus on Web Components in the following chapters.
- *Progressive Web Apps (PWA)*: as mentioned before, it lets us make our web app installable on an operating system, offering a standalone app experience once installed, with optional offline and tight OS integration capabilities.

NOTE You can read more about PWAs in the free licensed course available at <https://firt.dev/learn-pwa>, written by the same author of this book and published by Google.

2.2.1 Web APIs

Web APIs, also known as browser APIs, are the "hidden gems" of the web platform. Beyond HTML and CSS manipulation, modern browsers expose dozens of APIs that cover everything from device sensors to payment flows. You can access geolocation, local storage, IndexedDB, audio/video streams, clipboard, notifications, and even hardware like Bluetooth or USB, all without a framework. Many developers are unaware of the native capabilities available to them because online tutorials and courses often assume a library, such as React or Vue. But the Web APIs are always there.

A Vanilla mindset means exploring the API surface directly. Want to fetch data? Use `fetch()`. Want offline caching? Use the Cache API. Want to measure performance? Use the Performance API. Each of these is just a JavaScript object you can call, no need to import 50kb of helper functions. Once you are comfortable with the APIs, you'll see that many of the "features" frameworks advertise are just wrappers around these same capabilities.

API DEPRECATION ON THE WEB PLATFORM

In the history of Web standards, there have been some minor moments where APIs changed or were deprecated, primarily due to security issues with very old web APIs or code targeting non-standard APIs on outdated browsers, such as Internet Explorer, in the early 2000s. In these rare cases, you had to update your app to make it compatible with modern browsers. However, since the introduction of all new APIs starting in 2015, this situation has been reduced to almost zero. Recently, restrictions on older APIs have been implemented from a security standpoint: they now require HTTPS or are limited to specific security requirements.

STABLE VS. EXPERIMENTAL

Not all APIs are created equal. Some, like Fetch, Web Storage, or Geolocation, are stable and universally available. Others start as experimental APIs, like the File System Access API, the AI Prompt API or WebNN (Neural Network), available initially only behind flags or in specific browsers. Over time, some of these experimental APIs have evolved into standards and are now safe to use everywhere.

The strategy is simple: build on stable APIs, and progressively enhance with experimental ones. Avoid locking your entire app to an API that works only in one browser, unless you can gracefully degrade. Feature detection is your friend: a simple `if` is still the canonical pattern. Using this approach is not extra work, it's the essence of future-proof Vanilla development.

Let's see a simple example checking the Permissions API availability:

```
if ('permissions' in navigator) {  
  navigator.permissions.query({name: "geolocation"})  
    .then(console.log);  
}
```

NOTE As a Vanilla developer, you need to know which category your chosen APIs fall into, such as stable and widely available, stable only on one platform or experimental. This is where MDN, CanIUse, and the Baseline project come in handy.

BASELINE PROJECT

While we will focus in this book on standard technologies and specs that are compatible with all the latest web browsing engines (including web browsers, AI agent browsers, and Web views), the web platform evolves every year, so we will also understand how to deal with technologies that may need fallbacks or by designing using a Progressive Enhancement approach.

In this book, we will use compatibility data based on the Baseline project. *Baseline* is a project from the WebDX Community Group (<https://web-platform-dx.github.io/web-features>) that provides clear information about which web platform features are supported across our core browser set today.

If the features you will use are all part of Baseline, you can trust the level of browser compatibility. There is a new version of the Baseline list every year, so when a feature is included as 'Widely available' in Baseline 2024, it means it was available on all major browsers from that year, including Safari, Chrome, Edge, and Firefox.

Baseline is a multi-vendor effort to simplify the information available to web developers about the safety of using a particular feature, thereby avoiding compatibility issues between different providers.

Features in Baseline include HTML elements, CSS declarations, JavaScript syntax, and browser APIs. Every year, they are organized into three categories as illustrated in MDN Web Docs in Figure 2.5.

The baseline status of every feature can be viewed on sites such as <https://caniuse.com> and the MDN Web Docs at <https://developer.mozilla.org/>.

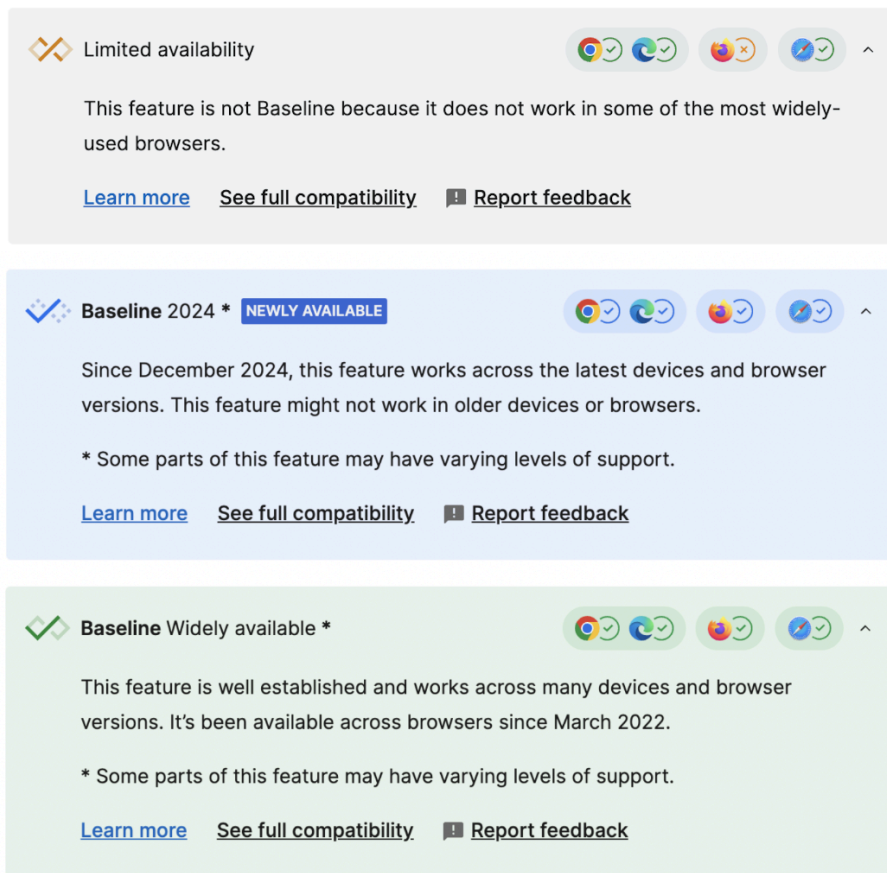


Figure 2.5 How different Baseline status is shown in MDN Web Docs when browsing one specific feature of the API.

2.3 The power of Vanilla Web

The real power of Vanilla Web is not in doing less, but in doing more with fewer layers. Every time you remove a dependency from your code, you gain control: over performance, over security, over long-term maintenance. When you stop outsourcing logic to a framework, you rediscover that the platform already gives you what you need.

NOTE This shift is not about being a purist or a retro-lover; it's about being pragmatic. Why ship megabytes of abstractions when the browser already speaks the language you need?

For developers, this means your mental model gets simpler: HTML defines structure, CSS defines presentation, JS defines behavior. That's it. No build pipeline, no magic runtime, no black box diffing engine. This simplicity is liberating, and it makes your app more robust in the long run.

NOTE As we'll see later in this book, a Vanilla project can utilize micro-libraries (like Alpine) or, in specific cases, meta-tools like JSX.

2.3.1 Web performance

A Vanilla project will many times have better performance in terms of coding, deploying, and, more importantly, execution in users' browsers and rendering engines.

Web performance is a key metric for increasing user conversions and resource efficiency.

JAVASCRIPT FRAMEWORKS VS VANILLA BENCHMARKS

A well-known community benchmark, Stefan Krause's JS Framework Benchmark (available at <https://github.com/krausest/js-framework-benchmark>), tests everyday UI operations (such as creating thousands of rows, updating them, and swapping) across various frameworks and a vanilla implementation. In recent runs, a vanilla DOM implementation is typically at or near the top in terms of speed. One report noted that a new framework (Deku) was "only slower on average (by a hair) than Vanilla JS and Solid JS" and significantly faster than most other frameworks.

Performance results indicate that Vanilla JavaScript still sets the bar for maximum UI update performance – frameworks that are highly optimized, such as Solid, can match it in some operations. Still, mainstream frameworks (React, Angular, Vue) generally lag behind Vanilla in these raw benchmarks. For example, manipulating 10,000 table rows: a vanilla implementation can do it with minimal overhead beyond the DOM itself, whereas frameworks incur costs for diffing and reconciliation.

WARNING To achieve good performance, Vanilla projects must utilize effective design patterns and best practices while understanding the fundamental concepts of web performance.

WEB VITALS

Web Vitals are Google’s recommended set of metrics for measuring real-world performance. They have a subset known as the Core Web Vitals, which are the most important things to see initially for improving performance: Largest Contentful Paint (LCP), Interaction to Next Paint (INP), and Cumulative Layout Shift (CLS). They capture loading speed, visual stability, and responsiveness; the pattern also gives us goals and values to achieve, as we can see in Figure 2.6. For Vanilla developers, the good news is that you can optimize these directly. No framework overhead, no hydration delays, no hidden JavaScript bundles blocking the main thread.



Figure 2.6 With Core Web Vitals, we can quickly analyze the health of our web performance by checking just three metrics and where our values fit between good, needs improvement, or poor.

A well-structured Vanilla site with semantic HTML, optimized images, and minimal JS often scores higher out of the box than a framework-heavy site. You can measure Web Vitals with the `web-vitals` library, Chrome DevTools, or even the Performance Observer API natively. The takeaway: Vanilla isn’t just simpler to build, it’s naturally aligned with the performance metrics that matter for users and search engines.

To read more about it, <https://web.dev/vitals>

2.3.2 Accessibility

Accessibility (usually expressed with the acronym `a11y`) is not optional. Screen readers, voice control, and AI agents all benefit when your markup is meaningful. By contrast, many frameworks encourage `div` soup that requires bolting accessibility on later. A Vanilla approach makes it easier to get right because you’re working closer to the platform. Semantic HTML elements (`<button>`, `<nav>`, `<label>`) come with accessibility baked in. ARIA attributes extend them when needed.

For a developer used to frameworks, the shift is refreshing: you don't need an extra library to make a button accessible, you just use `<button>`. You don't need a plugin for focus management, the browser handles it when you use the right elements. The discipline involves choosing the correct component for the job and testing it with fundamental accessibility tools. In a Vanilla project, a11y is not an afterthought, it's the default.

We'll be taking an accessibility first approach during this book, covering all the topics that are relevant to this matter as we expand on new ideas.

2.3.3 Distribution

One of the biggest superpowers of the web is distribution. You don't need an app store, a review process, or a signed binary. You need a URL. That's it. Share a link, and your app is live on every device with a browser. For users, that means instant access. For developers, it means frictionless deployment. You can also deep link into specific parts of your app easily with URLs and Text Fragments. A Vanilla project maximizes this because there are no proprietary dependencies or extra build steps to block you. AI coding tools are rapidly distributing new apps, as evident in Figure 2.7 from V0, available at <https://v0.app>.

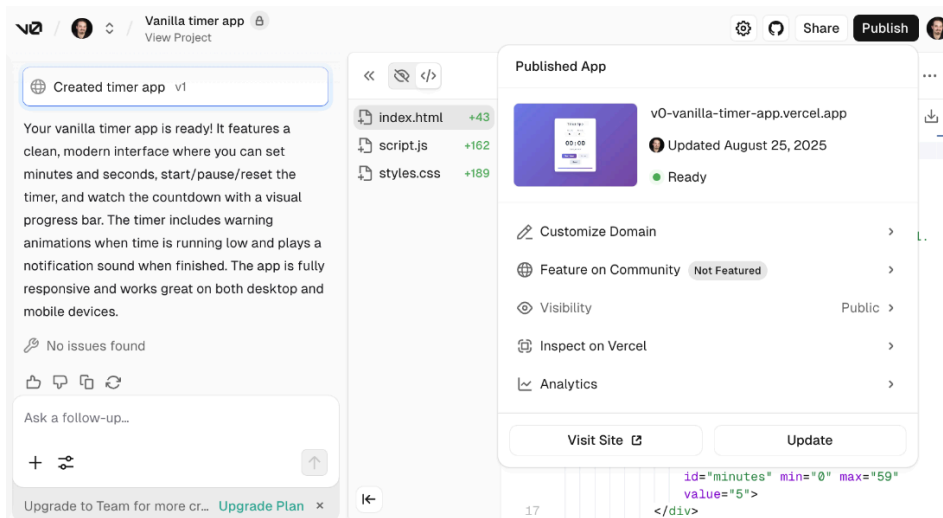


Figure 2.7 AI-driven coding tools are leveraging the Web platform as their primary app distribution channel. Here we can see how in just a few seconds, Vercel's v0 can create and deploy a Vanilla app ready to use.

URLs can also be shared in richer ways: QR codes for physical spaces, NFC tags, or deep links in native apps. Because the Web is universal, your Vanilla app can be discovered from anywhere. When designing your architecture, remember that the URL is part of your UX. It is clean, readable URLs are easier to trust and share.

2.3.4 Discardability

One of the unique advantages of the Web is *discardability*. A user can open your app, try it, and close it without friction. No forced install, no uninstall, no leftover files. This behavior reduces the commitment barrier and increases adoption. For developers, it also means you can ship experiments quickly: if a feature doesn't work, you remove it, and the next refresh delivers the change instantly.

The flip side is that you need to design for ephemeral usage. Assume users may not come back, or may come back from a different device. That makes fast loading, stateless design, and progressive enhancement even more critical. Discardability is not a weakness, it's a feature that makes the Web the most user-friendly platform in existence.

Discardability is becoming increasingly critical as vibe coding and AI tools enable the creation and publication of web apps in mere seconds.

NOTE Paul Kinlan, Chrome and the Open Web Developer Relations team at Google, wrote about this in his blog in an article titled "The disposable web" that you can read at <https://paul.kinlan.me/the-disposable-web>.

2.4 Modern web apps

Modern web apps are not just documents with links. They can be installable, offline-capable, data-driven, and hardware-integrated. The key message is that you don't need a framework to build one. When using Vanilla projects, you have a lot of modern capabilities out of the box.

2.4.1 Installability

Installability is one of the prominent features of the design pattern previously mentioned as Progressive Web App.

Installability comes from the Web App Manifest. It is a simple JSON file that describes your app's name, icons, colors, and start URL. With a manifest and a secure origin, the browser can offer users an "Install" button, turning your website into an app on the home screen or desktop. Installability is not theory. It is supported by Chrome, Safari, and Edge today. No frameworks required. No compilation required.

Some browsers now support installability without the Web App Manifest file, reverting to default configuration, as seen with Safari on every Apple platform in Figure [2.8](#).

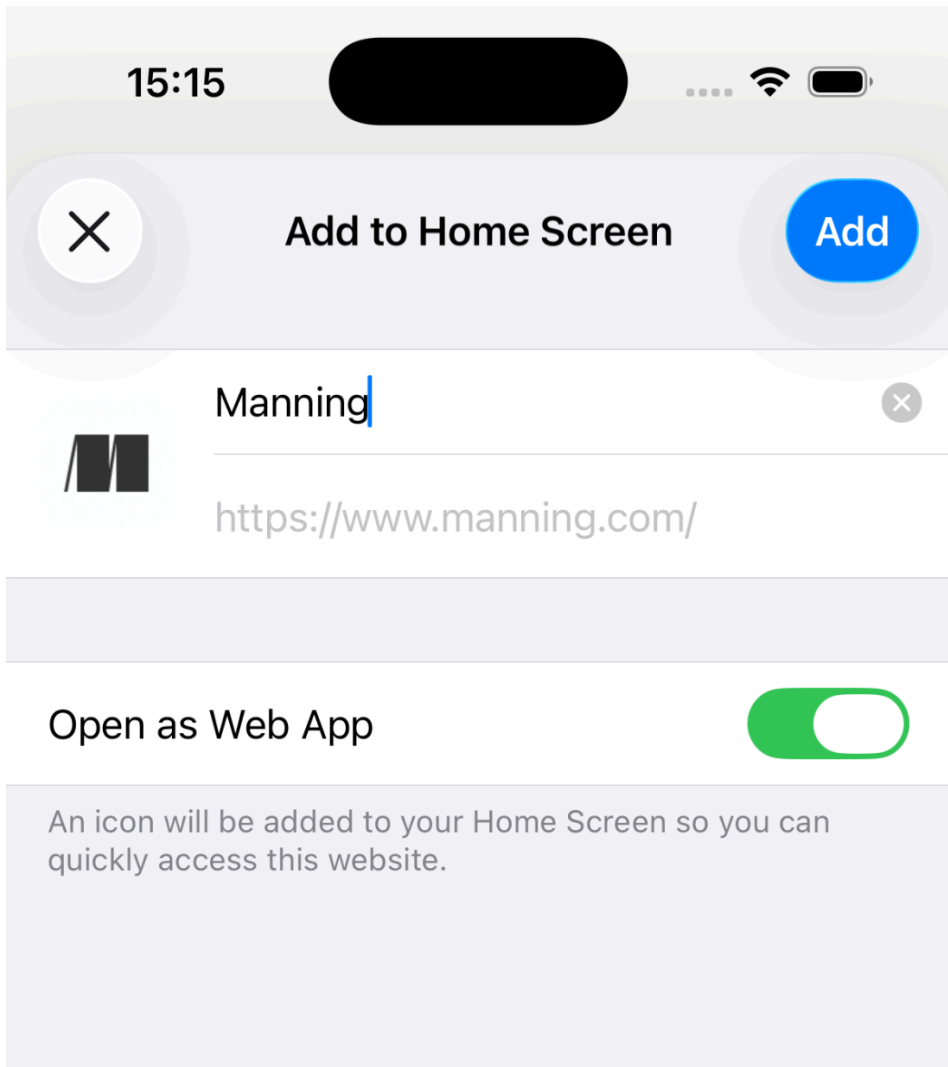


Figure 2.8 When you use Safari on iOS and iPadOS and use "Add to Home Screen," it will ask you if you want to open that icon as a web app once installed, enabled by default. On macOS, "Add to Dock" has the same behavior on any website you visit.

For Vanilla developers, the manifest is a low-effort, high-impact addition. It makes your app feel like a first-class citizen of the OS. You can customize splash screens, orientation, theme colors, and more. Combined with Service Workers, it allows for a native-like launch experience, all while staying 100% web.

We will get into all these details in Chapter 3.

2.4.2 Offline support

Offline support is powered by *Service Workers*, a background script that intercepts network requests. With them, you can cache assets, serve fallback pages, and even sync data when the connection comes back. The API is low-level, but flexible: you decide which resources to cache and how to update them. For a Vanilla project, this means you can start small (cache static assets) and grow into more complex strategies (background sync, stale-while-revalidate).

A Service Worker is a background thread registered by a website, enabling it to control the application later. It includes syncing data in the background, receiving and parsing push messages even when the web app is closed, or acting as a local web server to serve files for the web app.

Service Worker installations are happening today on millions of devices without the need for any user's permission. To test, open your usual Chromium browser (such as Opera, Edge, or Chrome) and type in the URL `about://serviceworker-internals`. You will find hundreds of little web servers or proxies already installed on your computer, ready to serve some web apps, even if you never installed them.

The key is to treat offline as part of the UX. Users may be on a train, in a rural area, or behind a firewall. A Vanilla app that fails gracefully when offline is more trustworthy than one that breaks. Frameworks may generate Service Worker templates, but you don't need them. Writing your own gives you complete control and understanding of how your app behaves offline.

To register a service worker, you call the right API, as in:

```
if ('serviceWorker' in navigator) {  
  navigator.serviceWorker.register('/sw.js');  
}
```

And then, that file will include the server logic to serve files, as we will see later in this book.

2.4.3 Data Storage

The web offers multiple layers of storage:

- Local Storage and Session Storage for small key/value pairs.
- IndexedDB for structured data and larger datasets.
- Cache API for HTTP request/response pairs, usually functional with Service Workers.
- File System Access API (where supported) for working with user files.

You can combine these as needed. For example, store app settings in Local Storage, cache API responses in the Cache API, and keep user-generated data in IndexedDB. All features are available directly in the browser, with no libraries required. Browsers offer developer tools to debug your storage as seen in Figure 2.9.

WARNING Be mindful of quotas and eviction policies. Browsers may clear storage under pressure. The Storage API will help you with this matter. Always design for sync or recovery. If the data is critical, keep a copy server-side.

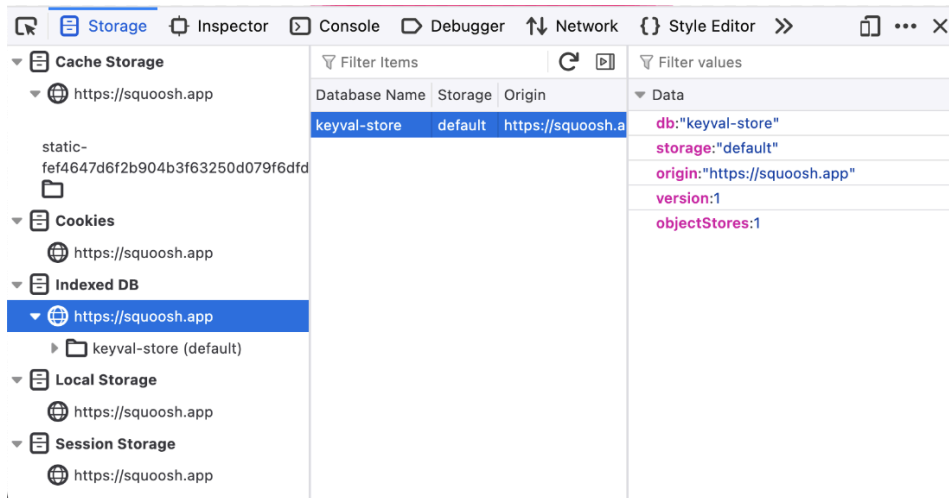


Figure 2.9 Every browser developer tool has a Storage section where you can view and debug client-side data stored from your Vanilla app. Here we see the Storage section in Firefox.

We'll use some of these APIs later in this book.

2.4.4 Internationalization

The Web is global by default. Vanilla JavaScript gives you the `Intl` object, a full suite of internationalization APIs: number formatting, date/time formatting, collation, plural rules, relative time, and more. You don't need Moment.js or i18n libraries for most tasks, the platform already covers them. Combined with Unicode-aware HTML and CSS, your app can be multilingual without heavy dependencies.

Here you have a quick sample on how to use the Intl API.

Listing 2.2 Internationalization API sample

```
const price = new Intl.NumberFormat('en-US', {      #1
  style: 'currency', #1
  currency: 'USD' #1
}).format(1234.5); #1

const date = new Intl.DateTimeFormat('es-AR', {      #2
  dateStyle: 'full' #2
}).format(new Date()); #2
```

#1 Format a number as a currency for American English

#2 Format a date for Argentinian Spanish

The challenge is organizational, not technical: preparing translations, managing locales, and testing RTL layouts. But the API surface is already there. A Vanilla developer who embraces `Intl` gains global reach with minimal effort.

CONTENT TRANSLATION

The Web platform does not include an API for automatic translation of text between languages. The `Intl` family of APIs provides localization features, such as number, date, time, list, and plural formatting, but not translation. If your project requires multilingual content, you need to supply your own translation data (for example, JSON files with strings) or connect to an external translation service (such as Google Translate or DeepL). A typical Vanilla pattern is to combine a translation dictionary with `Intl.PluralRules` and other `Intl` objects to handle pluralization and locale-sensitive formatting. On some browsers, such as Google Chrome, you also have access to the built-in Translation API that uses a local Gemini Nano AI model to translate text. You can also use it to translate the content in the DOM dynamically, client-side, as an option.

2.4.5 Capabilities API

Modern browsers expose APIs to access cameras, microphones, GPS, accelerometers, gyroscopes, Bluetooth, NFC, and even USB devices. These are gated behind permissions for security, but they are standard APIs, not add-ons. With Vanilla JS, you can build apps that record video, scan QR codes, track steps, or connect to IoT hardware.

The key lesson is that the gap between "native apps" and "web apps" has shrunk dramatically. If you think a feature requires native code, check MDN (<https://developer.mozilla.org>) first, it might already be a web API. You can combine these APIs with Service Workers and installability to create full-featured apps that live entirely in the browser.

NOTE To see a list of Capabilities APIs and read more about them, check <https://web.dev/learn/pwa/capabilities>.

2.4.6 Client-side AI

AI is no longer server-only. With WebGPU, WebAssembly, WebNN, built-in AI models, and micro-libraries like TensorFlow.js, you can run machine learning models directly in the browser. Client-side AI enables tasks such as image classification, speech recognition, and text generation to occur client-side, eliminating the need to send data to the cloud and pay for tokens used. Privacy-sensitive apps especially benefit from this.

For Vanilla developers, the key is to see AI as "just another API surface." You can load a model, run inference, and display results, no special framework required. As browser vendors integrate AI acceleration, expect client-side AI to become as normal as using Canvas or WebRTC.

For example, you can use the Summarizer high-level API that, in Chrome, uses Gemini Nano, if available:

Listing 2.3 Using the built-in AI Summarizer API

```
async function summarize(longText) {
  if ('Summarizer' in self) {
    const availability = await Summarizer.availability();
    if (availability === 'readily'
        || availability === 'downloadable') {
      const summarizer = await Summarizer.create({
        type: 'key-points',
        format: 'plain-text',
        length: 'short',
      });
      const summary = await summarizer.summarize(
        longText,
        { context: 'Summary for busy developers.' }
      );
      console.log(summary);
    } else {
      console.warn('Summarizer API not supported or ready.');
```

On some devices, you may also have the low-level Prompt API available that will let you send any prompt to the built-in AI model:

Listing 2.4 Using the built-in AI Prompt API

```

async function explain() {
  if ('LanguageModel' in globalThis) {
    const session = await LanguageModel.create({
      initialPrompts: [
        {
          role: 'system',
          content: 'You are a helpful and friendly assistant.'
        }
      ]
    });

    const result = await session.prompt(
      "Explain Vanilla JavaScript in one sentence."
    );
    console.log(result);

    /* One possible result: Vanilla JavaScript is plain JavaScript code
    without any external libraries or frameworks, allowing you to
    directly interact with the Document Object Model (DOM) to manipulate
    web page content and behavior. */
  }
}

```

These APIs are still experimental at the time of publishing this week and work only on Google Chrome, but they are proposed as standards so that they may become Baseline at some point.

2.5 Developer experience

There is no specific set of tools required for a Vanilla Web project, so your favorite suite will work perfectly. Options include Visual Studio Code with an npm-based local web server, an agentic AI code editor in the cloud like Cloud Code or Firebase Studio, or a classic WebStorm setup with an Apache server. You can also use Notepad or any other basic text editor.

Most AI chat apps, such as ChatGPT, Gemini, Grok, and Claude, offer a direct way to create and test Vanilla Web apps directly in the chatbot user interface because it's easy and standard; they use a web view, as you can see in Figure [2.10](#).

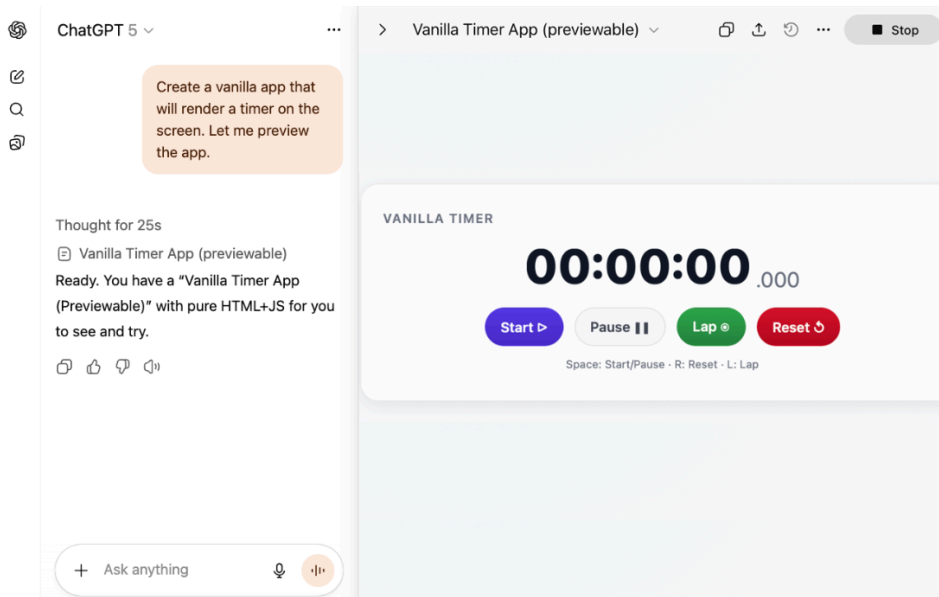


Figure 2.10 ChatGPT lets you preview and use a recently created Vanilla Web app directly in the chat interface.

If you are coming from a framework, you may be used to testing your app in a local server, executing a command such as `npm run dev`. This feature is not immediately available on a Vanilla web. Still, it can be easily emulated using npm or other tools that may vary across different backend architectures, including live-reload solutions that will let you change the code and get it automatically updated.

Because no build tools are needed, when you run and test your app in the local browser through a local server, it is simpler to use the browser developer tools as editors, such as the Workspace Chromium's DevTools, as seen in Figure 2.11. That creates a faster cycle of code-test-debug-code using one tool without ever leaving your browser.

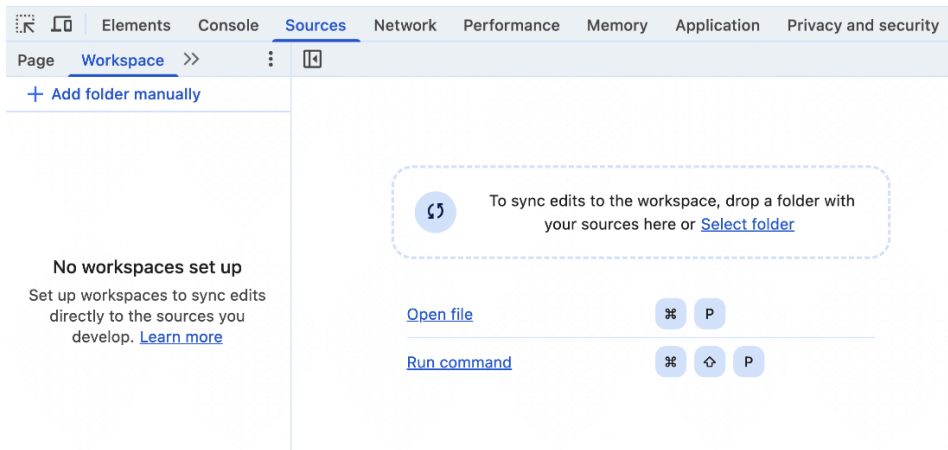


Figure 2.11 In Chromium's DevTools, you can define your Workspace in Sources to edit your source code directly from the browser, eliminating the need for an IDE or code editor.

A Vanilla Web project doesn't need any build tools, so your source code can be deployed without any change. That doesn't mean there aren't some lightweight toolchains that you may want to apply, which may include linters, compression tools, bundlers, and transpilers.

Some tools available for Vanilla Web projects are:

- ESLint: static analysis and code-quality checks.
- Prettier: consistent formatting for HTML, CSS, and JavaScript (<https://prettier.io/>).
- Lighthouse CI: automated performance, SEO, and accessibility audits (<https://github.com/GoogleChrome/lighthouse-ci>).
- Axe Core: automated accessibility testing (<https://www.deque.com/axe/>).
- WebHint: best-practice analyzer for compatibility, performance, and a11y (<https://webhint.io/>).
- Playwright: end-to-end browser automation and testing (<https://playwright.dev/>).
- GitHub Actions: CI/CD for building, testing, and deploying web projects (<https://github.com/features/actions>).

2.5.1 Simple and more secure organization

With most frameworks or libraries, a predefined structure is provided by the framework's rules or the community's best practices. That structure typically involves several configuration files, numerous folders containing source code, and hundreds of files that organize your templates, logic, controllers, styles, and more. When working with large projects, you may also end up with several projects that clone the structure, including import and export declarations.

Even when you are the best professional in organizing projects, that means your source code ends with hundreds or even thousands of files. While this complexity is understandable for massive projects, it's unnecessary and a waste of resources on many others.

WARNING There have been some issues in the past with libraries' dependencies that have affected thousands of projects worldwide, exposing security problems or even making them inaccessible or preventing them from being built at all because a single dependency in the NPM repository shipped malicious code or a bug. When the dependency tree is so big, you lose control of how much external code is merged with your code, which may cause problems.

A Vanilla Web project may also have dozens and even hundreds of files, but you will be in control of how you organize your code.

The surface area for bugs and security issues is reduced by eliminating hundreds of dependencies on third-party projects and relying on your codebase and the browser's native implementations. There are no framework upgrades to chase; you aren't forced to refactor your code whenever a major version of a library comes out. The browser's API doesn't suddenly deprecate all your components, and it simplifies audits. Web standards evolve slowly and backward-compatibly, ensuring that a vanilla codebase can run for years with minimal intervention.

WARNING Keeping your code well-organized is mandatory for maintainability; we won't advocate for monolithic code in just a few files. With Vanilla projects, you are responsible for defining your style guidelines.

Building apps using a simpler project structure leads to:

- Better readability
- Faster deployment
- Stronger security and reliability, as there are fewer external factors that you don't control

2.5.2 Interoperability with libraries and frameworks

Vanilla Web doesn't lock you into a particular vendor or ecosystem, which can be valuable for both longevity and flexibility. For instance, if parts of an app truly need a framework later (due to new requirements), they can often be introduced alongside existing vanilla code.

You could have a primarily Vanilla site and then mount a React component in one section without rewriting everything. In contrast, the opposite (introducing Vanilla in a heavily framework-dependent app) might be harder because the app expects everything to be part of its system. Being framework-agnostic also means you can integrate libraries from different sources more freely.

A Vanilla Web app might use a Chart.js graph here, a raw WebGL canvas there, a bit of Alpine.js for one part, etc., picking the best tool for each job with minimal friction. The a la carte flexibility makes Vanilla stand out against the one-size-fits-all nature of big frameworks.

MODERN FRAMEWORKS IMITATING VANILLA

Interestingly, the influence also goes the other way – modern frameworks have been trying to become more "vanilla-like" in their output to solve performance issues. The popularity of server-side rendering (SSR) and static site generation (SSG) (e.g., Next.js, Astro) is about delivering apps that behave more like static HTML pages on initial load, similar to what a vanilla site naturally does. There's also a movement towards using the platform more directly – frameworks like Lit and even newer versions of Angular are aligning with Web Components standards, essentially meeting Vanilla halfway. The existence of these trends validates the Vanilla Web ethos.

2.5.3 Cost reduction

From a business standpoint, a vanilla approach can reduce operational and maintenance costs. No expensive framework licenses (if any apply), fewer dependency updates (less developer time on maintenance), and often cheaper hosting (because you can serve mostly static files over a CDN).

TIP Vanilla projects are especially useful for small companies or indie developers who want to deliver something with minimal overhead.

2.5.4 Reduced Complexity Tax

When using Vanilla in our project, we are reducing what is known as the complexity tax.

You "no longer need Node.js/npm, build steps, or a development server" just to start coding. This ability can speed up onboarding and simplify the project environment. Fewer things can go wrong, which also eliminates entire classes of security and maintenance concerns related to build tools (no vulnerable webpack-dev-server or outdated Babel plugins to worry about).

The building process with frameworks involves merging the final shipped code with dozens, if not hundreds, of dependencies (typically found in `node_modules`). That means the code you are shipping is out of your complete control, adding much complexity and debugging issues when something doesn't work as expected.

Doing a project vanilla-style avoids problems like huge `node_modules` caches (which can eat dozens of gigabytes) and dependency audits where multiple packages demand upgrades. In essence, the "complexity tax" of frontend tooling is minimized, which makes vanilla projects attractive for long-term maintenance: after a few years, a vanilla site will still build and run as it did originally (since it's just HTML/JS), whereas a framework-based project might not even compile without updating the build chain. The near "zero-maintenance" nature of plain web standards is a significant long-term benefit.

As expressed by Joeri Sebrechts on his website plainvanillaweb.com:

The plain vanilla style of web development makes a different choice, trading off a few short-term comforts for long-term benefits like simplicity and being effectively zero-maintenance. This is made possible by today's browser landscape, which offers excellent web standards support. — Joeri Sebrechts

Once built, a vanilla site may not require heavy ongoing maintenance except for adding new features. There are no framework "dependency hell" scenarios and fewer breaking API changes.

SERVER-SIDE SIMPLICITY

Unlike technologies such as Next.js (which tightly integrates frontend and backend rendering), a vanilla approach might have a clear separation: the server provides data (or prerendered pages), and the client JS enhances the UI.

That means you can easily change backend technologies and infrastructure based on how your project scales and how the cloud and backend industry evolve, without being tied to one specific technology.

Vanilla solutions are portable and broadly compatible with any server solution or infrastructure. You can quickly transition from a self-hosting solution to a cloud-based provider to a serverless solution without affecting your frontend solution.

BUILD TOOLS

A Vanilla Web project does not require a build step to exist. You can ship raw HTML, CSS, and JS files directly to a server, and they will work. That is the default and often the best approach. However, build tools can still help in some instances: bundling multiple JS modules into one file for performance, minifying and compressing assets, or generating responsive image sets. The difference in Vanilla is that you add them because you want to, not because the framework forces you to.

Tools like Vite, esbuild, or Rollup can be used in a "lightweight" way (to bundle and optimize) without letting them take over your architecture. The principle is: don't use a build tool to make your app runnable, use it only to make your app faster to load. That distinction keeps you in control. If you remove the build pipeline tomorrow, your app should still run by opening `index.html`.

PACKAGE MANAGERS

Package managers like npm, Yarn, or pnpm are powerful for managing third-party code. Still, a Vanilla project often needs very few (if any) external packages, which reduces complexity and risk. Instead of installing dozens of dependencies, you might only use a linter, a bundler, or a couple of micro-libraries. In many projects, you won't even need a `package.json`.

That doesn't mean you should avoid package managers altogether. They are still helpful for developer tooling or when you do want to pull in a specific library. The key is to treat them as optional helpers, not mandatory infrastructure. Your app should not depend on `npm install` to run. A clean Vanilla repo can be cloned and served as-is without extra steps, and that's a significant advantage.

2.5.5 Deploying the apps

Deploying a Vanilla Web project is straightforward, involving sending files to a web server. Any deployment tool is valid, including publishing code on a Git server connected to a deployment cloud solution or using FTP over SSH.

A Vanilla Web project doesn't have any specific requirements in terms of a server (from a frontend point of view), so the frontend can be published on free hosting solutions, such as Firebase hosting or GitHub Pages.

WARNING Your Vanilla Web project may utilize backend and cloud-specific technologies, but there are no special requirements for being Vanilla.

2.6 Summary

- The web platform is built on rendering engines, browsers, and protocols that power every modern device and context.
- A Vanilla approach uses these building blocks directly without abstractions to create apps that are simpler, faster, and longer-lived.
- Rendering engines (Blink, WebKit, Gecko) drive browsers, PWAs, web views, and even AI agentic browsers; they will be responsible for executing our code and rendering the user interface for us.
- Core protocols like HTTPS, WebSockets, and WebRTC are natively available and cover most communication needs.

- Application architecture is your choice: MPA, SPA, SSR, CSR, SSG, or hybrids, a framework does not constrain you.
- Browser APIs already provide storage, sensors, performance metrics, hardware access, and even client-side AI.
- The Baseline project gives you a clear guide on which features are safe to use across browsers today.
- Vanilla projects naturally align with high performance, accessibility, discoverability, and distribution via simple URLs expressed in links, social networks, QR codes, or NFC tags.
- Modern web apps (installable, offline-capable, data-driven, and hardware-integrated) can all be built with Vanilla.
- Developer experience stays lightweight: you add build tools or package managers only if you need them, not because you must.

3 The User Interface

This chapter covers

- Designing for every surface, screenless agents, SEO, and AI-optimized content.
- The core web metadata that you need on every document.
- Using the semantic and accessible HTML content.
- Prioritizing native rich UI controls with declarative HTML and CSS.

The user interface (UI) is clearly one of the most essential features of your app. It's what the user will see and with the experience they will interact with. When creating Vanilla Web apps, we won't advocate for any specific UI or UX (User Experience). Still, we will encourage the use of the platform to maximize reachability, accessibility, and usability, ensuring future-proofing.

We prefer declarative code for our user interface in Vanilla Web. *Declarative* means describing what you want the UI to be, rather than writing step-by-step instructions for how to build or update it.

HTML and CSS are inherently declarative: you express structure, meaning, and presentation, and the browser handles the logic, state, rendering, accessibility, and edge cases. This matters because declarative interfaces are more resilient, easier to maintain, and far more accessible by default. When you rely on native, declarative controls, you get built-in keyboard support, focus management, semantics, and accessibility behaviors for free, all without hand-crafting fragile JavaScript to imitate what the platform already does well.

3.1 Defining our user interface

We will make every effort to define our user interface as declaratively as possible, avoiding the use of large JavaScript libraries or complex code, and leveraging the current capabilities of web rendering engines to achieve this goal.

The first decision that we have to make is which set of technologies will empower our user interface, the UI stack:

- *HTML and CSS*: this is the most common stack for apps and content, and the one that we will focus on in this book. We define the data and structure using HTML, and the design and layout using CSS. JavaScript can be used to make the content dynamic and responsive to user interaction, if suitable.
- *Canvas*: this is mostly the standard stack on games, multimedia apps, immersive experiences, such as VR or AR, and it involves a `<canvas>` HTML element whose user interface is rendered by JavaScript and/or a web assembly module, in 2D or 3D.

Using the Canvas UI is not a common practice for content-based websites or apps, but some frameworks, such as Flutter, can create Canvas-based apps from Dart code. It's also common to mix both stacks in different ways, with the most common approach using HTML and CSS, and then utilizing Canvas for specific parts of the app, such as when rendering charts or graphics.

Regardless of the UI stack you choose, every website or web application begins with an HTML document and metadata as the container and navigation entry point. Therefore, even if you are making web games or VR experiences, you will need to write a basic HTML document with its metadata.

In the classic UI stack, your document will contain:

- Metadata with external links for web app manifest and icons
- CSS stylesheets external links
- Semantic HTML for structure and data

It may include references to one or more JavaScript modules, as we'll start seeing in the next chapter.

EXPANDING HORIZONS FOR UI

These days, a web application has various ways to interact with the user beyond the display, including web speech, hand gestures, media controls (such as those available in a smartwatch or Android Auto and Apple CarPlay while playing media), and AI agent support. Think about all the possible interaction models when designing your user experience.

3.2 The ultimate responsive design

The key to success is to design our interface to match any possible screen, even screenless architectures. Display sizes can range from as small as smartwatches or smartglasses to folded foldable phones, smartphones, tablets, or dual-monitor desktop computers. When discussing screenless systems, we will also design content and apps for accessible screen readers, social networks, search engines, AI chatbots, and AI agents.

That means we should design our app in a way that each piece of the user interface can adapt itself to any context and form factor.

NOTE Every website can be installed as a standalone web app in the operating system, so we should plan for any screen size in that situation, even smaller than the minimum possible browser window.

It's not enough to list a limited set of screen sizes and form factors as we used to do during the first years of responsive design. These days, your content may appear on many platforms and devices, and we can't predict or test it on all of them. Therefore, the answer is to design interfaces as flexibly as possible. We also need to remember that we are not designing for screens only anymore, as AI agents and AI tools may also be browsing our content to enhance their responses to users.

SEO AND GEO

Search Engine Optimization, also known as classic SEO, is the practice of making your content easier for search engines like Google, Bing, or DuckDuckGo to discover and understand. It typically involves using descriptive metadata, clean URLs, semantic headings, alt text for images, and fast-loading content, allowing crawlers to parse and index your site effectively. Effective SEO ensures that your content appears when users type queries into traditional search engines.

Since 2025, every website and application needs to understand *Generative Engine Optimization* (GEO). GEO is the set of tools and techniques to make your website more accessible and readable for AI tools, such as ChatGPT, Gemini, or Grok chatbots. These agents no longer crawl links; they parse meaning, context, and relationships between pieces of content and then present summarized answers directly to users. If your structure is messy, the bots may ignore or misinterpret your information.

The difference is subtle but essential. Traditional SEO focused on keywords and ranking signals, while GEO focuses on clarity, semantics, and structured data. You are writing not just for humans and search crawlers, but also for AI systems that generate content on top of your site.

Following modern best practices, semantic HTML, correct metadata, accessible forms, and ARIA when needed already gets you most of the way there. These topics are outside the scope of this book. Still, it's essential to note that if you follow the Vanilla Web pattern using semantic HTML, as we will discuss later in this chapter, you will be better optimized for any situation.

What we will always have, no matter what, is at least one HTML document. We need to start setting up our web application there, along with its metadata.

3.3 Web metadata

The *metadata* is the set of information for a website or web application that defines settings and information for the web rendering engine to enhance the visualization experience, without altering the contents of the viewport. It's what allows us to customize elements outside of the HTML document, such as the title bar, the window or tab, the browser's chrome UI, and integration with the underlying operating system. Every web content will need some basic metadata to enhance the experience in every context.

The recommended set of metadata to define is:

- Title
- Web app manifest
- Icons
- Theme color
- Viewport definition
- Semantic metadata

NOTE Some metadata, such as title, icons, and theme color, can be set differently on a multipage app and can change dynamically with JavaScript at any time while in a client-side application.

3.3.1 Title

The `<title>` element names your document for tabs, task switchers, bookmarks, and assistive technologies. Keep it short, descriptive, and unique for each route so users and agents can easily differentiate between views.

If you render server-side, set it per page at build time and still adjust it client-side when state changes.

Single-page apps should update the title on navigation. The following code shows how you can update it using `document.title`.

```

<head>
  <title>Coffee Roasters · Home</title>
  <script>
    // Example for SPA route changes
    function setTitle(section) {
      document.title = `Coffee Roasters · ${section}`;
    }
  </script>
</head>

```

3.3.2 Web app manifest

The Web app manifest is a JSON file that matches the W3C specification with that name and is linked in every HTML document of your website using a `link` element as in:

```
<link rel="manifest" href="app.webmanifest">
```

The browser uses this file to define metadata about your application, including its name, icon set, preferred settings, active color, and more. We will cover more settings about this file later in this book, as it's the heart of a Progressive Web App, but for a basic manifest, we should start with something with these properties:

- *id* is any string that the OS will use to recognize updates as the same app. Different values will mean it's a new app served in the exact origin. If you have only one app in your origin, the most common value for the *id* is `/`.
- *name* is any string that will be used as the name of the application within the OS.
- *short_name* is an optional string that will be used for the name that appears in the installed icon, when *name* is too large.
- *scope* limits navigation that remains inside your installed app, so any navigation to a URL within the scope will be rendered in the app.
- *start_url* is the entry point when the user launches it. You can use querystring arguments to detect that the user is opening it from an installed app.
- *background_color* defines a color used on some platforms (mainly Android) for the launch screen while your app is loading.
- *theme_color* hints to the browser and OS which color to use for UI chrome, like the URL bar or standalone app frame.
- *display* tells the browser your installed icon experience preference, such as `standalone` for a native app experience of `browser` for using a browser's tab when opening the icon.

The following JSON content defines a Web App Manifest file for an app that you can see in action in Figure [3.1](#).

Listing 3.1 A simple Web App Manifest for a Progressive Web App

```
{
  "id": "/",
  "name": "Coffee Roasters",
  "short_name": "Roasters",
  "start_url": "/?source=app",
  "scope": "/",
  "display": "standalone",
  "orientation": "any",
  "background_color": "#ffffff",
  "theme_color": "#996239",
  "icons": [
    { "src": "/icons/icon-512.png", "sizes": "512x512",
      "type": "image/png", "purpose": "any"},
    { "src": "/icons/icon-512-maskable.png", "sizes": "512x512",
      "type": "image/png", "purpose": "maskable"},
    { "src": "/icons/icon.svg", "sizes": "all",
      "type": "image/svg+xml"}
  ]
}
```

Use at least one maskable PNG so that adaptive icon shapes appear correctly on Android and some desktop shells. We will cover adaptive icons in more detail later in this book.

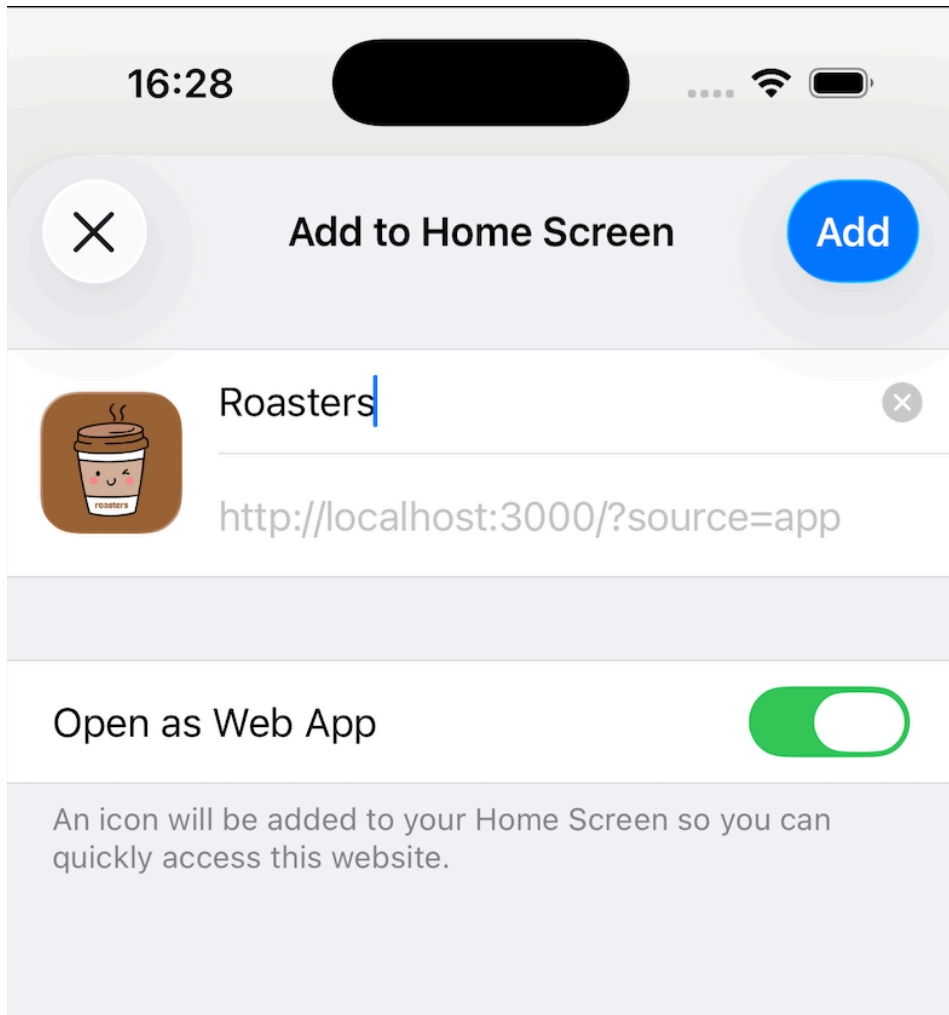


Figure 3.1 When your website defines a manifest, the browser will parse that file and enhance the UX around your content. For example, a browser's toolbar can be styled in different colors based on the manifest file, and the short name and icons are used when the browser is installed or added to the home screen or dock.

3.3.3 Icons

Icons are used in several contexts: the small favicon in tabs, larger icons for pinned shortcuts, and the icon set declared in the Web App Manifest for installation. Provide at least a favicon (favorite icon), PNGs for standard sizes, an Apple Touch icon for home screen shortcuts on iOS, and a mask icon for Safari pinned tabs. The manifest will supply the multi-size set for installation.

WARNING While the canvas of our project can utilize images in recent new formats, such as WebP, icons are typically managed by the operating system, which may still not support these formats. Therefore, it's a good idea to stay with PNG as a bitmap format for icons.

We should prepare a set of icons in SVG and PNG and link them in every HTML document of our Vanilla Web project, as in:

```
<link rel="icon" href="/icons/favicon.svg" sizes="any">
<link rel="icon" type="image/png" sizes="32x32" href="/icons/icon-32.png">
<link rel="apple-touch-icon" href="/icons/apple-touch-icon-180.png">
<link rel="mask-icon" href="/icons/safari-pinned-tab.svg" color="#5d4037">
```

As the minimum set of icons you need to prepare per project, the recommendation is as follows:

- An SVG version, linked in the manifest and in the HTML with `link rel=icon` and `link rel=mask-icon`
- A 32x32 PNG version linked in the HTML with `link rel=icon`.
- A 512x512 PNG version (square, with no rounded corners) linked in the manifest
- A 512x512 maskable PNG version with larger paddings to match Android and other OSs' adaptive icon shape system, linked in the manifest with `purpose=maskable`

NOTE In Chapter 14 we will see other images that we may want to add as metadata, including screenshots and splash screens for installed apps.

Optionally, you can add many other versions to improve the quality of your icons on different contexts, some recommended ones are:

- A 180x180 PNG version for iOS and iPadOS (square, with no rounded corners)
- A high-resolution 1024x1024 PNG icon linked in the Web App Manifest
- An .ico version for backwards compatibility reasons, linked in the HTML with `link rel=icon`

3.3.4 Theme color

The `theme-color` meta tag hints to the browser and OS which color to use for UI chrome, like the URL bar or standalone app frame. Pair it with light and dark preferences, and also set `theme_color` in the manifest so the installed window uses the right tone. To increase compatibility with most browsers and operating systems, stay with CSS named colors, as in `orange` or RGB colors using the `#RRGGBB` syntax. Have in mind the browser will try to honor your preference, but for accessibility reasons it may ignore your hint.

The following HTML code shows how to define a theme color for light and dark themes taking the preference from the operating system.

```
<meta name="theme-color" content="#ffffff"
➔media="(prefers-color-scheme: light)">
<meta name="theme-color" content="#121212"
➔media="(prefers-color-scheme: dark)">
```

The definition of the `theme-color` meta tag overrides the one you set in the Web App Manifest `theme_color` declaration.

WARNING In the Web App Manifest file, the property is `theme_color` (with an underscore), while in the HTML meta element, it's `theme-color` (with a dash).

Some versions of Safari ignore the `theme-color` meta tag and instead use the manifest's `theme_color` property or the page's most prominent background color.

3.3.5 Viewport definition

Mobile browsers lay out pages on a virtual desktop width unless you define the viewport. Use `width=device-width`, `initial-scale=1` to match the device width and avoid unpredictable zoom. Add `viewport-fit=cover` so content can extend under notches, then use safe-area CSS `env()` variables.

The most common viewport definition that extend the viewport under the notches looks like the sample below.

```
<meta name= "viewport"
      content="width=device-width, initial-scale=1, viewport-fit=cover">
```

To decide which resize behavior the browser should use when a virtual keyboard appears on the screen you can set the `interactive-widget` property of the manifest with values such as `resizes-visual`, `resizes-content` or `overlays-content`.

3.3.6 Semantic metadata

Besides the Web App Manifest, there is a set of classic metadata that lives in the `<head>` of an HTML document. These `<meta>` tags are not visible to the user but help browsers, search engines, and agents understand what your page is about. While some attributes are less influential today (for example, `keywords`), they are still harmless and can provide extra hints for crawlers or documentation tools to understand your content.

The most relevant meta tags are:

- *description*: a summary of the page. Search engines often use it as the snippet shown under the title in results.
- *keywords*: a comma-separated list of terms. Search engines no longer rely on it for ranking, but some internal engines and tools still parse it.
- *author*: identifies the author of the document. Useful in corporate, academic, or publishing contexts.
- *generator*: identifies the tool or CMS used to generate the page.
- *robots*: controls indexing and crawling (for example, `noindex`, `nofollow`).
- *charset*: defines the character encoding. Today, UTF-8 is the standard and should always be declared as the very beginning of the document or through an HTTP header alternative.

NOTE Some agents and browsers may also have their own custom meta tags. For example, you can use `<meta name= "googlebot" content= "nottranslate">` to prevent Chrome and other browsers from offering content translation on the whole page.

Even if not all of these tags affect ranking, they improve clarity and documentation. They also serve as signals for AI engines that parse your content semantically, helping them extract summaries, authorship, and categorization.

For semantic metadata we can also cover OpenGraph which lets websites embed rich previews (title, description, image, etc.) when their links are shared on social media or messaging platforms. Facebook created the protocol, which is now widely used by social networks, messaging apps, and even AI agents to display rich previews of links. When you paste a URL in a chat or timeline, the service scrapes the page and looks for these `<meta>` tags. Without them, your link may appear as just a raw URL or with a generic title.

At minimum, to get those rich snippets in your URLs you should provide `og:title`, `og:description`, and `og:image`. The image should be a large rectangular preview (1200x630 is the recommended size) in JPG or PNG format. You can also add `og:type`, `og:url`, and even `article:` meta tags if your page is a blog post or news article. These tags reside within your `<head>` section and do not impact the visible page.

NOTE OpenGraph tags are handy for marketing. A good image and description make your app look professional when shared, and they also help AI agents extract structured meaning from your content.

Adding OpenGraph metadata does not guarantee that every platform will display it the same way, but it increases the chances of a consistent preview. Twitter uses its own `twitter:` tags, but if those are missing, it usually falls back to OpenGraph. Other platforms, such as LinkedIn, WhatsApp, Telegram, and Slack, also rely on it.

3.3.7 Putting the metadata together

Let's create one example of all the metadata we need for a Vanilla Web application that will sell coffee. The HTML container, no matter which UI stack you will use for the inner content, will be:

```
<head>
  <meta charset="utf-8">
  <title>Coffee Roasters</title>

  <meta name="*description*" #1
    content="Order specialty coffee beans online. <lineararrow />
    Freshly roasted, globally sourced."> #1
  <meta name="*keywords*"
    content="coffee,espresso,latte,beans,roast"> #1
  <meta name="*author*"
    content="Vanilla Web Book Inc."> #1

  <meta property="*og:title*" #2
    content="Coffee Roasters: Fresh Beans Online"> #2
  <meta property="*og:description*" #2
    content="Order specialty coffee delivered to <lineararrow />
    your door.Freshly roasted, globally sourced."> #2
  <meta property="*og:image*" #2
    content="https://coffee.example.com/og.jpg"> #2
  <meta property="*og:url*" #2
    content="https://coffee.example.com/"> #2
  <meta property="*og:type*" #2
    content="website"> #2
</head>
```

#1 Classic semantic metadata

#2 OpenGraph metadata

3.4 Semantic HTML

When your UI stack consists of HTML with CSS, your code will start with your HTML markup. And, while HTML has been with us for more than 35 years, it hasn't been used correctly on every occasion. For a Vanilla Web project with a good architecture, we should focus on what is more important for the HTML markup: data and semantics.

Semantic HTML is HTML that uses tags to convey the meaning and structure of content, not just its appearance.

Think of the data as the raw information you want to put on the screen. *Semantics* are what give that information structure and meaning. When you write semantic HTML, you make it possible to read the document and instantly see what matters most, which pieces belong together, which parts stand on their own as complete units, and which ones are just supporting details inside something bigger.

NOTE HTML today is a living specification, which means we no longer version it. The last W3C specification of HTML that was versioned is 5.2, but it was retired in favor of the living version specified by WHATWG, available at <https://html.spec.whatwg.org>.

If you are coming from a library or native app programming, you may think that HTML is insufficient for building apps and that it's only suitable for text-based documents. Let me be clear here: every app on your phone can be expressed in HTML. Opening one app from your phone and trying to think about the markup you should use is a good exercise. Forget about the design, the layout, and the organization of the elements on the screen. Just focus on the structure, what's a title, what's a navigation bar, what's a list of elements, what's clickable, what are the parts of each component, and if they are organized in sections, headers, and footers. All that will become your HTML structure. CSS, later, will make the magic so that it looks as you expect, but don't worry about that for now.

3.4.1 Using the Right Element

While we won't cover HTML from scratch in this book, we'll focus on understanding which elements are the most useful for different situations. Using the correct semantic tag won't have a significant impact on the user interface. Still, it will have a profound effect on usability, accessibility, and discoverability, both in terms of SEO and GEO. It also makes our project easier to maintain.

3.4.2 Avoiding divitis and other problems

During several years, we ended up shipping most websites and web applications without the proper HTML semantics, resulting in code such as the following:

Listing 3.2 A simple markup not following semantic HTML best practices

```
<!-- Bad practice -->
<div id="header">
  <div class="title">Coffee Roasters</div>
</div>
<div class="main content">
<div id="nav">
  <div><a href="/">Home</a></div>
  <div><a href="/products">Products</a></div>
</div>
<div>
</div>
```

The previous code lacks semantics, and most containers are using `<div>`, which is a fallback container that we should use only when no other semantic container seems to be a suitable fit. Because everything is a `div`, we are mostly forced to add a class or an ID to every element on the screen to target them from CSS or JavaScript. This overuse of `<div>` (commonly referred to as *divitis*) makes the document harder to read, maintain, and navigate. Because it doesn't specify any `aria` attributes, by default the screen reader has no way of knowing which parts are headers, which are sections, or what role the different containers play. Search engines and AI crawlers also receive less context, making your content less discoverable. Even developers coming back to the project after a few months will find it painful to understand what each anonymous `<div>` is supposed to mean.

NOTE In Chapter 7, we will explore how to extend the available HTML elements using Custom Elements and Web Components.

The antidote to divitis is to use semantic elements whenever possible. Replace `<div id="header">` with `<header>`, or `<div class="footer">` with `<footer>`. Use `<main>`, `<section>` and `<article>` to group related content whenever they apply, use `<p>`, `<output>` and `<h1>`-`<h6>` elements, and `<nav>` for navigation links. Use lists `<ul>`, `<ol>`, and `<dl>` whenever you have a set of more than one element that are grouped by semantics.

Using the semantic HTML elements not only improves accessibility and SEO but also reduces the need for excessive classes, as the element type already conveys this information. It helps and improves the accuracy of AI agents and tools in understanding and coding your project.

The following example shows how you can use semantic HTML to provide a better markup for the previous markup seen in Listing 3.2 full of `<div>`.

Listing 3.3 A simple markup following semantic HTML best practices

```

<header>
  <h1>Coffee Roasters</h1>
</header>
<main>
<nav>
  <ul>
    <li><a href="/">Home</a></li>
    <li><a href="/products">Products</a></li>
  </ul>
</nav>
</main>

```

Working with forms and input validation is another set of semantic situations that we will cover later in this book.

NOTE While class names are often used for styling, remember they are also part of HTML semantics. They allowed us to categorize elements into categories such as `featured`, `active`, or `secondary`. That information can be reused by CSS, JavaScript, and even automated tools.

3.4.3 Semantic attributes

Every HTML element has a couple of attributes that may be used to add semantic information to the browser, and to CSS and JavaScript, such as:

- *class*: it classifies the element with tags (known as classes)
- *role*: define the explicit semantic role for accessibility when the HTML element used is not sufficient. The list of roles is listed in <https://developer.mozilla.org/docs/Web/Accessibility/ARIA/Reference/Roles>.
- *hidden*: it's a boolean attribute that, if present, makes the element invisible on the screen and layout but still available in the DOM
- *disabled*: for interactive elements, it's a boolean attribute that, if present, makes the aspect read-only and non-interactive.
- *inert*: When present, it tells the browser that the element and all of its subtree are inactive: they are not focusable, not clickable, and will be skipped by assistive technologies.
- *translate*: indicates whether the content of an element should be considered for automatic translation by the browser or external tools (`yes` or `no`).

- *title*: defines a short description for the element. It can be used by the browser (as a tooltip on desktop devices when the pointer hovers over the element) or via CSS and JavaScript.
- *lang*: defines the language of the content
- *dir*: defines the text direction from the language (ltr for left to right, and rtl for right to left)
- *draggable*: indicates whether an element can be dragged using the browser's native drag and drop API.

NOTE Some HTML elements or groups of them (such as form elements) have their own specific set of semantic attributes, including `inputmode` for `<input>`, `download` for `<a>`, or `alt` for the `img` element. These attributes are preferred over using JavaScript or CSS styles that may achieve the same effect. You can also target elements with those attributes in CSS and JavaScript.

3.5 Accessibility

Accessibility (often shortened as *a11y*) is the practice of ensuring that every person, regardless of ability or context, can access and use your website or application. Caring about accessibility is not just about compliance with laws, but about building products that reach the widest possible audience. An accessible product benefits everyone: a keyboard shortcut is useful for power users, while also empowering those who cannot use a mouse.

In a Vanilla Web project, accessibility starts by using the correct HTML elements. If you choose semantic tags (`<button>`, `<header>`, `<nav>`, `<section>`, `<form>`), you already get built-in accessibility support from the browser. Many of the painful problems developers face stem from reinventing widgets using `<div>` and `<span>` elements that have no inherent meaning. The less you fight the platform, the more accessible your app will be.

Accessibility is also about resilience. Users can turn off images, use a zoom level of up to 200%, navigate entirely with the keyboard, or rely on a screen reader to parse the DOM tree. If your interface only works in one context (for example, when a pointer is hovered over), then part of your audience is excluded. The principle of progressive enhancement applies here as well: start with semantic HTML and then layer interactivity with JavaScript.

ARIA (Accessible Rich Internet Applications) fills the gaps when HTML alone is not enough. It allows us to define roles, states, and properties, enabling assistive technologies to understand custom controls. However, ARIA is a sharp tool: used wisely, it enhances accessibility; used recklessly, it breaks it. The golden rule is: don't use ARIA if a semantic element exists for your purpose.

3.5.1 Landmarks

Landmarks help users navigate large documents. Elements like `<header>`, `<main>`, `<nav>`, `<aside>`, and `<footer>` are interpreted by assistive technologies as landmarks. A screen reader user can jump directly to the "main" region or "navigation" region. It provides us with free accessibility simply by using the right HTML element.

```
<body>
  <header>My Website</header>
  <nav>...menu links...</nav>
  <main>
    <h1>Welcome</h1>
    <p>Content goes here...</p>
  </main>
  <footer>Copyright © 2025</footer>
</body>
```

3.5.2 Forms

Forms are one of the most critical interaction points on the Web. Accessibility in forms starts with labels: every input should be explicitly associated with a `<label>` element, either by wrapping the input or using `for/id`. Placeholder text is not a substitute for a label. You should also provide clear error messages and associate them with the correct input via `aria-describedby`.

```
<form>
  <label for="email">Email address</label>
  <input type="email" id="email" required aria-describedby="hint">
  <small id="hint">We'll never share your email.</small>
</form>
```

Forms are often where accessibility breaks down in custom UI kits. If you build your own form controls, be ready to manage focus, labels, and ARIA attributes manually.

3.5.3 Keyboard Navigation

An accessible app must be operable with a keyboard alone. The DOM determines the natural order, but you can refine it with `tabindex`. Avoid removing outlines: they are the only focus indicator many users have. If you design a custom focus style, ensure it's at least as visible as the default. Test your entire app using only `Tab`, `Enter`, `Space`, and arrow keys.

TIP A quick test: unplug your mouse or trackpad for 10 minutes and try to complete all main tasks in your app. If you get stuck, so will your users.

3.5.4 ARIA Roles

Roles are labels that describe the function of an HTML element. For example, a `<div>` can be turned into a `role="dialog"`, `role="alert"`, or `role="button"`. Use them only when there is no native equivalent, as browsers already map most semantic elements to roles.

```
<div role="dialog" aria-labelledby="title1" aria-describedby="content1">
  <h2 id="title1">Confirm action</h2>
  <p id="content1">Are you sure?</p>
  <button>OK</button>
  <button>Cancel</button>
</div>
```

3.5.5 ARIA states and properties

States communicate dynamic changes in the UI. For example, collapsible menus can use `aria-expanded="true"` or `aria-expanded="false"` to indicate whether they are open. Properties such as `aria-label` or `aria-describedby` provide extra context. Update these attributes dynamically using JavaScript whenever the component's state changes.

Listing 3.4 Using ARIA states and properties to communicate dynamic changes in the UI

```
<button aria-expanded="false" aria-controls="menu" id="toggle">Menu</button>
<ul id="menu" hidden>
  <li><a href="#">Home</a></li>
  <li><a href="#">About</a></li>
</ul>

<script>
const btn = document.getElementById("toggle");
const menu = document.getElementById("menu");
btn.addEventListener("click", () => {
  const expanded = btn.getAttribute("aria-expanded") === "true";
  btn.setAttribute("aria-expanded", String(!expanded));
  menu.hidden = expanded;
});
</script>
```

3.5.6 Color and contrast

Visual design is part of accessibility. Text must meet minimum contrast ratios against its background (4.5:1 for standard text and 3:1 for large text, as per WCAG 2.1 AA). Avoid using color alone to convey meaning; pair it with text or icons. Tools like browser DevTools or the WAVE extension can automatically check your pages.

NOTE The Web Content Accessibility Guidelines (WCAG) define official criteria for accessibility. You don't need to memorize them, but it's helpful to be aware of their existence. Following semantic HTML, testing with screen readers, ensuring keyboard navigation, and checking color contrast will take you most of the way there in a Vanilla project.

3.5.7 Interactive elements

Interactive elements should rely on native HTML controls whenever possible, as they come with built-in semantics, keyboard interaction, focus management, and accessibility support. If you create custom widgets, you must expose their roles, states, and behaviors using the appropriate ARIA attributes and ensure they respond to both pointer and keyboard events. On touch devices, controls must also respect the platforms' recommended minimum target size to remain usable and avoid accidental taps.

3.5.8 Screen readers

Screen readers parse your DOM and read the structure and content aloud. They rely on headings (`<h1>` to `<h6>`), landmarks, and semantic roles to provide shortcuts to users. Testing with a free screen reader (VoiceOver on macOS/iOS, NVDA on Windows, Orca on Linux) is the best way to ensure your site is usable in that context.

WARNING Do not test accessibility by turning off CSS. That is not how assistive tech works. Use real screen reader tools.

Accessibility is not an afterthought. It is built in if you use the platform as intended. Start with semantic HTML, add ARIA only when necessary, ensure keyboard navigation, maintain good contrast, and test with assistive tech. Following these practices will put your Vanilla Web project in a much better position than most framework-heavy sites that ignore accessibility.

3.6 Media resources

Media is a first-class citizen on the Web. Modern browsers support a wide range of formats for images, video, and even 3D or AR models. Choosing the correct format is part of building a fast and accessible UI. The wrong choice can increase loading times, waste bandwidth, or cause compatibility issues. The good news is that safe formats are consistent across platforms today.

3.6.1 Image formats

For bitmap images, the safest modern formats are PNG, JPEG, WebP, and AVIF. PNG works best for transparency and simple graphics, such as icons or logos. JPEG is still commonly used for photographs, but it is less efficient than WebP or AVIF. WebP is supported in every modern browser and offers smaller sizes with good quality. AVIF provides even better compression, but encoding can be slower, and not every tool supports it yet.

For vector graphics, the standard format is SVG. Being XML-based, SVG is resolution-independent, accessible, and styleable with CSS. It is the best choice for logos, icons, diagrams, and they can even get animated.

You can use the standard `<picture>` element to provide multiple compatible image resources, optionally tailored with media queries. The browser evaluates the available sources—from format to size to media conditions—and automatically selects the most appropriate image for the device, viewport and optional media queries defined.

```
<picture>
  <source srcset="photo.avif" type="image/avif">
  <source srcset="photo.webp" type="image/webp">
  
</picture>
```

3.6.2 Video formats

The most compatible video format for the Web is MP4 with H.264 encoding. It plays on every browser and device, including legacy ones. For modern environments, WebM (utilizing VP9 or AV1 codecs) provides superior compression and quality at smaller file sizes. AV1 in MP4 containers is becoming the new standard, but support for encoding is still catching up.

When embedding video, always include controls unless the footage is decorative, and provide multiple sources so the browser can pick the best available format. Avoid autoplay with sound, as it harms both accessibility and user experience.

```
<video controls width="480">
  <source src="coffee.av1.mp4" type="video/mp4; codecs=av01.0.05M.08">
  <source src="coffee.webm" type="video/webm">
  <source src="coffee.mp4" type="video/mp4">
  Sorry, your browser does not support embedded videos.
</video>
```

3.6.3 AR formats

Web browsers can display 3D models and augmented reality (AR) content using different approaches. The most portable asset formats today are glTF (`.gltf` or `.glb`) and USDZ files (`.usdz`). glTF is the open standard optimized for the Web and works well in Chromium and Firefox and USDZ is Apple's preferred package for AR Quick Look and for Safari's new inline 3D support.

Apple offers a Quick Look on the web that uses `<a rel="ar">` to tell Safari to open the model directly in AR and return to your page when done, compatible with iOS, visionOS, and iPadOS. The following code snippets shows AR Quick Look in action:

```
<a rel= "ar" href="/models/coffee.usdz">
  
</a>
```

On Vision Pro headsets, Safari supports the new HTML `<model>` element for displaying interactive 3D models directly on the page, with features like orbit controls and a `ready` promise for load state. While `<model>` is also being adopted on other devices, the open-source library at <https://modelviewer.dev> provides broader support across iOS, visionOS, Chrome on Android, and other browsers. This library introduces the `<model-viewer>` web component, ready to use today.

Let's see an example of using `<model>` and `<model-viewer>`:

```

<model id="cup" stagemode="orbit" #1
  style="width: 360px; height: 240px;">
  <source src="/models/coffee.usdz" type="model/vnd.usdz+zip">
  
</model>

<!-- Include the script for the model-viewer component -->
<model-viewer #2
  src="/models/coffee.glb" ios-src="/models/coffee.usdz"
  alt= "3D coffee cup" ar camera-controls auto-rotate
  style="width: 360px; height: 240px;">
</model-viewer>

```

#1 HTML element proposed by Apple

#2 Open source web component

A practical strategy is progressive enhancement. Use `<model>` when it is available. Provide a standards-based viewer, such as `<model-viewer>`, everywhere else. Keep an AR Quick Look link as a minimal fallback for iOS and iPadOS.

CODING WITH AI

These days, you can code Vanilla Web projects with AI tools and agents, including giving the tool an image of an app and getting the HTML markup and CSS styles for such an app. However, LLMs are trained on hundreds of thousands of websites that do not utilize any semantic techniques and were developed using outdated HTML and CSS techniques. Therefore, by default, you won't get the best possible results from LLMs. To achieve the best results, be explicit in the prompt that you want semantic HTML, avoid using `<div>` and abusing `class`, and specify that you want to maximize accessibility and utilize modern CSS layout techniques. Listing the methods you wish to use may also increase the opportunity to get the proper markup and styles.

3.7 Rich web UI controls

Modern browsers are increasingly supporting native UI controls, or widgets, that previously required heavy JavaScript or third-party libraries to get right. Several elements and APIs now enable accessible, interactive components that look good out of the box, in addition to all the familiar ones from HTML, such as tables, form elements, video players, and lists.

When needed, you can enhance them with CSS or a bit of markup, not whole frameworks. It reduces code, improves performance, and preserves accessibility, automatically inheriting it from native controls.

3.7.1 Summaries

The `<details>` and `<summary>` elements form the simplest disclosure widget built into HTML. They allow users to hide or show content without requiring JavaScript. Native interaction includes toggling open and closed, keyboard support, and screen reader announcements. You can style the summary marker and the open/closed content using CSS, and even use the `:open` pseudo-class for styling different states. A `::details-content` pseudo-element is available from CSS to style and animate the contents of the details excluding the summary.

```
<details>
  <summary>More about our coffee</summary>
  <p>We roast beans daily in small batches...</p>
</details>
```

3.7.2 Accordions

Accordions are UI components that expand and collapse content sections to save space and improve readability. Accordions built with `<details>` are simple, but a new feature allows "exclusive accordions" using the `name` attribute. When multiple `<details>` share the same name, opening one will close the previously open one. It removes the need for coordinating open states in JavaScript. You get keyboard support, consistent toggling, and a clean group behavior without additional code.

```
<details name="faq">
  <summary>What is Vanilla Web?</summary>
  <p>Vanilla Web is about using native platform features...</p>
</details>
<details name="faq">
  <summary>Why semantic HTML?</summary>
  <p>It improves accessibility, SEO, and maintainability...</p>
</details>
```

3.7.3 Modal dialogs

A *modal dialog* is a pop-up window that temporarily interrupts the page flow to focus the user on a specific task or message.

You can build modal dialogs using the `<dialog>` element. It supports native focus trapping, embedding forms inside modals, and programmatic show and close. Newer enhancements, such as the `closedby="any"` attribute and `command= "show-modal"` on buttons, allow light-dismiss behavior and declarative triggering without requiring full JavaScript. It makes modals easier to implement while preserving accessibility and focus control. We will use modal dialogs later in this book.

Listing 3.5 A simple example of using the <dialog> element

```

<button command="show-modal" commandfor="prefs">Settings</button>
<dialog id="prefs" closedby="any">
  <form method="dialog">
    <label><input type="checkbox" name="dark"> Dark mode</label>
    <menu>
      <button type="submit">Close</button>
    </menu>
  </form>
</dialog>

```

In Figure 3.2 we can see the previous source code with a <modal> dialog in action, closed and opened on a compatible browser.

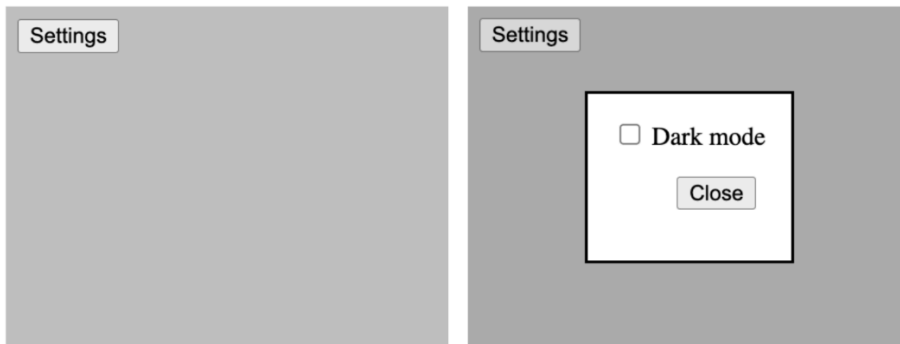


Figure 3.2 We can create modal dialogs that can be opened and closed without any JavaScript, just with HTML markup, optionally customized with CSS. The dialog starts closed (image at the left) and clicking on the button will open it (image at the right).

3.7.4 Carousels

Carousels are rotating containers that let users browse through multiple pieces of content or images within a single, swipeable or clickable area. For example, the sliding image galleries you see on most e-commerce homepages are carousels. Carousels are one special kind of scrolling lists.

Scrolling lists, such as galleries, carousels, or item selectors, can be enhanced using just CSS today. You can use properties like `overflow-x: auto`, `scroll-snap-type`, and new pseudo-elements like `::scroll-button(left)` and `::scroll-button(right)` to create interactive carousels. The browser handles focus, keyboard navigation, and layout. It provides fully scrollable, accessible galleries with minimal code and a familiar feel, requiring no JavaScript libraries.

```
.gallery {
  display: flex;
  overflow-x: auto;
  scroll-snap-type: x mandatory;
}
.gallery > figure {
  scroll-snap-align: center;
}
.gallery::scroll-button(left) {
  content: "<";
}
.gallery::scroll-button(right) {
  content: ">";
}
```

In Figure 3.3, you can see one of the most complex examples from the Chrome team using this technique to create a fully working carousel without any JavaScript code, available at <https://codepen.io/una/pen/ZYbXgpN>.

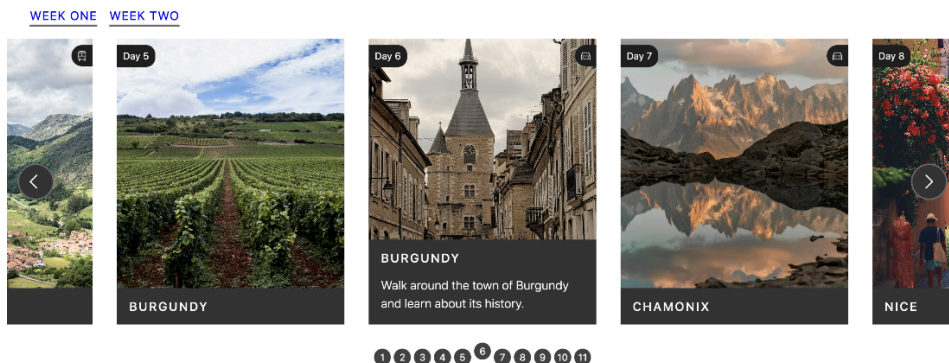


Figure 3.3 A fully working carousel with hand gestures, physical buttons to move forward and backwards, and with page dots, all without any library or JavaScript involved.

3.7.5 Popovers

Popovers are small floating panels anchored to an element, often used for tooltips, menus, or contextual actions. In the past, they required JavaScript libraries to handle positioning, focus, and dismissal. Today, browsers include a native `<popover>` attribute and related APIs that handle these tasks automatically. The element that triggers the popover declares `popovertarget`, and the panel itself carries the `popover` attribute. The browser manages focus, accessibility roles, and closing when the user clicks outside or presses the Escape key.

```
<button popovertarget="info">Show info</button>

<div id= "info" popover>
  <p>Our coffee is roasted daily and shipped fresh.</p>
</div>
```

You can style the popover with CSS, and use the `:popover-open` pseudo-class to animate its appearance or customize its layout. Compared to older libraries, this approach is smaller, faster, and automatically accessible. Using the Popover API in JavaScript you can control and extend its functionality.

3.7.6 Rich selectors

Native `<select>` elements have long resisted styling, leading many to build custom dropdowns. Recent improvements change that. You can now enable a custom mode with `appearance: base-select`, and style parts of the `<select>` like `::picker(select)` for the option box, `::picker-icon` for the arrow, `:checked` options, and `<selectedcontent>` for the chosen item. It gives complete visual control while preserving keyboard, focus, and form integration.

Let's make an example of a customized dropdown selector, first with the HTML semantic markup that looks like a standard code for using a `<select>` element:

Listing 3.6 The HTML markup for a customized dropdown selector

```
<h1>Choose your coffee</h1>
<label for="coffee">Select blend:</label>
<select id="coffee" name="coffee">
  <option value="espresso">Espresso</option>
  <option value="house">House Roast</option>
  <option value="decaf">Decaf</option>
  <option value="coldbrew">Cold Brew</option>
</select>
```

After that, we can customize it using the following stylesheet:

Listing 3.7 The CSS stylesheet for a customized dropdown selector

```
:root { --brand: #6f4e37; }
body {
  font: 16px/1.4 system-ui, sans-serif;
  padding: 2rem;
  background: #f9f9f9;
}
label { display: block; margin-bottom: 1rem; }
select, ::picker(select) { #1
  appearance: base-select; #2
  font: inherit;
  padding: .5rem 2rem .5rem .75rem;
  border: 1px solid #ccc;
  border-radius: 8px;
  background: #fff;
  color: var(--brand);
}
::picker-icon { color: var(--brand); } #3
option:checked { background: var(--brand); color: white; }
selectedcontent { font-weight: 600; } #4
```

#1 Targets the built-in dropdown picker of a <select>

#2 Allows the <select> to be customized

#3 Targets the built-in dropdown arrow icon of a <select>

#4 Used inside a <select> element to display the contents of its currently selected <option>

In Figure [3.4](#) we can see our customized <select> in action. The important part is that if a non-compatible browser is rendering it, it will appear as just a classic select working correctly, so it falls back properly.

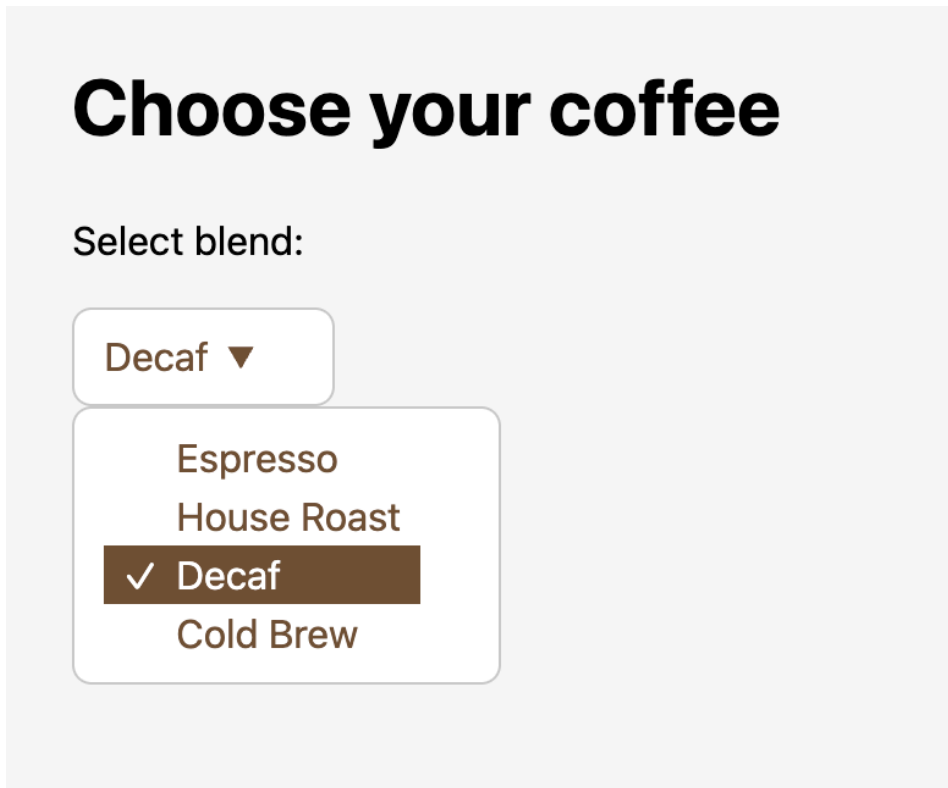


Figure 3.4 With a rich selector, we can now design `<select>` elements with much customization without the need for any JavaScript library, making these user interfaces more accessible and ready for the AI era.

3.7.7 Other UI controls

Other emerging UI controls are also worth watching.

- *Hovercards* allow you to show rich previews when users hover or focus on a link, without custom JavaScript.
- The `<progress>` element has been extended with a progress ring variant, giving you circular indicators natively.
- Toolbars and menus are also getting native upgrades with commands and menu items, reducing the need for custom menu systems.

While support is still evolving, these additions point to a future where more of the widgets we used to build from scratch are part of the platform itself.

When the available widgets are insufficient for our user interface, we will create our own Web Components. We will start working with them in Chapter 7.

3.8 Beyond basic UI

Beyond the HTML and CSS stack, we have numerous other new capabilities in the Web platform that we can utilize to enhance or create our user interface. Most of them require the usage of JavaScript and a Web API from the browsers and rendering engines.

NOTE In Chapter 14, we will focus on Progressive Web Apps (PWAs) and installed web apps, as well as the specifications available for enhancing the user interface in this context.

3.8.1 Using canvas

Canvas was one of the first APIs that appeared with the HTML5 era in the late 2000s. It allows us to define a portion of the HTML document (which can also be the entire document) whose content will be rendered by JavaScript using one of the available contexts, such as the simple 2D context, or 3D experiences using APIs like WebGL or other 3D techniques.

These topics are outside the scope of this book, but you will find many resources, such as MDN Web Docs (<https://developer.mozilla.org>) and even micro-libraries that can help you based on the specific use case you have.

WARNING Do not use Canvas for rendering text, form controls, or other content that can be rendered with the HTML and CSS stack unless you know what you are doing. It's much harder to comply with accessibility, usability, and discoverability using that technique.

3.8.2 Using UI-related APIs

The browser platform provides us with more than just HTML and CSS for designing interfaces. Some APIs directly impact how users perceive, experience, and interact with your app. They do not replace semantic markup, but they extend what is possible on the screen and beyond.

The most relevant UI-related APIs today are:

- Fullscreen API: lets you toggle elements such as a video player, map, or gallery into immersive fullscreen mode. Use it only when the user explicitly asks for it, never on page load.
- Picture-in-Picture API: allows video, and in some cases other elements, to float in a small window above other apps in the operating system. It is common for media apps, but they can also support workflows where users need to multitask.

- Casting APIs: enable your app to send media or presentations to external displays or smart TVs. Support varies, but most Chromium-based browsers expose the Presentation API and Remote Playback API.
- Foldables and multi-screen support: with new form factors like folding phones or dual screens, the CSS `env()` variables and APIs such as Window Segments help adapt your layout.
- Media Session API: integrates with operating system media controls on lock screens, car displays, and smartwatches. It can display track information and handle play and pause events associated with your web content.
- WebXR: provides immersive access to virtual and augmented reality devices, integrating with headsets or AR-capable phones. It is lower level than libraries but future-proof for native-like XR on the Web.
- Web Speech API: offers speech recognition for voice input and speech synthesis for text-to-speech. It can enhance accessibility or enable hands-free scenarios using cloud or local AI speech models.

WARNING Some of these APIs are available on Baseline, while others are not yet widely available and may require additional permissions or user gestures. Always use feature detection and offer fallbacks. The Vanilla Web pattern is about enhancing experiences when possible, while keeping the core usable everywhere.

3.9 Summary

- Prefer platform UI first: semantic HTML + modern CSS covers most app needs with less code and better accessibility.
- Think beyond screens: design for any viewport, assistive tech, search engines, and AI agents.
- The web experience today can exist beyond the screen.
- Ship the proper metadata on every HTML document: title, icons, theme color, viewport, OpenGraph, and a solid Web App Manifest.
- On top of the basic metadata, add meaningful markup: use native semantic elements as much as possible; add roles/states only when HTML can't express intent.
- Choose safe media formats and provide icons for your website.
- Reach for native UI controls first, such as using details/summary, `<dialog>`, popovers, and enhanced `<select>` reduce JS while improving a11y.
- Keep canvas specialized: great for games/graphics—avoid for general text and form UI unless you know the trade-offs.
- Enhance progressively: feature-detect UI-related APIs and provide graceful fallbacks.

4 *Vanilla CSS*

This chapter covers

- Vanilla CSS for real apps with modern layout systems.
- Using modular CSS with Cascade Layers and variables.
- Responsive design with Container Queries and new units.
- Animating content with semantic transitions.

In the previous chapter we cover the user interface basics of the modern web, focusing on the semantic markup. The next step is to talk about CSS, or Cascading Style Sheets.

CSS is your layout and design language. In a Vanilla Web project, we use modern CSS directly without frameworks or CSS libraries, such as Tailwind or Bootstrap. The goal is to keep styles small, predictable, and easy to reason about. You will rely on platform features such as custom properties, layers, container queries, and improved selectors to clearly express your system.

A maintainable stylesheet starts with naming things by meaning, not by appearance. Prefer semantic classes that match the UI role or component, and then let properties and media features adapt them according to context. Avoid over-specifying rules. Keep selectors short and stable.

NOTE While this book is not a comprehensive guide to CSS, we will expand on and apply all the techniques shown in this section in the following chapters as we work with Web Components.

Modern CSS provides you with first-class primitives to structure complexity. Custom properties carry variable values (tokens) and theme values. Cascade layers sequence entire groups of rules. Container queries let components adapt to their parent rather than the viewport. New units and selectors reduce JavaScript and simplify responsive behavior.

You do not need a preprocessor to be productive. If you like nesting or variables, you already have them. If you need design tokens, you can encode them with custom properties. If you want a reset, you can write a tiny one that matches your project. The platform is ready, as shown in the following code:

```
/* Minimal reset and base tokens */
:root {
  --space-2: .5rem; #1
  --space-3: .75rem; #1
  --radius: 12px; #1
  --brand: #5d4037; #1
}
* { #2
  box-sizing: border-box; #2
} #2
img { #2
  max-width: 100%; #2
  height: auto; #2
  display: block; #2
} #2
nav { #3
  button { #3
    border-radius: var(--radius); #4
    background: var(--brand); #4
    color: #fff;
  }
}
```

#1 Design base tokens in custom properties

#2 Minimal reset

#3 Nested selectors

#4 Using tokens

4.1 Maintainable and modular CSS

Many developers believe that working with Vanilla CSS involves fighting against the limitations of centering a `<div>` or using outdated techniques, such as `float`. The reality is that modern CSS is powerful, and you can keep your code semantic and straightforward, making it easy to read and maintain, by understanding the current capabilities of the language.

NOTE CSS maintainability in professional environments may include some tools for continuous integration testing or linting with Stylelint.

In the next chapter, we will focus on Web Components and analyze how to make your CSS modular for them. For now, let's examine some recent additions that make a difference in keeping a maintainable and modular stylesheet. These are all in Baseline, so they are ready to use for all browsers.

4.1.1 Cascade Layers

Cascade layers let you control the order in which groups of rules participate in the cascade. Instead of relying on selector specificity tricks, such as using `!important` or changing the priority of selectors or stylesheets in the file, you decide which values come first and which ones override.

If you integrate third-party CSS, wrap it in its own layer to prevent it from unexpectedly overriding your own components.

For example, the following sample creates four layers: reset, base, component, and utilities.


```

@layer reset, base, components, utilities; #1

@layer reset { #2
  *, *::before, *::after {
    box-sizing: border-box; margin: 0; padding: 0;
  }
}

@layer base { #3
  body {
    font: 16px/1.5 system-ui, sans-serif; color: #222;
    background: #fff;
  }
  button {
    font: inherit; padding: .5rem 1rem;
    border: 1px solid #ccc; border-radius: 6px;
  }
}

@layer components { #4
  .btn-primary {
    background: #5d4037;
    color: #fff;
    border: none;
    border-radius: 12px; #5
  }
}

@layer utilities { #6
  .round { border-radius: 999px; }
  .text-center { text-align: center; }
}

```

#1 Layer's priorities definition

#2 Reset layer: stripped down defaults

#3 Base layer: element-level defaults

#4 Components layer: reusable UI elements

#5 Overrides base button radius

#6 Utilities layer: quick one-off helpers

Here, the cascade order is explicit:

- Reset comes first, then base, then components, then utilities.

- A base `button` sets defaults, the `.btn-primary` in the components layer cleanly overrides radius and colors, and the `.round` utility wins when used in markup because utilities are last in the stack.

4.1.2 Custom properties (variables)

Custom properties (also known as variables) hold your design's dynamic values, such as color, spacing, typography, radius, and even timing or easing. Instead of hard-coding values for them, we define them in custom properties, which we then reference every time we need them using `var(id)`. They inherit through the DOM, can be overridden per component or container, and update live without the need for JavaScript.

Keep the values for your variables at the `:root` level (which applies to the entire document); then, you can override them in themes or containers. Later, components can introduce their own internal variables for sizing and state, while still consuming global tokens and keeping concerns clear.

NOTE For CSS Custom Properties, prefer semantic names, such as `--brand` or `--surface-1`, over literal names, like `--blue-500`. Semantic names survive redesigns better.

Variables can participate in calculations and media queries, such as pairing them with `@media (prefers-color-scheme)` or container queries to adapt values without branching in JavaScript.

```

@layer global {
  :root {
    --brand: #5d4037;
    --surface: #ffffff;
    --text: #222;
    --radius: 12px;
    --gap: 12px;
  }
  @media (prefers-color-scheme: dark) { #1
    :root { #1
      --surface: #121212; #1
      --text: #eee; } #1
    } #1
  }
}

@layer components {
  .button {
    --pad-x: 1rem;
    --pad-y: .625rem;
    background: var(--brand); #2
    color: #fff;
    padding: var(--pad-y) var(--pad-x); #2
    border-radius: var(--radius); #2
  }
}

```

#1 Media query overriding tokens for custom properties when a dark theme is set in the OS

#2 Usage of tokens or values from custom properties with var()

DEFINING THEMES

Theming is overriding a known set of tokens (values for our custom properties) at a higher scope. You can theme the whole document by toggling a class on `html`, or theme a subtree by applying a class to a container. Child components inherit the new values automatically.

Expose only the tokens you intend to be themed. Keep component internals stable. Use media queries to provide automatic light or dark defaults, then let a user preference or setting class override them.

You can also define brand variants for white label deployments. Ship a single codebase and swap theme values per customer or environment.

Store the user selection in local storage or a cookie and reflect it with a class on `html`, which will let you keep the CSS static and fast.

We can create a simple example with custom properties and themes as classes overriding the tokens, applying the theme class to the `<html>` tag using JavaScript, as follow:

Listing 4.1 Working with themes using Custom Properties and classes

```

<html class="theme-espresso">
<head>
  <style>
    :root { #1
      --brand: #5d4037; #1
      --surface: #fff; #1
      --text: #222; #1
    }
    .theme-espresso { --brand: #6d4c41; } #2
    .theme-matcha { --brand: #2e7d32; } #2
    .theme-berry { --brand: #8e24aa; } #2
    body {
      background: var(--surface);
      color: var(--text);
      font-family: sans-serif;
      padding: 1rem;
    }
    button {
      background: var(--brand);
      color: #fff;
      padding: .625rem 1rem;
      border-radius: 12px;
      border: none;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <h1>Coffee Roasters</h1>
  <button onclick="setTheme('espresso')">Espresso</button>
  <button onclick="setTheme('matcha')">Matcha</button>
  <button onclick="setTheme('berry')">Berry</button>

  <script>
    function setTheme(name) {
      document.documentElement #3
        .className = 'theme-' + name; #3
    }
  </script>
</body>
</html>

```

- #1 Base token declarations in custom properties
- #2 Each theme in a class replaces values
- #3 We apply the theme to the `<html>` element

4.1.3 Container queries and units

Container queries enable components to respond to the size of their nearest container, rather than the viewport, as is done when using media queries. It solves the classic problem where a card must switch layout when placed in a narrow sidebar, even if the page is wide.

Mark a parent as a size container with `container-type: inline-size` or `size`. Then write `@container` rules that trigger at container widths or heights. It keeps components portable and reduces global media breakpoints.

Pair container queries with container units, such as `cqw` and `cqh`, when you want measurements relative to the container. Use dynamic viewport units, such as `dvh`, to avoid mobile browser UI jumps. Prefer `dvh` over `vh` for small screen layouts.

Use container names to scope queries to specific components when needed. It avoids accidental matches and keeps intent clear, as we can see in the following example. We start with a simple semantic HTML code:

Listing 4.2 CSS Container Queries demo markup

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Container Query Demo</title>
  <link rel="stylesheet" href="container.css">
</head>
<body>
  <main>
    <article class="card">
      
      <section>
        <h2>Espresso Blend</h2>
        <p>Strong and rich, roasted fresh daily.</p>
      </section>
    </article>
  </main>

  <aside>
    <article class="card">
      
      <section>
        <h2>Matcha Tea</h2>
        <p>Smooth Japanese green tea powder.</p>
      </section>
    </article>
  </aside>
</body>
</html>

```

We then define in container.css the following code:

Listing 4.3 CSS Container Queries in action

```

body {
  font-family: system-ui, sans-serif;
  display: grid;
  grid-template-columns: 2fr 1fr;
  gap: 2rem;
  padding: 1rem;
}

main {
  display: grid;
  gap: 1rem;
  container-type: inline-size; #1
}
aside {
  container-type: inline-size; #1
}

article.card { #2
  border: 1px solid #ccc; #2
  border-radius: 12px; #2
  padding: 1rem; #2
  background: #fafafa; #2
} #2

@container (min-width: 400px) { #3
  article.card { #4
    display: grid; #4
    grid-template-columns: 120px 1fr; #4
    gap: 1rem; #4
  } #4
  article.card img { #4
    width: 100%; #4
    border-radius: 8px; #4
  } #4
}

```

#1 Enable width-based container queries on elements

#2 Cards start with image above text

#3 Responsive behavior tied to container, not viewport

#4 Cards with image and text side by side when the container holding <article> is at least 400px wide.

In Figure 4.1, we can see the previous example in action, in different screen sizes. On the same viewport, sometimes you have cards with image above text and image and text side by side and sometimes all of them are using the same layout. That's because the rules applied are based in `<article>` containers size and not the whole screen size.

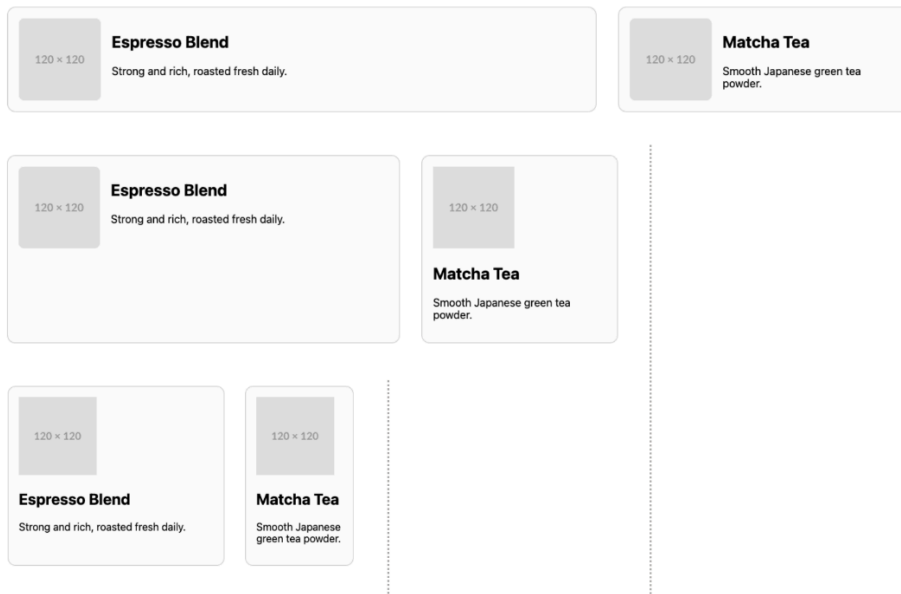


Figure 4.1 Here we can see **Container Queries** in action. We can specify different layout techniques or designs based on the container's dimensions, rather than focusing on the entire available screen size, as with classic responsive techniques using **Media Queries**.

4.1.4 Nested selectors

CSS nesting lets you write selectors in a tree that mirrors your HTML structure. It reduces repetition and improves readability without increasing specificity if used carefully. The `&` represents the parent selector when you need to append classes or states.

Nesting works in regular stylesheets and inside layers. It plays nicely with custom properties and container queries, as shown in the following sample.

This code is much cleaner than creating four root-level selectors, such as `.card`, `.card.is-featured`, `.card a` and `.card a:hover`.


```
.card {
  padding: 1rem;
  border: 1px solid #ddd;
  border-radius: 12px;

  &.is-featured { border-color: #5d4037; }

  a {
    text-decoration: none;
    &:hover { text-decoration: underline; }
  }
}
```

4.1.5 New pseudo-selectors

Modern pseudo-selectors reduce boilerplate and improve accessibility. It also helps keep our HTML with less need for `id` and `class` declarations.

These features help you author fewer rules with more apparent intent. Some of the additions are:

- `:is()` lets you group multiple selectors in one rule while keeping only the highest specificity among them.
- `:where()` works like `:is()`, but always has zero specificity, making it perfect for safe defaults and resets that other rules can override.
- `:has()` acts as a parent selector, so you can style an element based on what it contains (for example, a card that has an image inside).
- `:focus-visible` applies focus styles only when they are expected, such as during keyboard navigation, avoiding distracting outlines when users click with a mouse.

WARNING Use `:has()` with care to avoid heavy selectors on extensive documents. Limit its scope to components or containers.

Let's see some quick examples of using these new selectors.

```

:is(h1, h2, h3, article) a { color: #5d4037; } #1

:where(nav a) { text-decoration: none; } #2

.card:has(img, video) { #3
  display: grid; grid-template-columns: 160px 1fr;
}

button:focus-visible { #4
  outline: 2px solid #5d4037;
  outline-offset: 2px;
}

```

#1 Style links in headers or articles

#2 Zero specificity defaults, no other style will override

#3 Parent styling if a card has a media element

#4 Accessible focus

4.1.6 New units

For a long time, we had only a handful of units in CSS: pixels, percentages, `em`, and `rem`. They worked, but they were insufficient for complex, responsive layouts or precise typography. The platform has grown, and now we have a family of modern units that solve problems we used to patch with workarounds.

Viewport units were the first step, but the original `vh` and `vw` had a flaw on mobile devices: they did not account for the dynamic browser chrome that appears and disappears when the user scrolls. The new dynamic viewport units `dvh`, `lvh`, and `svh` address that. In practice, you will most often use `dvh` to size sections that should always fill the visible height without jumping when the address bar is hidden or shown.

Typography also benefits from new units that align to text metrics rather than pixels. Units like `lh` and `rlh` let you size elements relative to the line height, while `cap` and `ic` align to the actual shapes of characters, such as the height of a capital letter or the width of an ideograph. These provide you with much finer control when designing text-heavy interfaces that must appear consistent across fonts and languages.

Responsiveness is not only about the viewport. Container units, such as `cqw`, `cqh`, `cqi`, and `cqb`, allow you to measure sizes relative to the nearest container, pairing perfectly with container queries. Logical units, such as `vi` and `vb`, follow the writing mode, so a layout will adapt automatically whether the text flows from left to right, right to left, or vertically.

For Vanilla projects, it is recommended to prefer relative units whenever possible. Use modern viewport, container, and text-relative units for scalable designs. Keep absolute units only for borders, hairlines, or fixed measurements. Above all, be consistent: if spacing and type scale are defined with the same logic across your project, the whole system will be easier to maintain and evolve.

The following stylesheet shows a sample on how to use relative units.

```

.hero {
  min-height: 100dvh; #1
  padding-block: 4lh; #2
  display: grid;
  place-items: center;
  background: #5d4037;
  color: white;
}

.title {
  font-size: 8cap; #3
  margin-block-end: 2lh; #4
}

```

#1 fills the visible viewport height

#2 vertical spacing based on line height

#3 size relative to the capital letter height

#4 spacing consistent with line rhythm

4.2 Modern Layout

Modern layouts for Vanilla Web projects sit on top of a semantic HTML document that is not affected by `divitis` or `classitis`. If we keep our HTML clean and straightforward, we will also copy that advantage to our stylesheets using these modern layout systems:

- For laying out elements in 1 dimension (1D), as in vertically or horizontally, we will use Flexbox.
- For laying out elements in 2 dimensions, we will use mainly grids and subgrids.
- We may also want to position elements absolutely for specific purposes, typically in an application, such as a tab bar, a menu bar, or an action button. We will avoid as much as possible legacy layout CSS techniques, such as changing the display nature of elements and using float and clear.

NOTE In Chapter 8, we will also focus on shadow DOMs, specifically targeting web components, allowing us to use CSS that affects only one portion of the document, isolated from the rest.

4.2.1 1D layout with Flexbox

Flexbox is the modern way to lay out elements in one dimension, either horizontally in a row or vertically in a column. Before Flexbox, we often hacked layouts with floats or tables. Those approaches were fragile and hard to maintain. Flexbox provides a native way to distribute space, align items, and reorder them as needed, all with a few properties.

The key idea is that you mark a parent container as a flex container. Its direct children then become flex items. By default, they sit in a row, but you can switch to a column with a single property. The container controls how items are distributed with `justify-content`, and how they align with `align-items`. Each child can also grow or shrink according to available space.

Flexbox excels in UI components, such as toolbars, navigation menus, cards, and controls. It is not meant for complex page grids but is perfect for any situation where you want items to adapt in one axis. A horizontal row of buttons that grows and shrinks gracefully is much easier to achieve with Flexbox than with percentages or inline-block hacks.

Another advantage is responsiveness without the need for extra media queries. By setting flexible growth and shrink values, items adapt automatically. For example, buttons expand to fill the container, or cards wrap to the following line when space is limited. Flexbox also handles vertical centering, which was notoriously difficult with older techniques.

```
nav {
  display: flex;
  gap: 1rem;
  justify-content: space-between; #1
  align-items: center; #2
}

nav a {
  padding: .5rem 1rem;
  background: #5d4037;
  color: white;
  border-radius: 8px;
  text-decoration: none;
  flex: 1; #3
}
```

#1 distribute items across the row

#2 vertical alignment

#3 each link grows to share available space

There is much more to talk about Flexbox, a feature that will definitely be one of the most used in a Vanilla Web project. We will use different Flexbox's features along the rest of the book.

4.2.2 2D layout with grids

CSS Grid is the native system for two-dimensional layout. Where Flexbox excels along a single axis, Grid allows you to place items in rows and columns simultaneously, define track sizes, and control how content spans across them. You describe the structure once in the container, then position children without extra wrappers. It keeps markup clean and makes complex layouts predictable.

A typical workflow starts by declaring a grid with named areas or explicit columns and rows. You can create responsive tracks using `minmax`, fractional units, and the `auto-fit` or `auto-fill` keywords, allowing the grid to expand and collapse fluidly. Components inside the grid can span tracks or align themselves to edges without resorting to floats or manual calculations.

NOTE Prefer `gap` on grids and flex containers to create space between items, rather than using margin properties. It avoids margin-collapsing issues, maintains consistent spacing in both axes, and makes alignment rules easier to follow. Use margins for spacing outside the layout context, not inside the grid itself.

Grids are great for pages, dashboards, product listings, and any composition that needs consistent alignment in both directions. They are also skilled at rearranging content visually without altering the DOM, which can be helpful for responsive designs where a sidebar moves up or down. Combine Grid with intrinsic sizing to let content determine where tracks are positioned.

When you mix Grid with container queries, each grid can react to its own container width. It means a card gallery adjusts the number of columns when placed in a narrow sidebar or a wide main area, without global breakpoints. Grid also plays well with modern units, such as `cqi` and `dvh`, providing fine control across various contexts.

Let's see a simple CSS Grid demo starting with the HTML markup.

Listing 4.4 The HTML markup for a Simple CSS Grid Demo

```

<!doctype html>
<html lang="en">

<meta charset="utf-8">
<title>Simple CSS Grid Demo</title>
<meta name="viewport" content="width=device-width,initial-scale=1">
<link rel="stylesheet" href="grids.css">

<h1>Products</h1>

<section class="grid" aria-label="Product grid">
  <article class="card">
    <div class="thumb" role="img" aria-label="Espresso blend"></div>
    <h2>Espresso Blend</h2>
    <p>Strong and rich, roasted daily.</p>
    <p class="price">$14.90</p>
  </article>

  <article class="card">
    <div class="thumb" role="img" aria-label="House roast"></div>
    <h2>House Roast</h2>
    <p>Smooth and balanced for every day.</p>
    <p class="price">$12.50</p>
  </article>

  <!-- More articles -->
</section>

```

Now we can write the grids.css file that defines the layout for the previous code listing.

Listing 4.5 The CSS stylesheet for a Simple CSS Grid Demo

```

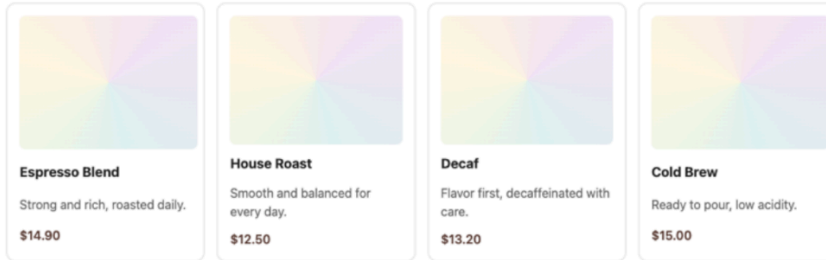
:root {
  --gap: 16px; --line: #ddd; --radius: 12px;
  --ink: #222; --brand: #5d4037;
}
body {
  * { box-sizing: border-box; }
  margin: 0; padding: 2rem;
  font: 16px/1.5 system-ui, sans-serif; color: var(--ink);
}
.grid { #1
  display: grid;
  gap: var(--gap);
  grid-template-columns: repeat(auto-fit, minmax(16rem, 1fr));
}
article.card {
  border: 1px solid var(--line);
  border-radius: var(--radius);
  padding: 1rem;
  background: #fff;
  display: grid;
  gap: .75rem;
}
.thumb {
  aspect-ratio: 4 / 3;
  border-radius: 8px;
  background: conic-gradient(from 45deg, #f3e5f5,
    #e0f2f1, #fff8e1, #f3e5f5);
}
h2 { margin: 0; font-size: 1.1rem; }
p { margin: 0; color: #666; }
.price { color: var(--brand); font-weight: 600; }

```

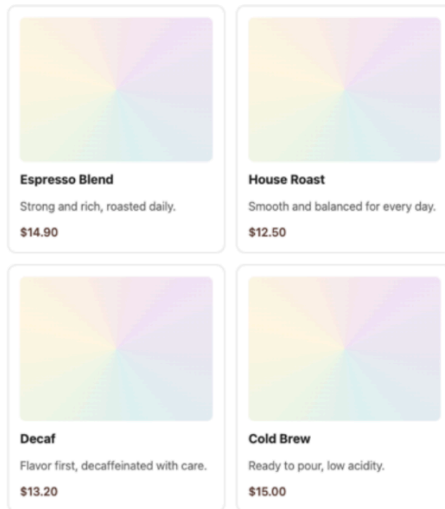
#1 Responsive 2D grid: as many columns as fit, minimum 16rem each

In Figure [4.2](#), we can see the grid sample in action in different viewport sizes.

Products



Products



Products

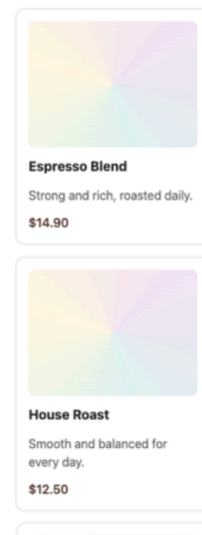


Figure 4.2 CSS Grids will be useful for rendering 2D lists in many apps, such as the product lists, as seen in this sample. It can easily adapt to various container sizes.

SUBGRIDS

Subgrid lets a nested grid inherit the tracks of its parent grid. Before the introduction of subgrid (added in Baseline 2023), aligning items across component boundaries required duplicated track definitions or cumbersome workarounds. With `grid-template-columns: subgrid` or `grid-template-rows: subgrid`, a child grid uses the same track lines as its ancestor, so elements line up perfectly even when they are in different components.

Use subgrids when you have repeated patterns, such as a list of cards where the header, body, and actions should align across all cards, or a complex page where a sidebar and main content need to share column boundaries. The parent establishes the rhythm, and children snap to it without redefining sizes.

Subgrid improves maintainability because track sizes live in one place. If you change the parent grid, children adjust automatically. It reduces duplication and maintains consistent alignment across modules. It also enhances accessibility and readability, as columns and baselines remain stable.

4.3 Animations

Modern Vanilla Web projects rely on a small set of animation tools built into the platform, rather than relying on JavaScript libraries. You will use CSS transitions for simple state changes, CSS keyframe animations for choreographed motion, View Transitions for page and route changes in SPAs and MPAs, and scroll-driven animations to bind motion to scroll. Keep animations purposeful and short, and always respect the user's motion preferences using the media query `prefers-reduced-motion`.

NOTE For optimal performance, prefer transform and opacity for transitions and animations. They animate on the compositor in most engines and avoid layout thrashing. If in doubt, use DevTools for testing.

4.3.1 Transitions

CSS *transitions* smoothly animate a property from its current value to a new one when the state changes, such as when hovering or toggling a class. You control the properties, duration, delay, and easing. Transitions are ideal for micro-interactions, such as buttons, menus, and disclosure controls.

```
.button {
  background: #5d4037; color: #fff; border-radius: 12px;
  transition: transform .18s ease, box-shadow .18s ease;
}
.button:hover {
  transform: translateY(-2px);
  box-shadow: 0 6px 20px rgba(0,0,0,.15);
}

@media (prefers-reduced-motion: reduce) { #1
  .button { transition: none; }
}
```

#1 Respect user preference

4.3.2 Keyframe animations

Keyframe animations enable you to choreograph multi-step motions over time and loop them if needed. Use the `@keyframes` at-rule and the `animation` shorthand. Reach for this when a simple one-step transition is not enough. Check https://developer.mozilla.org/docs/Web/CSS/CSS_animations for more information.

```
@keyframes float {
  from { transform: translateY(0); opacity: .9; }
  to   { transform: translateY(-6px); opacity: 1; }
}
.badge {
  animation: .9s ease-in-out float alternate infinite;
}

@media (prefers-reduced-motion: reduce) { #1
  .badge { animation: none; }
}
```

#1 Respect user preference

4.3.3 Page transitions

The *View Transitions API* provides built-in page and route transitions; therefore, you can animate a transition between two different views or pages of your application without any JavaScript code, emulating native app transitions. In SPAs, you wrap DOM updates with `document.startViewTransition`. For MPAs, you can opt into cross-document transitions with the `@view-transition` at-rule, which applies to same-origin navigations when both pages opt in.

We will focus on these transitions in Chapter 14.

4.3.4 Scroll-driven animations

Scroll-driven animations enable you to tie motion to scroll progress rather than time. Use `animation-timeline` with `scroll()` to scrub a keyframed animation as the user scrolls, or use `view()` and view-timelines to animate elements as they enter the viewport. Support is growing, so test the specific properties you use. Check https://developer.mozilla.org/docs/Web/CSS/CSS_scroll-driven_animations for more information.

4.4 Summary

- Vanilla CSS is the primary layout and design language in a Vanilla Web project, replacing utility frameworks with small, semantic stylesheets that rely on modern platform features instead of abstractions.
- Cascade layers give you explicit control over the cascade order across resets, base styles, components, and utilities, reducing the need for `!important` and making third party CSS predictable to integrate.
- Custom properties act as design tokens for colors, spacing or typography; they inherit through the DOM, power themes and variants, and can be updated or overridden without recompiling or touching JavaScript.
- Container queries and container units let components adapt to the size of their parent instead of the viewport, while new viewport and text-relative units fix mobile chrome issues and improve typographic consistency.
- Modern selectors like `:is`, `:where`, `:has`, and `:focus-visible` reduce boilerplate, enable parent-aware styling and safer defaults, and help keep HTML free of unnecessary ids and classes without sacrificing clarity.
- Flexbox, Grid, and Subgrid form the core layout toolkit: Flexbox for 1D alignment, Grid for 2D tracks, Subgrid for shared columns, all using gap, intrinsic sizing, and logical properties instead of floats or hacks.
- Animations rely on native tools first: transitions for state changes, keyframes for choreographed motion, View Transitions and scroll-driven animations for larger flows, always honoring `prefers-reduced-motion` and sticking to transform and opacity for performance.

5 *Vanilla JavaScript*

This chapter covers

- How JavaScript has evolved into ECMAScript versions and what this means for Vanilla projects.
- The essentials of modern JavaScript syntax and patterns you will use on a daily base.
- Coding using ES Modules in Vanilla Web projects.
- Handling errors for robust apps without external libraries.

One of the most prominent parts of a Vanilla Web project is JavaScript, specifically what we usually refer to as *Vanilla JavaScript*: the use of the core language and browser APIs to develop apps without large libraries or frameworks.

In the previous chapters we covered the Web UI basics, including the semantic usage of HTML and modern CSS features. In this chapter we'll start covering how to use JavaScript to enhance HTML and CSS through dynamic features without using libraries or frameworks. JavaScript is the only programming language compatible with all browsers and web rendering engines.

Before moving on with the JavaScript code, it is essential to understand how JavaScript is versioned under its original name, *ECMAScript*. By versioning, we are thinking about how we name new abilities in the language and in the execution engine. Do we have a JavaScript 3.0? Or maybe a JS 9.1.1?

Versioning is particularly important if you are coming to Vanilla Web from a framework or library, such as Angular or React. In such cases, you typically utilize the latest features of the language, or even supersets like TypeScript, which is later transpiled or compiled into a specific version of ECMAScript. When working with Vanilla JavaScript, you will mostly ship the same code that you write, unless you decide to use a tool to bundle, compress, or obfuscate your code.

5.1 Versioning

JavaScript approves new ideas for its syntax and core API through the TC39 committee in the ECMA organization. The process involves discussions and a draft that moves from stage 0 to stage 4. The following table outlines the levels of the TC39 process and what to expect at each one. That will help you understand if you are confident on using a feature that is still in early stages, even if you find a browser supporting it or a compiler that will let you use it.

Table 5.1 The TC39 process stages before becoming part of the ECMAScript standard

Stage	Meaning
Stage 0	Strawman: an initial idea or proposal, not yet formally accepted, often experimental.
Stage 1	Under Consideration: the committee accepts it as worth exploring. The feature will be specified in detail, but it may undergo significant changes or be dropped.
Stage 2	Draft: the syntax and semantics are defined. Browser vendors may start implementing for testing purposes, but it is not yet final.
Stage 2.7	Approved: the syntax and semantics are approved.
Stage 3	Candidate: the feature is complete and is undergoing feedback from implementers and developers. Major browsers usually experiment here.
Stage 4	Complete: the feature is ready to be part of the ECMAScript standard. It will be included in the next yearly edition.

Features in Stage 3 may take some time to appear in all browsers, so always check compatibility on MDN (<https://developer.mozilla.org/>), Baseline (<https://web.dev/baseline/>), or CanIUse (<https://caniuse.com/ecmascript>).

NOTE Stages are managed by TC39, the committee responsible for evolving the JavaScript core language specification. Most browser and platform APIs are separated from TC39 and defined mainly by the W3C, the OpenJS Foundation, and other organizations. The Internationalization API is a separate ECMAScript-related specification (ECMA-402), also managed by the TC39.

ECMAScript publishes a new edition each year, incorporating all proposals that have reached Stage 4 by the cutoff date. A Stage 4 feature is fully standardized, but that doesn't guarantee immediate availability: JavaScript engines still need to implement it, which can take months or even a couple of years. However, reaching Stage 4 requires that at least two independent engines have already implemented the feature, ensuring it is both viable and interoperable.

ECMAScript is a living specification, and browsers don't implement its features in edition-sized batches. Instead, they ship individual capabilities as soon as they're ready. As a result, features from different ES editions may appear out of order across engines, depending on each vendor's priorities and release cycle.

COMING FROM THE EARLY 2000S JAVASCRIPT

If you learned JavaScript during the early 2000s, the language has evolved significantly, and the most significant change happened with ES2015, also known as ES6 - ECMAScript 6. That edition introduced class syntax, ES modules, Promises, and many other new features. Later versions, such as ES2017 and ES2020, introduced some important new features that we will use every day, including `async/await` and the `??` operator. Other annual versions included minor changes that we may not use daily.

5.2 Latest additions to the language

The following table shows the most prominent additions up to ES2025 that you may want to use in your Vanilla Web projects.

Table 5.2 Most prominent additions to ECMAScript language

ECMAScript Version	Key Additions
ES5 (2009)	Strict mode, JSON support, <code>Array.prototype.forEach</code> , <code>Object.create</code> .
ES2015 (ES6)	<code>class</code> syntax, <code>let</code> and <code>const</code> , arrow functions, template literals, modules (<code>import/export</code>), Promises, new operators.
ES2016	Exponentiation operator (<code>^</code>), <code>Array.prototype.includes</code> .
ES2017	<code>async / await</code> , <code>Object.values / Object.entries</code> .
ES2018	Rest/spread for objects, asynchronous iteration (<code>for await...of</code>), RegExp improvements.
ES2019	<code>Array.prototype.flat / flatMap</code> , <code>Object.fromEntries</code> , optional catch binding.
ES2020	Nullish coalescing (<code>??</code>), optional chaining (<code>?.</code>), <code>BigInt</code> , <code>Promise.allSettled</code> .
ES2021	Logical assignment operators (<code>&&=</code> , <code> =</code> , <code>??=</code>), <code>String.prototype.replaceAll</code> , <code>WeakRefs</code> .
ES2022	Top-level <code>await</code> , class fields and private methods, <code>Array.prototype.at</code> .
ES2023	Array copy methods (<code>toSorted</code> , <code>toReversed</code> , etc.), Hashbang (<code>#!</code>) grammar, <code>Symbol.prototype.description</code> .
ES2024	Resizable and transferable <code>ArrayBuffer/SharedArrayBuffer</code> , <code>RegExp /v</code> flag, Unicode string methods, <code>Promise.withResolvers()</code> , <code>Array.prototype.groupBy</code> .
ES2025	Iterator Helpers, new Set operations, JSON Modules, <code>Promise.try</code> , <code>RegExp.escape</code> , Import Attributes, duplicate named capture groups in <code>RegExp</code> , <code>Float16Array</code> .

Not every addition is equally relevant in day-to-day Vanilla Web projects. For example, while `BigInt` is critical in crypto or finance, most apps rely more often on `async/await`, modules, and optional chaining.

WARNING Most of the new syntax abilities are entirely incompatible with older engines, which means that if you ship code that is not compatible with your target engine, your code will fail. Some other new abilities, such as new methods or constructors, may be polyfilled or emulated with code. Keep in mind that we can't specify in our code which ES version we expect as a minimum.

5.2.1 ESNext

ECMAScript Next (ESNext) is a non-official name for features that will be in the next version of ES, and they are already at Stage 3 or Stage 4. In those cases, they will likely be implemented in the spec at some point in the future, so that some browsers may have already implemented these features alone in newer versions.

5.3 Using a transpiler

Suppose you want to use a recent JavaScript syntax, or even a syntax that hasn't been approved yet. In that case, you may want to use a *transpiler* to convert that code into a more stable version before shipping your code, unless you control the engine that is going to execute your code, such as a web app that you know is going to be executed in devices with a minimum version of Chrome. One of the most prominent transpilers for JavaScript is Babel, but TypeScript and esbuild are also available.

For example, suppose we use optional chaining (`?.`) in ES2020+ to access nested properties safely:

```
// ES2020+ code
const user = {};
console.log(user.profile?.email); // undefined, no error
```

If we transpile this down to ES2019, Babel rewrites it into longer conditional checks:

```
// Transpiled to ES2019
const user = {};
console.log(
  user.profile === null || user.profile === undefined
    ? undefined
    : user.profile.email
);
```

In practice, transpilers allow you to use the modern syntax you prefer without worrying about immediate compatibility. The cost is that your shipped code may become longer or slightly less performant, but in many cases, the developer experience and safety you gain outweigh those trade-offs.

NOTE Optional chaining looks cleaner, but under the hood, transpilers expand it into equivalent `null/undefined` checks to ensure older browsers can run the code without syntax errors.

5.4 Using polyfills

Beyond transpiling syntax, another key strategy for backward compatibility is the use of polyfills. A *polyfill* is a piece of code that provides a modern API on older engines that do not support it natively. Unlike a transpiler, which rewrites syntax before the code is shipped, a polyfill works at runtime by checking if a feature exists and, if not, creating a compatible implementation.

Polyfills are not suitable for syntax changes in the language, such as the usage of new operators. This approach is beneficial in Vanilla Web projects where the goal is to avoid a heavy build step. Instead of converting an entire codebase, you can include small, targeted polyfills only for the specific features you need, ensuring your modern code runs reliably in a broader range of browsers.

NOTE The two most common polyfill sources today are `core-js`, which provides polyfills for ECMAScript features up to 2025, and `es-shim`, a community-driven collection of polyfills designed to closely follow the ECMAScript specifications.

As a quick example, the following polyfill adds the `includes` method to arrays when it is not provided natively by the current engine:

```
if (!Array.prototype.includes) {
  Array.prototype.includes = function (value) {
    return this.indexOf(value) !== -1;
  };
}
```

5.5 ECMAScript Modules

A web app always starts with an HTML document, as we saw in the previous chapter. That HTML document may load and execute one or more scripts. While you can place JavaScript code directly in the HTML, this approach is only suitable for straightforward cases, such as a small snippet or a specific performance optimization. Most of the time, you will load JavaScript from external files using the `<script>` element, the `src` attribute with the URL to load, and, recently, the `type="module"` declaration.

ECMAScript modules (ESM) give JavaScript a native, browser-level module system. Each file is a module with its own scope, and you explicitly share functionality via `export` and consume it via `import`. Modules are fetched by URL, form a dependency graph, and are evaluated once; subsequent imports reuse the same module instance (including its state).

A `<script type="module">` is fetched in parallel and executed after HTML parsing (defer-like). Module scripts run in strict mode by default, support `import/export`, and can use top-level `await`.

A module is a scope by itself, which means that if you declare a variable, a function, or a class, it will be available only in that module, and it will not be attached to the global object. Therefore, those definitions can't be accessed by default by other modules or the main script in the HTML document. But you can decide to export data from a module to other modules, making them visible to different files.

WARNING ES modules obey the platform's CORS rules. If you load a module from another origin, the server must set the appropriate CORS headers; otherwise, the browser will block the request.

From a module, you can always access the global context and attach elements to it through the `globalThis` object.

```
// This is a module
const book = "Vanilla Web";      #1
globalThis.book = "Vanilla Web"; #2
```

#1 only available in this module

#2 globally available

5.5.1 Named exports

Let's start explaining ES Modules with named exports and a default export:

```
// math.js
export function add(a, b) { return a + b; }
export const PI = 3.14159;
export default function square(n) { return n * n; }
```

```
// app.js
import square, { add, PI } from './math.js';

console.log(add(2, 3), PI);    // 5 3.14159
console.log(square(4));        // 16
```

ESM exports are live bindings, which means that if a module updates an exported variable, importers see the new value (it's not a copy).

DEFAULT EXPORTS

A module can also export a single default value (function, class, or object) that will be imported without curly braces:

```
// logger.js
export default function log(message) {
  console.log("[LOG]", message);
}
```

```
// app.js
import log from './logger.js';

log("Application started"); // [LOG] Application started
```

A default export defines what a module is, while a named export represents something the module provides. A file can have only one default export, but it may include any number of named exports alongside it. Many libraries use a default export for the module's primary feature and named exports for supporting utilities.

5.5.2 Importing modules

Relative and absolute URLs work out of the box. When you import elements from other modules, you use the `import` keyword at the top level of the file.

There are several `import` declarations available in one module to receive declarations and values exported from other modules:

```
// Default import
import myThing from './module.js';

// Named import
import { readFile, writeFile } from './fs-utils.js';

// Aliasing named import
import { readFile as read, writeFile as write } from './fs-utils.js';

// Mixing default and named imports
import parser, { tokenize, stringify } from './parser.js';

// Importing everything under one namespace object
import * as utils from './fs-utils.js';
utils.readFile('file.txt');

// Importing for side effects only
// (runs the file but doesn't find anything)
import './setup.js';
```

You can freely combine different import declarations within the same module, or create modules that act as bundlers by importing from many modules and re-exporting everything together.

WARNING When importing from modules in a browser or client rendering engine, the path must start with `http`, `/`, `./`, or `../` unless you register an import map for aliases.

5.5.3 Import Maps

It's possible to avoid the rule of using absolute or relative URLs when importing from modules by using just a string identifier, or bare specifier; for example: `import from "lodash"` or `import * as router from "app-router"`.

To use bare specifiers when importing, we must use a feature known as Import Maps, defining an object in a special `<script>` tag as follows:

```
<script type="importmap">
{
  "imports": {
    "lodash": "https://cdn.example.com/lodash.esm.js"
  }
}
</script>

<script type="module">
  import _ from "lodash";
  console.log(_.chunk([1,2,3,4], 2));
</script>
```

If you're a framework or library developer, you've probably used bare specifiers, like in `import { useEffect } from 'react'` when working with React. In those cases, a bundler such as Webpack is responsible for mapping the bare specifier to a real module.

IMPORT MAPS VS BUNDLERS

Import maps are ideal for small to medium-sized apps where you prefer the browser to resolve modules. For complex production setups (such as code splitting, minification, and tree-shaking), a bundler or build step may still be necessary.

5.5.4 Top-level await

Modules can `await` directly at the top level (no wrapper function), a feature that is useful for configuration or one-time initialization, and it was added in ES2022. For example, we can use a top-level `await` in the following example:

```
// config.js
export const settings = await fetch('/api/settings').then(r => r.json());
```

The `settings` object in the previous example can be imported in other modules, who will have to wait until the awaited promise ends, as in:

```
// app.js
import { settings } from './config.js';
console.log('Ready with', settings);
```

If module A imports module B that uses top-level `await`, A will wait for B to resolve. Avoid long chains of top-level awaits across multiple modules. Fetch what's critical, then lazy-load the rest after first paint.

5.5.5 Preloading and execution order

Module graphs are fetched on demand: when a module is requested, the browser loads it along with all of its dependencies according to the import graph. To reduce startup latency, you can preload a module (and any modules it imports) so the browser begins fetching those resources earlier in the page load by using `modulepreload` link elements.

WARNING Have in mind `modulepreload` fetches the module from the network but doesn't execute it until it's actually imported (by a static or dynamic import). Don't rely on preloads alone to run code.

When using it, the browser will have a hint that you may need that module and it will prepare the download, parse and compilation of it, as with:

```
<link rel="modulepreload" href="/charts.js">
```

Module scripts preserve execution order relative to one another, similar to using `defer` with classic scripts. This ensures that modules referenced later in the document do not run before earlier ones. If you add the `async` attribute to a module script, you opt out of this deterministic ordering: the module (and its dependency graph) will execute as soon as it finishes loading, which may be before other modules on the page.

5.5.6 Lazy loading

Dynamic `import()` loads a module on demand and returns a Promise that resolves to its module namespace object. This technique is ideal for routes, rare actions, code splitting, or heavy widgets, and it was incorporated in ES2020.

In the following HTML code we listen for a button click and load a module dynamically using `await import()`.

```
<button id="show-chart">Show chart</button>
<script type="module">
  const btn = document.getElementById('show-chart');
  btn.addEventListener('click', async () => {
    const { initChart } = await import('./charts.js');
    initChart('#chart'); // render into a container
  });
</script>
```

Preloading modules you may load on demand is a good practice to reduce latency.

5.5.7 Error handling and timeouts for dynamic imports

When using dynamic imports with `import()` we need to remember we are using the network, so treat `import()` like any network operation: handle failures and consider timeouts for a better user experience, as with the following sample

```
// utils/lazy.js
export async function lazyImport(specifier, {timeout=8000}={}) {
  const t = new Promise( (_, rej) =>
    setTimeout(() => rej(new Error('Timeout')), timeout));
  return Promise.race([import(specifier), t]);
}
```

Then, when calling `lazyImport` we can define a desired timeout for it before the promise is rejected.

```
// app.js
import { lazyImport } from './utils/lazy.js';

try {
  const mod = await lazyImport('./feature.js', {timeout: 5000});
  mod.start();
} catch (err) {
  console.warn('Feature unavailable:', err);
  // fall back UI
}
```

5.5.8 JSON modules import

In ES2025, new abilities were added to dynamic import. Modern browsers support importing JSON as a module with import attributes:

```
// app.js
import data from './config.json' with { type: 'json' };
console.log(data.title);
```

If targeting older engines, you can fetch JSON or use a build step that inlines it. Import attributes are standardized, but adoption may lag on some platforms.

5.5.9 Re-exports and module organization

You can structure features by re-exporting from index modules:

```
// features/index.js
export * from './auth.js';
export { store } from './store.js';
export { default as router } from './router.js';

// app.js
import { login, logout, store, router } from './features/index.js';
```

This pattern clarifies public API surfaces and lets you refactor internals without changing import paths across the app. If you use a bundler with tree shaking, prefer explicit named re-exports over deep chains of `export *` barrels for the best results.

5.5.10 Caching, single evaluation, and side effects

Each module URL is evaluated only once. Any subsequent imports of that module return the same namespace object, which means all consumers share the same state and side effects. Because modules can be imported from many parts of your app, it's best to keep top-level side effects to a minimum. Instead of running setup logic automatically when the module loads, expose an explicit `init()` function and call it from `app.js` or another entry point. This makes module behavior predictable, clarifies load order, and avoids unexpected interactions caused by implicit execution.

CLASSIC FALLBACK

For legacy browsers such as Internet Explorer 11, pair a modern module entry with a classic fallback:

```
<script type="module" src="/app.mjs"></script>
<script nomodule src="/app.legacy.js"></script>
```

While this is not common these days and ES modules are widely available in Baseline, there may be some rare and niche use cases where you will still need to comply with ES5.

5.5.11 Practical rules of thumb

When using import, we should follow this practical rules of thumb:

- Use static imports for critical path code; use dynamic `import()` for routes, admin-only tools, large visualizations, or third-party integrations.
- Preload the module you know you'll need soon (user hover, route prefetch), but still call `import()` to execute it.
- Avoid long top-level `await` chains—perform heavy work after first render or lazily.
- If you need bare specifiers without a build step, add an import map and pin versions to stable URLs.

With these patterns, ES modules become your project architecture: a clear, URL-based graph you can reason about, split, and ship incrementally—no framework required.

5.6 Modern JavaScript Essentials

It's now time to review the most prominent features of modern JavaScript that you have to understand to code a Vanilla Web app successfully. We will assume that you are familiar with ES2015 (ES6) code.

When developers say "modern JavaScript," ES2015/ES6 is the turning point that most people are familiar with: `let/const`, arrow functions, template literals, `class`, modules, and Promises. From there, the language continued to evolve with small but impactful features that reduce boilerplate and make Vanilla projects more pleasant to write and maintain. We may also add the `async/await` addition to ES2017 as a base of JavaScript knowledge for most developers these days.

This section focuses on the most prominent post-ES2017 improvements you will use every day.

5.6.1 Optional chaining and nullish coalescing

Two operators from ES2020 dramatically cut defensive code. *Optional chaining* (`?.`) safely walks nested properties without throwing, while *nullish coalescing* (`??`) gives defaults only when a value is `null` or `undefined` (not when it's `0`, `"`, or `false`).

```
const user = { settings: null };

const email = user.profile?.email ?? "anonymous@example.com";
const theme = user.settings?.theme ?? "light";

console.log(email, theme); // "anonymous@example.com" "light"
```

Prefer `??` over `||` when you want to distinguish "empty but valid" values (like `0` or `"`) from "missing" (`null/undefined`).

5.6.2 Logical assignment operators

Local assignment operators are useful when you want to use "assign if missing" or "append if present" in a single step. ES2021 added `&&=`, `||=`, and `??=` to combine short-circuiting with assignment.

```
const state = { count: 0, title: "" };

state.count ||= 1; #1
state.title ??= "Untitled"; #2
state.ready &&= false; #3

console.log(state); // { count: 1, title: " ", ready: undefined }
```

#1 0 is falsy → becomes 1
 #2 "" is not nullish → stays ""
 #3 only assigns if truthy → stays undefined

Use `??=` when you truly mean "only if null/undefined." Use `||=` when you also want to treat empty strings/zeros as "needs default."

5.6.3 Destructuring + rest/spread in practice

Destructuring (ES6) grew into an everyday pattern with object rest/spread (ES2018). It's perfect for extracting what you need and forwarding the rest by using the rest and spread operators (...).

```
const payload = { id: 7, title: "Post", tags: ["vanilla", "js"],
                  draft: true };

const { id, title: name, author="unknown", #1
      ...meta } = payload; #2

const updated = { ...payload, draft: false }; #3

console.log(id, name, author, meta, updated.draft);
// Output: 7, 'Post', ["vanilla", "js"] true, false
```

#1 we create three constants: id, name, author and meta
 #2 meta will have { tags: ["vanilla", "js"], draft: true }
 #3 shallow merge

Object spread is shallow. For nested structures, copy inner objects/arrays as needed to avoid unintended mutations.

5.6.4 Classes with public & private fields

Modern classes have public fields, private fields (denoted by a leading #), private methods/accessors, and static fields. They remove constructor boilerplate and encapsulate internal state without conventions.

```

class Counter {
  #value = 0;           #1
  label;               #2

  constructor(label = "count") {
    this.label = label;
  }

  increment(step = 1) { #3
    this.#value += step;
  }

  get value() {         #4
    return this.#value;
  }

  #log() {              #5
    console.log(`[${this.label}] = ${this.#value}`);
  }

  print() {
    this.#log();        #6
  }
}

const c = new Counter("likes");
c.increment(2);
c.print();             // [likes] = 2
// c.#value;           // SyntaxError: private field not accessible

```

#1 Private field

#2 Public field

#3 Public method

#4 Public getter

#5 Private method

#6 Calling a private method from inside the class

You can see on previous example that there is no way to access `#value` from outside the class.

PRIVATE VS. UNDERSCORE

The `private #value` is enforced by the language (access checks at parse time). Naming a property `_value` is merely a convention and remains publicly accessible.

5.6.5 Cleaner errors: optional catch binding and error cause

ES2019 made `catch` parameters optional; ES2022 added an error cause to preserve context when rethrowing.

```
try {
  // ...
} catch { #1
  console.warn("Non-critical issue");
}

try {
  await doCritical();
} catch (err) {
  throw new Error("Startup failed", { cause: err });
}
```

#1 no (err) needed when you don't use it

Later, you can inspect `err.cause` to debug without losing the original stack.

5.6.6 Small quality-of-life wins

A handful of tiny features add up to cleaner code:

- `Object.fromEntries` / `Object.entries` / `Object.values`: quick object ↔ iterable transforms.
- `Object.hasOwn(obj, key)`: safer than `obj.hasOwnProperty(key)`.
- **Numeric separators**: make big numbers readable: `1_000_000`.
- `String.prototype.replaceAll`: no more global regex for simple cases.
- `String.prototype.matchAll`: iterate over all regex matches without manual loops or index tracking.

```
const pairs = [{"a",1},{"b",2}];
const obj = Object.fromEntries(pairs);
console.log(Object.hasOwn(obj, "a")); // true

const budget = 12_500_000;
console.log("foo-bar".replaceAll("-", " ")); // foo bar
```

5.7 Collection Management

Collections are the backbone of most Vanilla Web apps. Whether you're working with arrays of API responses, sets of unique IDs, or maps keyed by DOM elements, modern JavaScript gives you a toolbox that goes far beyond plain arrays and objects. Let's explore the essential collection features that make daily tasks easier, eliminating the need for external libraries.

5.7.1 Arrays

An Array is an ordered, zero-indexed list used to store multiple values in a single variable. Arrays have grown much more capable since ES6. Beyond the classics (`forEach`, `map`, `filter`), modern versions introduced methods that reduce boilerplate:

- `includes` (ES2016) — checks membership without `indexOf` `>= 0`.
- `find` / `findIndex` (ES2015) — retrieve the first matching item.
- `flat` / `flatMap` (ES2019) — handle nested arrays.
- `at` (ES2022) — access by index with support for negatives.
- Copy methods (`toSorted`, `toReversed`, `toSpliced`, `with`) — introduced in ES2023 for non-mutating operations.

The following code shows some examples of the new Array functions.

```
const arr = [10, 20, 30, 40];
console.log(arr.includes(20)); // true
console.log(arr.find(n => n > 25)); // 30
console.log(arr.at(-1)); // 40
console.log(arr.toReversed()); // [40, 30, 20, 10]
```

When you can choose, prefer the non-mutating copy methods (`toSorted`, `toReversed`, etc.) when you want safer, more predictable data handling without unexpected side effects.

5.7.2 Sets

A *Set* stores unique values and is perfect when you need to avoid duplicates without writing manual checks.

```
const ids = new Set([1,2,2,3]);
ids.add(4);
console.log([...ids]); // [1,2,3,4]
```

Modern `Set` operations (ES2025) make them even more powerful: `union`, `intersection`, `difference`, and `symmetricDifference`.

```
const a = new Set([1,2,3]);
const b = new Set([3,4]);
console.log([...a.intersection(b)]); // [3]
```

5.7.3 Maps

A *Map* is a key-value store like an object, but keys can be any type, not just strings, which makes it ideal for DOM elements, configurations, or caching.

```
const m = new Map();
m.set("user", { name: "Ana" });
m.set(document.body, "root");
console.log(m.get("user").name); // Ana
console.log(m.has(document.body)); // true
```

Maps remember insertion order and have clear iteration APIs (`keys()`, `values()`, `entries()`).

5.7.4 Weak collections: memory-friendly

WeakMap and *WeakSet* are specialized collections where keys are objects only, and they don't prevent garbage collection. They're handy for attaching metadata to DOM nodes or objects without creating leaks.

```
const wm = new WeakMap();
const el = document.querySelector("button");
wm.set(el, { clicked: 0 });

el.addEventListener("click", () => {
  const state = wm.get(el);
  state.clicked++;
  console.log(state.clicked);
});
```

Once `e1` is removed from the DOM and not referenced elsewhere, it, along with its associated metadata, will be automatically cleaned up.

5.7.5 Putting it together

Post-ES6 JavaScript is about removing friction: fewer guards, less boilerplate, and more apparent intent. Optional chaining and logical assignments keep conditionals readable; modern class fields provide real encapsulation; top-level `await` simplifies bootstrapping; and small utilities like `Object.hasOwn` and `replaceAll` make everyday tasks smoother. Use them confidently—they being part of today's Baseline means you can rely on them in Vanilla projects without extra libraries or heavy build steps.

5.8 Functional Programming

Functional programming (FP) can sound intimidating, but in practice, most Vanilla projects already incorporate FP ideas every day. The core principles are simple: write functions that take input, return output, and avoid unexpected side effects. JavaScript is not a purely functional language, but it has embraced many FP techniques since ES6.

If you are coming to Vanilla Web from libraries such as React, you may already be using many functional programming techniques in your code.

5.8.1 Array transformations

The most visible FP style in JavaScript comes from array methods that accept functions as arguments. Instead of writing manual loops, you express your intent with higher-order methods:

```
const nums = [1, 2, 3, 4, 5];
const doubled = nums.map(n => n * 2);           // [2,4,6,8,10]
const evens = nums.filter(n => n % 2 === 0);     // [2,4]
const sum = nums.reduce((acc, n) => acc + n, 0); // 15
```

This style is more declarative: you describe what you want, not how to do it.

5.8.2 Pure functions and immutability

A pure function depends only on its inputs and produces the same output every time. It does not modify the external state. Combined with immutability, this makes your code easier to reason about and understand.

```
function addUser(users, name) { #1
  users.push(name); #1
  return users; #1
} #1

function addUserPure(users, name) { #2
  return [...users, name]; #2
} #2

const list = ["Ana"];
console.log(addUserPure(list, "Bob")); // ["Ana","Bob"]
console.log(list); // ["Ana"]
```

#1 impure: modifies array in place

#2 pure: returns a new array instead

5.8.3 Function composition

Functions can be chained together to create pipelines. While libraries offer helpers for this, plain JavaScript already makes it easy to compose operations.

```
const words = ["Vanilla", "Web", "is", "great"];
const result = words
  .map(w => w.toUpperCase())
  .filter(w => w.length > 2)
  .join(" ");
console.log(result); // "VANILLA WEB GREAT"
```

Each step is small and testable, and the chain reads almost like natural language.

5.8.4 Everyday benefits

You don't need to adopt advanced FP patterns to gain from this style. Using pure functions, immutability, and array methods already improves clarity and reduces bugs. When combined with modern syntax features like optional chaining and spread, the functional style makes your Vanilla Web code shorter, safer, and more predictable — without adding any libraries.

5.9 Error Management

JavaScript applications must handle errors that occur both at runtime and asynchronously. In a Vanilla project, there is no framework layer handling mistakes for you, so understanding the native mechanisms is essential. Errors can arise from invalid input, network failures, DOM operations, or simply from developer mistakes. Consistently managing them keeps the app reliable and prevents silent failures.

5.9.1 Exceptions and try/catch

The most direct way to handle errors is with a `try/catch` block. Any synchronous code inside `try` that throws an error will jump into `catch`. You can add an optional `finally` block to clean up resources, such as timers or connections, regardless of whether the operation succeeds or fails.

```
try {
  const json = JSON.parse("{ bad json }");
  console.log(json);
} catch (err) {
  console.error("Failed to parse JSON:", err.message);
} finally {
  console.log("Parsing attempt finished");
}
```

Starting with ES2019, the `catch` binding is optional, which is useful when the error object is not needed.

5.9.2 Error objects

Errors in JavaScript are instances of `Error` or its subclasses (`TypeError`, `ReferenceError`, `RangeError`, etc.). Creating and throwing your own error instances provides more context and makes debugging easier.

```
function divide(a, b) {
  if (b === 0) throw new RangeError("Division by zero");
  return a / b;
}
```

Errors can now include a `cause` property (ES2022), which helps preserve the original reason when rethrowing an error.

```
try {
  await initApp();
} catch (err) {
  throw new Error("Startup failed", { cause: err });
}
```

Later inspection of `err.cause` reveals the original problem.

From ES2021 `AggregateError` object was added, and it represents an error when several errors need to be wrapped in a single error. It is thrown when multiple errors need to be reported by an operation.

5.9.3 Asynchronous errors

Promises and `async/await` bring their own patterns. Any unhandled rejection will trigger the global `unhandledrejection` event. To avoid noisy logs or silent failures, always explicitly catch promise rejections.

```
async function loadData() {
  const res = await fetch("/api/data");
  if (!res.ok) throw new Error("Network error: " + res.status);
  return res.json();
}

loadData().catch(err => {
  console.error("Data fetch failed:", err);
});
```

When using `await`, wrap the code with a `try/catch` block inside the `async` function to handle errors locally.

5.9.4 Global handlers

For last-resort cases, the browser platform provides global events through the DOM as we will cover in next chapter:

- `window.onerror` catches uncaught exceptions in scripts.
- `window.onunhandledrejection` captures rejected promises without a handler.

They are useful for logging and telemetry, not for controlling program flow.

```

window.addEventListener("error", e => {
  console.error("Global error:", e.message);
});

window.addEventListener("unhandledrejection", e => {
  console.error("Unhandled rejection:", e.reason);
});

```

The global error handlers should not replace structured error management in code. They are intended as safety nets for logging and monitoring.

5.9.5 Practical guidance for error management

As a summary, this is a practical guidance for error management in your Vanilla JavaScript code.

- Use `try/catch` locally around risky operations such as JSON parsing, DOM manipulation, or network requests.
- Throw `Error` instances with clear messages and consider using the `cause` option when rethrowing.
- Always handle promise rejections explicitly to prevent unhandled errors, including `catch` handlers for async operations.
- Use global error events only for diagnostics and reporting.

Managing errors explicitly in Vanilla JavaScript keeps applications predictable and easier to debug without relying on framework-level safety nets.

5.10 Summary

- ECMAScript evolves through TC39 stages and annual editions, with features reaching browsers at different times; compatibility checks remain essential.
- ES modules provide a native way to organize code with static and dynamic imports, import maps, top-level `await`, and preload strategies.
- Modern syntax such as optional chaining, nullish coalescing, logical assignment operators, and private class fields reduces boilerplate and improves clarity.
- Arrays, Sets, Maps, and Weak collections cover most collection management needs, with recent additions like non-mutating array methods and set operations.
- Error handling is essential when building Vanilla apps, including local `catch` and global handlers for unhandled exceptions.
- Vanilla JavaScript and browser APIs together provide the essential tools for building modern applications without large external frameworks.

6 *The Document Object Model API*

This chapter covers

- How the browser maps HTML hierarchy to JavaScript data structures
- Updating the UI without external libraries
- Working with the DOM API in Vanilla JavaScript
- Querying, modifying, styling, and managing events

In the previous chapter, we covered the current state of JavaScript and ECMAScript syntax. Now it's time to look at how browser APIs let us manage the user interface in Vanilla JavaScript through the DOM API.

The *DOM* is the in-memory representation of the document that the rendering engine uses to paint the page. It is a tree of nodes where every element, text node, and comment has a position and relationships to parent, children, and siblings. Understanding that structure is the foundation for manipulating UI with JavaScript.

You do not need a library to work with the DOM. The platform already provides selectors, node constructors, insertion and removal helpers, and a rich event system. Many helpers from the past are no longer necessary today.

Performance matters when changing the DOM. Making many uncoordinated edits can result in additional layout and painting work. Prefer batching changes, using fragments, delegating events, and avoiding forced synchronous layout reads between writes.

This section covers the document model, the most used interfaces, how to query and modify the tree, and the event patterns you will use daily. With these tools, you can build rich interfaces with no external dependencies.

6.1 The DOM tree

Each element in the document is represented by an object of the `HTMLElement` interface or another interface that inherits from it; they have instance properties and methods. Changes in properties or children will trigger updates in the user interface when the browser has a free thread. We can listen to events happening in that element and react in consequence, updating the user interface.

For example, take the structure of the following HTML document:

```
<html>
  <body>
    <header>
      <h1>The DOM</h1>
      <p class="tagline">Document Object Model</p>
      
    </header>
  </body>
</html>
```

When you think about the DOM of the previous HTML document, we can see Figure [6.1](#).

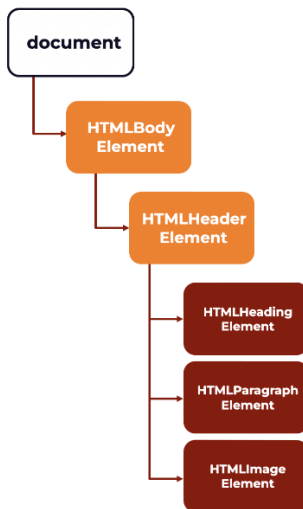


Figure 6.1 The Document Object Model (DOM) starts with a document object and then it creates a tree structure with objects representing each HTML element, plus text and comment nodes

Once we have the DOM element, we can work with it by manipulating its objects and properties to update and dynamically change the UI. For that purpose, we have to:

- Get a reference to those elements by querying the document.
- Create a new element and then inject it into the DOM using the API.

When we have a reference to the element object, we can do many things, such as:

- Read its content
- Change its content
- Update its style
- Remove it from the DOM or move it to a different tree
- Add children elements to it

6.2 The API

The DOM defines a small set of core node types. Document is the root; Element adds attributes, classes, and style; Text carries literal content. Each node knows its `parentNode` and `childNodes`, and Element exposes higher-level APIs such as `classList`, `dataset`, and convenience insertion methods like `append` or `before`.

The DOM tree is created from the parsed HTML and can then be mutated at runtime by JavaScript. Those mutations are reflected on screen by the engine as it synchronizes DOM changes with layout and paint.

NOTE Some collections are live (`HTMLCollection`) and reflect mutations as they happen, while others are static snapshots (`NodeList` from `querySelectorAll`). Converting collections to arrays is a practical way to use `map`, `filter`, and other array utilities safely, for example, in `Array.from(document.getElementsByTagName('a'))`.

When in doubt, prefer the simplest primitive that expresses intent: use `textContent` to set plain text, `createElement` for structure, and dedicated properties on specialized elements instead of parsing attributes manually.

```
const title = document.createElement("h1");
title.textContent = "Hello DOM";
document.body.prepend(title);
```

6.2.1 HTML`Element` interface

`HTMLElement` is the base for most concrete HTML elements and adds beneficial properties such as `innerHTML`, `innerText`, `hidden`, `dir`, and ARIA reflection through `aria-*` attributes. It also provides `classList` for class management, `dataset` for `data-*` attributes, and `style` for inline styles.

Concrete subclasses, such as `HTMLInputElement` or `HTMLAnchorElement`, add domain-specific properties and methods. Using these saves time and avoids string parsing. For example, `input.value`, `input.checked`, `a.href`, `video.currentTime`, and `dialog.showModal` are all high-level features.

Prefer `classList` and `dataset` over string manipulation of `class` and `data` attributes. Inline styles are available, but using CSS classes keeps concerns separate and leverages the cascade and media queries.

```
const btn = document.createElement("button");
btn.classList.add("primary");
btn.dataset.action = "save";
btn.innerText = "Save";
document.body.append(btn);
console.log(btn.dataset.action); // "save"
```

6.2.2 Global DOM objects

The DOM API is available on many objects:

- *window* is the global object for the browsing context. It aggregates timers, viewport and visibility events, history and navigation, and entry points to many Web APIs.
- *document* hangs from *window* and represents the active document, the main HTML in the navigation.
- One object per HTML element and other nodes in your document that you can get a reference to using different techniques.

Not everything global is strictly DOM; some features belong to the browser environment, and others to the language. In practice, *window* is the coordination point for UI, network, and tab lifecycle.

NOTE From ES2020, we have a standard way to access the current context's global object, such as *window* on a standard page navigation through the usage of `globalThis`.

6.2.3 Working with different documents

A page can include other documents via iframes, popups, or documents created in memory. Each has its `document` (even its own `window` sometimes) and its own node tree, even when nested inside another document.

Same-origin policy applies across documents. If an iframe is on a different origin, its DOM is not accessible for security reasons. Communication is still possible using `postMessage` between these documents.

```
const iframe = document.querySelector("iframe");
try {
  const inner = iframe.contentDocument; #1
  inner.body.append("Hello from parent");
} catch {
  iframe.contentWindow.postMessage({ type: "hello" }, "*");
}
```

#1 it works only on same origin

For off-DOM construction, use `DocumentFragment` or the template element. You can also parse HTML into a detached document with `DOMParser` and then move nodes into the live document when ready.

When moving nodes between documents, use `adoptNode` or `importNode` so the destination document owns the nodes before inserting them.

NOTE Every DOM node belongs to exactly one document. If you try to insert a node created in a different document without using `adoptNode()` or `importNode()`, the browser will throw a `DOMException`. `adoptNode()` transfers the node to the destination document, while `importNode()` creates a clone owned by that document. One of these steps is required before inserting cross-document nodes into the live DOM.

6.2.4 Querying the document

There are three types of lookup APIs for querying the document:

- *HTML`Element`*: returns one element or `null` if not found.
- *HTML`Collection`*: returns a live collection that updates as the tree changes.
- *Node`List`*: returns a static snapshot array.

WARNING Because an `HTMLCollection` is a live collection, it acts more like a pointer than a static collection. If you want to use array methods, such as `map`, `filter`, or `forEach`, you have to convert it into an array. It will loop through all the collection items at that time and create a static snapshot for you.

Classic getters, such as `getElementById` and `getElementsByClassName`, are fast and straightforward; querying using CSS selectors, using `querySelector` and `querySelectorAll`, is more expressive. Both can be scoped to any `Element`, not just the document, so that you can narrow a search.

The following table shows the most common APIs available and the return type:

Table 6.1 Most common DOM API functions available

Function	Return type	Argument explanation
<code>getElementById(id)</code>	HTMLElement (or null)	A string with the exact id attribute of the element. Only one element is returned.
<code>getElementsByClassName(classNames)</code>	HTMLCollection	A string with one or more class names separated by spaces. Returns all elements matching all given classes.
<code>getElementsByTagName(tagName)</code>	HTMLCollection	A string with the name of the tag (e.g., "div", "p"). Returns all elements of that tag.
<code>querySelector(selector)</code>	HTMLElement (or null)	A CSS selector string. Returns the first element in document order that matches.
<code>querySelectorAll(selector)</code>	NodeList	A CSS selector string. Returns all elements matching the selector.
<code>document.forms[nameOrIndex]</code>	HTMLCollection / HTMLFormElement	Access forms by index (number) or by name attribute. Returns a form or a collection.
<code>form.elements[nameOrIndex]</code>	HTMLCollection / HTMLElement	Access form controls by index or by name/id attribute. Returns a single control or collection.

Selectors that express intent and decouple structure from styling are preferred. IDs work for unique hooks, `data-*` attributes for behavior selectors, and relative selectors inside a known container minimize global coupling.

Limit search scope whenever possible. Querying inside a container is faster and safer than searching the whole document, and it reads like the actual component boundary in your code.

```
const form = document.querySelector("form#signup");
const required = [...form.querySelectorAll("[data-required]")];
console.log(required.map(el => el.name || el.id));
```

6.2.5 Browsing the tree

Once you have access to one element in the tree, the DOM API lets you browse up and down through parents, children, and siblings. Have in mind that the DOM tree not only includes `HTMLElements` but also other nodes, such as text nodes and comments.

Here you have a table with the most common functions and properties available in an `HTMLElement` instance. You will use them to browse in the DOM structure through properties and methods.

Table 6.2 Most used public methods and properties from HTMLElement to browse the DOM

Property / Method	Return type	Explanation
parentNode	Node (often HTMLElement or Document)	Returns the direct parent node of the element. It can be the document or another element.
parentElement	HTMLElement (or null)	Similar to parentNode, but guaranteed to return an HTMLElement or null if the parent is not an element.
children	HTMLCollection	Returns all child elements (only element nodes, excludes text and comments).
childNodes	NodeList	Returns all child nodes, including text nodes, comments, and elements.
firstChild	Node (or null)	Returns the first child node, which could be a text node, comment, or element.
firstElementChild	HTMLElement (or null)	Returns the first child that is specifically an element.
lastChild	Node (or null)	Returns the last child node of the element.
lastElementChild	HTMLElement (or null)	Returns the last child that is specifically an element.
previousSibling	Node (or null)	Returns the node immediately before the element, including text nodes.
previousElementSibling	HTMLElement (or null)	Returns the element immediately before the element, ignoring text nodes.
nextSibling	Node (or null)	Returns the node immediately after the element, including text nodes.
nextElementSibling	HTMLElement (or null)	Returns the element immediately after the element, ignoring text nodes.
closest(selector)	HTMLElement (or null)	Returns the nearest ancestor (including the element itself) that matches the CSS selector.

6.2.6 Elements available

Each HTML tag has a corresponding interface with high-level behavior. Media elements (from `<audio>` or `<video>`) expose `play` and `pause`; form controls (from `<select>`, `<input>`) expose `value` and `name`; `<details>` elements manage open state and toggle events; `<dialog>` implements accessible modals with `showModal` and `close`.

Using the right native element usually beats recreating it in `div`s. You get accessibility, keyboard behavior, and semantics for free, often with a single attribute or method call.

NOTE In the following chapters, we will explore how to define our own HTML Elements that can enhance the available components used in a web application.

Type checks and feature checks help you write robust code when manipulating heterogeneous nodes. Using `instanceof` and property existence checks allows you to safely handle each element type according to its capabilities.

Lean on the element's own API first. For example, `a.download` provides file download hints, `input.files` provides the `FileList`, and `meter`, `progress`, or `range` controls express numeric status without custom widgets.

```
const media = document.querySelector("video, audio");
if (media instanceof HTMLMediaElement) {
  media.volume = 0.5;
  media.play();
}
```

6.2.7 Modifying elements

Once we have an element instance, there are many ways to change its content:

- Defining `innerHTML`, which replaces content by parsing HTML, which is handy for templates, but risky if it includes user input
- Defining `textContent` sets plain text safely and is ideal for untrusted content
- Changing property values
- Changing styles or classes
- Add or remove children

For dynamic UI, prefer creating nodes and inserting them with `append`, `prepend`, `before`, `after`, or `replaceWith` instead of abusing `innerHTML` that adds security risks that may inject unwanted elements. The first technique is explicit, composable, and avoids reparsing HTML blobs. Finally, `classList` and its `toggle` method make visual states clear and minimize the use of inline styles.

```
const card = document.createElement("article");
card.className = "card";
card.innerHTML = `

## 


```

6.2.8 Properties vs. attributes

Attributes live in the HTML markup; properties live on the JS objects. During parsing, some properties are initialized from attributes, but they can diverge as the page runs.

```
const input = document.createElement("input");
input.setAttribute("value", "A");
document.body.append(input);
input.value = "B";
console.log(input.getAttribute("value")); // "A"
console.log(input.value);                // "B"
```

Form controls are the classic example: the attribute value is the initial value; the property value changes as the user interacts with it. The same applies to the checked or selected state in inputs and options.

WARNING Be mindful that properties and attributes are not always identical. Properties of an `HTMLElement` object reflect dynamic state and can be changed at any time by JavaScript, while attributes are always strings and represent the initial markup in the HTML.

Not every attribute has a direct property, and names or types may differ. Properties can coerce types to booleans or numbers, where attributes remain strings.

A *reflected attribute* is one where the attribute and the corresponding property stay in sync: changing the property updates the attribute, and changing the attribute updates the property. Examples include `id`, `value` on many form elements, and boolean attributes like `disabled` or `open` (where the property is a boolean but the presence/absence of the attribute represents the state).

A *non-reflected attribute* has no automatic linkage to a property. Updating the DOM attribute does not update the element's internal property, and vice versa. For these, you should use `getAttribute` and `setAttribute` because the property accessors won't reflect changes. Common examples include `data-*` attributes (reflected only through `dataset`), ARIA attributes, and many custom attributes.

NOTE The methods `getAttribute` and `setAttribute` are useful for non-reflected attributes; prefer direct properties when available for reflected ones.

For data attributes, the `dataset` reflects attributes as strings with camelCase properties. Changing the `dataset` updates the corresponding `data-*` attribute and vice versa.

6.2.9 Creating elements

The methods `createElement` and `createTextNode` are the safest way to build new UI in HTML.

In Chapter 9, we will also introduce the `<template>` element, which allows you to declare inert markup in HTML and clone it on demand.

NOTE When inserting many nodes, build them into an in-memory buffer using `DocumentFragment` to avoid intermediate reflows.

6.2.10 Element arrangement

Additionally, there are several APIs available to add, remove, and move elements in the DOM tree. Here you have a table of the most common functions available for arrange elements in the DOM:

Table 6.3 Most common methods to arrange elements in the DOM

Method	Return type	Explanation
<code>parent.append(childOrString)</code>	void	Appends one or more nodes or strings as the last children of the parent element.
<code>parent.prepend(childOrString)</code>	void	Inserts one or more nodes or strings as the first children of the parent element.
<code>element.before(nodeOrString)</code>	void	Inserts nodes or strings immediately before the element, as siblings.
<code>element.after(nodeOrString)</code>	void	Inserts nodes or strings immediately after the element, as siblings.
<code>element.replaceWith(nodeOrString)</code>	void	Replaces the element with the given node(s) or string(s).
<code>element.remove()</code>	void	Removes the element from the DOM.
<code>parent.insertBefore(newNode, referenceNode)</code>	Node	Inserts <code>newNode</code> before the <code>referenceNode</code> child of the parent. Classic DOM method.
<code>parent.replaceChild(newNode, oldNode)</code>	Node	Replaces <code>oldNode</code> with <code>newNode</code> in the parent's children. Returns the replaced node.
<code>parent.removeChild(childNode)</code>	Node	Removes <code>childNode</code> from the parent and returns it.
<code>element.appendChild(childNode)</code>	Node	Appends a node as the last child of the element. Returns the appended node.
<code>element.cloneNode(deep)</code>	Node	Creates a copy of the element. If <code>deep = true</code> , it also copies descendants.

Moving a node relocates it; it does not copy it. Reordering lists by moving existing nodes preserves state and event listeners, unlike recreating nodes from scratch.

NOTE When rearranging multiple nodes, perform the work in a fragment or detached container, then attach it once to reduce layout recalculations and improve perceived performance.

6.2.11 Custom properties with dataset

Data attributes attach small bits of metadata to elements without inventing new attributes. JavaScript exposes them through `dataset`, always as strings.

They are helpful for behavior hooks, IDs, flags, or tiny configurations. Avoid storing large JSON blobs in the dataset; use in-memory structures for complex state.

NOTE Changing the dataset updates the underlying attribute and is visible in DevTools, which helps with debugging. Use kebab-case in HTML and camelCase from JavaScript, so for example, `data-user-name` as an HTML attribute becomes `dataset.userName` in JavaScript.

Do not confuse `data-*` with CSS custom properties. `data-*` is DOM data; CSS variables live in style and are consumed from CSS.

```
const item = document.querySelector("[data-id]");
console.log(item.dataset.id);
item.dataset.state = "active"; #1
```

#1 writes `data-state="active"` in the HTML markup

6.2.12 Working with styles

Classes are the preferred way to apply styles in a large Vanilla project. They separate concerns, keep CSS powerful, and work with the cascade and media queries. Toggle classes to reflect state and let CSS define visuals.

Inline style is suitable for one-off, dynamic values such as computed sizes or coordinates. For final computed styles, `getComputedStyle` returns what the engine actually applied.

As we covered in Chapter 4, CSS custom properties (variables) bridge JS and CSS declaratively. From JS, you can set them on any element or on `:root`, and CSS can reference them everywhere with `var()`.

If style depends on state, prefer toggling classes or data attributes and let CSS handle the changes to keep JavaScript simple, making rendering predictable and fast.

```
const root = document.documentElement;
root.style.setProperty("--accent", "#0a84ff");
document.body.classList.toggle("sidebar-open", true);
console.log(getComputedStyle(root)
    .getPropertyValue("--accent"));
// Output: #0a84ff
```

CSSOM API

The *CSS Object Model (CSSOM)* exposes stylesheets and rules at runtime. `document.styleSheets` lists all stylesheets; each `CSSStyleSheet` exposes rules and methods to insert or remove rules.

Constructable stylesheets allow you to create `CSSStyleSheet` objects in JavaScript and adopt them via `adoptedStyleSheets` for modular styling and multi-document scenarios.

NOTE Direct rule manipulation is powerful but fragile. Prefer classes, data attributes, or CSS variables when possible. Use CSSOM when you must generate or share rules programmatically. Inline styling can be avoided through a CSP rule to increase security.

For diagnosis, `getComputedStyle` remains the simplest way to understand final styles. For advanced modular styling, `adoptedStyleSheets` maintains clean code across components and documents.

```
const sheet = new CSSStyleSheet();
sheet.replaceSync(`
  :root { --accent: #ff2d55 }
  .card{ border-color: var(--accent) }
`);
document.adoptedStyleSheets = [...document.adoptedStyleSheets,
    sheet];
```

We will see more styling management from JavaScript in the following chapters.

6.2.13 Working with events

Events are the way the browser notifies our code that something has happened: a user clicks, a form changes, a video finishes playing, or the page becomes hidden. Every interaction and many system-level updates are modeled as events, and handling them is at the core of making a web app interactive. As web developers, we listen to those events using functions known as *event listeners*.

The DOM defines a unified event system where event listeners can be attached to nodes, events can bubble up or be captured down the tree, and default browser actions can be canceled or extended. With a small set of consistent APIs, you can listen, filter, and respond to changes without external libraries.

NOTE Some events trigger default browser actions, such as following links or submitting forms. The DOM API supports a `preventDefault` method on the event object that cancels the default behavior when allowed. `stopPropagation` and `stopImmediatePropagation` control the propagation of the event.

There are two main techniques to listen for events:

- The `on*` properties
- Event listeners

ON* PROPERTIES

Each element exposes convenience properties, such as `onclick`, `oninput`, and `onload`, for each event that the component supports. Assigning to them is the simplest way to listen to an event, but only one handler can be attached per event type per element using this technique.

Reassigning overwrites the previous function, and you do not get options like `capture`, `passive`, or `once`, which makes `on*` properties more suitable for small demos and prototypes.

NOTE The spec's naming pattern is to use lowercase with no word separator for event names (as in `loadeddata`, `click`, or `loadedmetadata`). That's why we can end up with names like `webkitcurrentplaybacktargetiswirelesschanged`. There are some exceptions in the spec, such as in `DOMContentLoaded`. If you are coming from JSX, as in a React application, a different pattern is used there, such as `onClick`, `onSubmit`.

In real apps, prefer `addEventListener` for composition and control. You can combine both approaches if needed, for example, using `'onload'` for a quick hook and `addEventListener` for the rest.

Keep a consistent style in your codebase. Mixing too many patterns to attach handlers makes maintenance harder and complicates the teardown process.

```
const link = document.querySelector("a[href]");
link.onclick = (e) => {
  e.preventDefault();
  console.log("Simple click handler");
};
```

EVENT LISTENERS

For effective event management, we must utilize event listeners that adhere to the Observer (also known as Publish-Subscribe) design pattern.

Then, `addEventListener` supports multiple handlers for the same event on the same node (choosing the capture or bubble phase, as covered later), and options such as `passive` and `once`. It is the recommended API for Vanilla Web projects. The API includes `removeEventListener` that requires the same function reference and compatible options.

This method includes some advanced event management features, such as using `passive: true` for `scroll` and `touchstart` events when `preventDefault` is not called (this allows the browser to continue scrolling smoothly) and `once: true` for one-time handlers that should automatically remove after being fired.

```
function onScroll() {
  console.log(window.scrollY);
}
window.addEventListener("scroll", onScroll, { passive: true });

// ... later
window.removeEventListener("scroll", onScroll);
```

EVENT PHASES

Events travel in two phases: capture and bubble. Using capture means the event goes from the window down to the target element, while bubble means it goes back up from the target to the window.

Understanding the phases helps you choose where to attach listeners. In Figure [6.2](#), you can see a quick example of how they work on our previous example. Let's say the user clicks in the paragraph, it will fire `click` on the `<p>` first in the bubbling phase, followed by its parent `<header>` up to the document. Then it executes the same event from the top to the `<p>` again, but in the capture phase. Anyone can call `stopPropagation()` at any time and stop the chain execution.

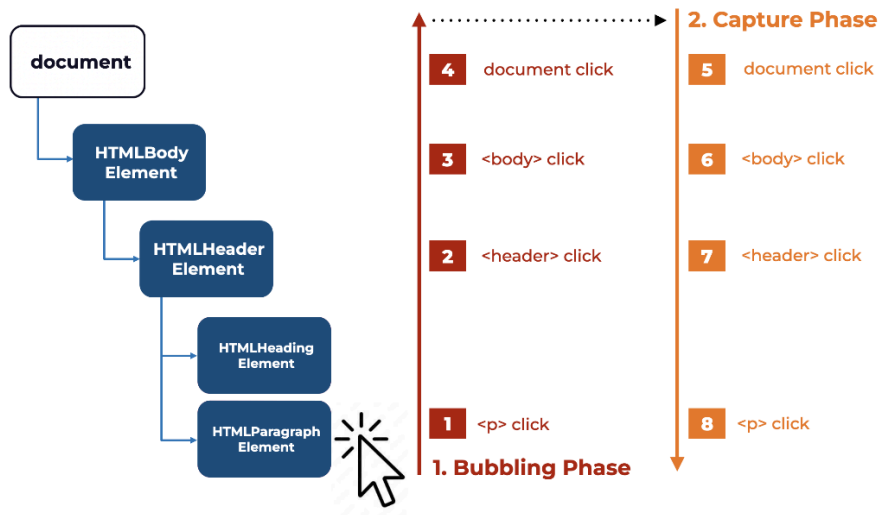


Figure 6.2 We can see a diagram of a DOM and what happens when the user clicks in a Paragraph element. Several listeners can listen to the same event; they will be executed in order at different moments in different phases.

In the following example, we may execute a listener when you click on the paragraph on all these listeners that you can see in the DOM structure in Figure [6.2](#) with the step numbers:

```

const header = document.querySelector("header");
const p = document.querySelector("p");

p.addEventListener("click", () => #1
  console.log("1 p bubble")); #1
header.addEventListener("click", () => #1
  console.log("2 header bubble")); #1
document.body.addEventListener("click", () => #1
  console.log("3 body bubble")); #1
document.addEventListener("click", () => #1
  console.log("4 document bubble")); #1

// Capture phase
document.addEventListener("click", () => #2
  console.log("5 document capture"), { capture: true }); #2
document.body.addEventListener("click", () => #2
  console.log("6 body capture"), { capture: true }); #2
header.addEventListener("click", () => #2
  console.log("7 header capture"), { capture: true }); #2
p.addEventListener("click", () => #2
  console.log("8 p capture"), { capture: true }); #2

```

#1 Event listeners in the bubbling phase

#2 Event listeners in the capture phase

When attaching a listener to a container instead of each child, the event will fire for every click inside that container. That means you must check whether the element that triggered the event is the one you care about. The `event.target` property tells you the actual node that originated the event (for example, the `<p>` that was clicked), while `event.currentTarget` always refers to the element that owns the listener (the container, such as the `<header>`). Using `target` allows you to filter interactions down to the correct node, while `currentTarget` is useful when you need a reference back to the container itself.

NOTE Delegate events to a container when possible to reduce the number of listeners. Suppose you have a list of 1,000 customers and want to attach a click listener to each one. In that case, it is better to attach one listener to the container (for example, a `<ul>`) than a thousand listeners to each element (for example, 1000 `<li>`).

Here you have a quick example of listening to events in the container:

```

<ul id="list">
  <li>Ana</li>
  <li>Bob</li>
  <li>Cara</li>
</ul>

<script>
  const list = document.getElementById("list");

  list.addEventListener("click", e => {
    console.log("target:", e.target); #1
    console.log("currentTarget:", e.currentTarget); #2
  });
</script>

```

#1 `e.target` is the clicked element, such as each `<li>`

#2 `e.currentTarget` is always the `<ul>`

CUSTOM EVENTS

`CustomEvent` enables you to define your own event names with an optional detail payload, allowing you to communicate between modules or components without creating direct dependencies. We select a node that will carry the event and fire `dispatchEvent` on it, which emits the event. Listeners receive an event object with type, target, and detail, just like native events. Naming and documenting the detailed contract keeps integrations clear.

Custom events work well as a simple event bus pattern when dispatched from a common ancestor, such as a document or a dedicated element. They are testable and easily observable in the Developer Tools. We will use them later in this book.

```

const bus = document.body;

bus.addEventListener("cart:add", (e) => {
  console.log("Add to cart:", e.detail);
});

bus.dispatchEvent(new CustomEvent("cart:add", {
  detail: { id: "SKU-123", qty: 1 },
  bubbles: true,
  composed: true
}));

```

Do not over use custom events over global objects in the DOM. Use them for important global events in your application.

6.3 Browser Web APIs

The DOM is only one piece of what the browser offers. The real power of the web platform stems from the hundreds of APIs exposed globally through objects such as `navigator`, `window`, and `document`. These APIs enable you to perform a wide range of tasks, from copying to the clipboard to controlling media playback and accessing sensors. The good news is that they are built in, widely available, and usually standardized across major browsers. You need to learn how to call them.

Many APIs require a secure context. That means you need to serve your app over HTTPS (or from localhost in development). Without it, features like geolocation, camera access, or notifications will fail silently or throw errors. Always test on a secure origin early to avoid surprises later.

As you explore these APIs, please note that availability may vary. Some are Baseline and guaranteed to work everywhere (like `fetch` or `localStorage`). Others are experimental and might only work in the latest browsers. MDN Web Docs maintains a complete list with compatibility tables: <https://developer.mozilla.org/en-US/docs/Web/API>

NOTE A Virtual DOM is an in-memory representation of the UI used by some libraries and frameworks to optimize updates. Instead of modifying the real DOM directly on every state change, the library builds a lightweight virtual tree, computes the differences between the previous and next version (a diff), and applies only the minimal set of changes to the actual DOM. This approach was originally designed to work around performance limitations of older browsers, where frequent DOM operations were expensive. Modern browsers have significantly improved DOM performance, so while the Virtual DOM is still useful in some architectures, it is no longer a universal requirement for high-performance UI updates.

6.4 Summary

- The DOM is the browser's live tree of nodes representing HTML, where every element, text, and comment has structure and relationships; manipulating these objects directly is how Vanilla JS updates the UI without external libraries.
- `HTMLElement` and its subclasses expose high-level behavior like `classList`, `dataset`, ARIA reflection, and form or media properties, so relying on native element APIs avoids brittle attribute parsing and unnecessary custom widgets.
- Each document (main page, iframes, popups, detached fragments) has its own DOM under same-origin rules; `DocumentFragment`, `template`, `DOMParser`, `adoptNode`, and `importNode` let you build and move UI safely across boundaries.

- DOM lookup and traversal combine selectors, live and static collections, and parent or sibling navigation; scoping queries to containers and converting collections to arrays keeps code predictable and avoids global coupling.
- Updating content means choosing between properties and attributes, using `textContent` for safety, `innerHTML` for controlled templates, and explicit node creation with `append`, `before`, or `replaceWith` to avoid unnecessary reparsing and reflows.
- Styling from JS works best by toggling classes or data attributes while delegating visuals to CSS; custom properties handle dynamic values, and CSSOM or constructable stylesheets cover modular or programmatic styling needs.
- Events follow a capture and bubble model; `addEventListener` with options like `once` and `passive` is preferred, delegation reduces listener overhead, and `CustomEvent` provides a simple and decoupled communication channel.
- Browser APIs exposed through `window`, `document`, and `navigator` go far beyond the DOM, powering media, sensors, storage, and more, often requiring secure contexts, so feature detection and compatibility checks remain essential.

7 *Web Components*

This chapter covers

- The component design pattern
- Breaking a user interface into reusable units
- Building Vanilla Web Components
- A walkthrough of the Custom Elements API and Shadow DOM
- How to use CSS to style components

In the previous chapter, we covered how to use the DOM API to interact with a vanilla web user interface. In this chapter, we will begin with one of the most significant additions to the DOM API for creating maintainable, reusable applications: web components.

7.1 The component design pattern

The component design pattern breaks a user interface into small reusable parts that encapsulate structure, behavior, and style. They are everywhere in the app development world: in mobile native apps, desktop apps, and UI libraries. On a Vanilla web project, rather than sprinkling pieces of markup and script throughout a page, you define a component once and then reuse it wherever you need it.

Web Components are custom HTML elements that behave like built-in tags. They solve long-standing problems around code reuse by providing a built-in way to package markup, behavior, and styling together.

They live alongside native elements, participate in the DOM and event system, and can be styled and scripted like any other node. Designing your UI as components encourages separation of concerns: the markup lives with the logic that manages it, and parent pages use the element through a well-defined interface (attributes, properties, and events). When a component is self-contained, you can drop it into any page without worrying that global CSS or JavaScript will break it.

In short, a web component is a new HTML tag you create on top of the built-in browser capabilities.

WARNING As a web developer, it's essential to understand that there is no such thing as a Web Components API in the browser. Several APIs and specs, used together, in whole or in part, let us implement the Web Components design pattern.

The importance of the component pattern becomes evident as an application grows. Large pages become difficult to maintain when every feature modifies the same global DOM. By encapsulating functionality into self-contained components, you can reuse them across projects, test them in isolation, and reason about them independently. Long-lived standards also need to be forward-compatible: features must last for decades and anticipate the future evolution of the Web. Native components fulfil these requirements because they are defined by web standards rather than tied to a framework.

INTEROPERABILITY

Another reason web components matter is interoperability. When you write a component, it tends to be long-lived and portable, and you can use it with any framework. Web Components are independent of any particular library, which means you can share a rating widget or modal dialog across projects regardless of whether the surrounding code uses jQuery, React, Angular, or no framework at all. This ability stands in contrast to framework-specific components that cannot be easily consumed outside the stack for which they were built.

7.2 Use cases

Let us look at a few quick examples of what you might build.

A simple `<popup-info>` component can wrap an icon and a text string and show a tooltip when the icon is focused. A `<word-count>` element could count the words in its children and display the total. A `<rating-widget>` could expose a star rating UI and dispatch events when the user selects a score. A `<price-label>` could render a price with two decimals and a currency symbol.

When creating an e-commerce website, you will probably have several components that you use along with the app, such as `<product-list>`, `<product-item>`, `<product-cart>`, `<product-cart-item>`, `<product-image-showcase>`, and so on. A library can expose `<book-catalog>`, `<book-header>`, `<book-details>`, `<authors-list>`, and `<author-details>`

These examples illustrate two categories of use cases:

- *Generalizable components*, such as tabs, rating widgets, or comboboxes, data formatters that extend what HTML can do across many projects.
- *Project-specific components* that encapsulate a piece of a particular application using the semantics of its entities.

WARNING While we use the term "web component", the browser API uses "custom element" to define it.

Implementing such a component is straightforward: define a class that extends `HTMLElement`, register it with `customElements.define()`. Listing 7.1 creates an autonomous `<hello-world>` element that renders a greeting:

Listing 7.1 Our first custom element

```
// hello-world.js
class HelloWorldElement extends HTMLElement {
  connectedCallback() {
    // set the text when the element is inserted into the document
    this.textContent = `Hello, ${this.getAttribute('name')} || 'World'!`;
  }
}

// register the element (must include a dash in the name)
customElements.define('hello-world', HelloWorldElement);
```

You can then use this element in any HTML page by importing the module and writing `<hello-world name="Vanilla"></hello-world>` in your markup. Even though the component is implemented in JavaScript, it behaves like a standard HTML tag and can be styled or manipulated like any other element.

NOTE If you are coming to Vanilla from a library such as React, you may be wondering whether there is a simpler syntax for defining a component than using a class declaration. We are in the world of JavaScript, a flexible language, so we can quickly create a utility function to make that happen, as we'll see later in this chapter.

7.3 Advantages

Web Components offer several advantages over ad hoc script-driven markup.

First, they can provide real encapsulation by attaching a shadow DOM to your element. We will cover this technique later in the chapter, and it will let you create a private subtree whose internal elements and styles are hidden from the page. This ability prevents global CSS from leaking into the component and likewise stops the component's styles from affecting the rest of the page.

Second, components promote reusability and interoperability. A well-designed component can be reused across projects and even across different frameworks because it speaks the language of the Web: HTML, CSS, and JavaScript. You can use a custom element in a React or Angular project without writing a wrapper, and you can share it between teams. Because the implementation runs in the browser, there are no framework dependencies or bundle overhead.

Third, standard components integrate seamlessly with the platform. They participate in the regular event flow, can be selected with CSS selectors, support accessibility attributes, and can be inspected in dev tools. They also take advantage of browser optimizations such as streaming and incremental rendering. Unlike framework components, there is no virtual DOM or diffing overhead.

7.4 Challenges

Despite their benefits, Web Components are not a silver bullet. The Custom Elements API can be verbose compared with the declarative syntax offered by frameworks. Authoring components without a library like Lit or Stencil can be tedious and prone to mistakes. You must manually reflect attributes, maintain state, update the DOM, and dispatch events. Because there is no built-in reactivity, you often need to write extra code to respond to property changes. You can mitigate these challenges by creating your own micro-library.

While basic support for web component APIs is now in Baseline and compatible with other browsers, as with every evolution on the Web, some APIs, such as form-associated custom elements, may be under active development and may have partial support.

Finally, Web Components can be more complex to use for reactive templating. Components primarily benefit generalizable elements, but are still painful to use for project-specific reactive UI. In Chapter 11, we will discuss data management and reactivity in Vanilla projects.

7.5 Vanilla web components

Vanilla Web Components rely on a handful of browser APIs that work together to provide a component model. At a high level, these include:

- *Custom Elements*: The `CustomElementRegistry` enables you to register new tags or extend existing ones. You create a class that extends `HTMLElement` (for autonomous elements) or another built-in element and register it with `customElements.define()`.
- *Shadow DOM*: The `Element.attachShadow()` method attaches a shadow root to a host element, creating a private DOM subtree. This tree is hidden from the page DOM and provides encapsulation for markup and styling.
- *HTML templates*: The `<template>` element allows you to declare inert markup that can be cloned and inserted into your component.
- *Element Internals*: The `ElementInternals` API method allows form-associated custom elements to participate in form submission, define validity/state (valid/invalid), form value, and ARIA hooks.

Templates will be covered in detail in Chapter 8, but they are an integral part of the component story. Element Internals will be covered later in Chapter 9.

These APIs give you the building blocks to implement the component pattern without imposing a framework. Libraries and micro-libraries such as Lit or Stencil build on top of these primitives to provide templating syntax, reactivity, and boilerplate reduction.

NOTE A web component is expressed in at least one JavaScript file, but it can reference other files, including HTML, JavaScript, CSS, and resources. In Chapter 9, we will cover practical distribution workflows and real publishing scenarios.

7.5.1 Architecture

At its core, a Web Component consists of a host element, an optional shadow DOM tree, and a set of public interfaces. The *host element* is the custom tag that appears in the DOM and is implemented by a class extending `HTMLElement`. If you attach a shadow DOM root, its tree contains the markup and styles that render the component. This tree is hidden behind a boundary that prevents accidental interaction; we will go deeper into the shadow DOM later in this chapter.

The public interface of a component is defined by its attributes, properties, methods, and events.

- Attributes configure behavior declaratively in HTML.
- Properties provide a typed API in JavaScript.

- Methods perform actions such as resetting a counter or opening a pop-up.
- Events notify consumers of state changes.

Together, these form a contract between the component and its host page.

When the browser encounters your custom tag, it instantiates the associated class and calls its constructor. In the constructor, we initialize state and attach a shadow root if needed. We then build the DOM in a method by either cloning a template (as we will see in Chapter 9), creating elements imperatively, or setting `innerHTML`. We set up event listeners and prepare any external resources. Later, when attributes change or the element is removed, the respective lifecycle callbacks run.

7.5.2 The puzzle

A web component is made of different pieces and concepts:

- *Custom element*: A new HTML element in the browser that we define and register.
- *Interface*: Different mechanisms to set data to the component and to read data and changes from it.
- *Document DOM* or *Page DOM*: Refers to the DOM outside of the custom element declaration, in the web component's consumer.
- *Shadow DOM*: A DOM tree within a custom element that is hidden from the parent's container.
- *Host element*: On a shadow DOM, it's the boundary: it's the element that separates the document DOM (outside) from the shadow DOM (inside).
- *Part*: A way to expose part of your inner shadow DOM tree to the outside for styling through a name.
- *Template*: Inert HTML that can be cloned, as a template, in every instance of our component.
- *Light DOM*: A DOM tree that the web component's consumer adds in the global document as children of the custom element, when used with a shadow DOM.
- *Slot*: A placeholder in your shadow DOM (inside) that displays content from the light DOM (outside as the custom element's children).

We will go deeper into each of those concepts in this chapter and the following chapters.

7.5.3 Abilities

Web Components unlock several capabilities that were not possible with plain HTML alone:

- Create your own tag
- Encapsulate markup and CSS under a shadow DOM
- Project markup into designated areas in a component's tree
- Expose parts of a component for styling

- Communicate data to components with attributes and properties
- Communicate events from the components
- Listen for lifecycle events
- Extend the form API

We will expand into all these abilities during the following chapters.

7.6 Custom Elements API

The *Custom Elements API* is the beginning of a web component. It enables developers to define new HTML elements with custom behavior and to extend existing ones. At runtime, the browser discovers these elements in the document and instantiates the associated class, allowing your code to run in response to lifecycle events. The API was designed to be both powerful and straightforward, exposing only a handful of methods and callbacks so that you can progressively enhance your markup without a framework.

There are two kinds of custom elements:

- *Autonomous custom elements*: inherit directly from `HTMLElement` and define completely new tags (for example, `<popup-info>`). Autonomous is the most common kind for most of our work with web components.
- *Customised built-in custom elements*: extend the behavior of existing tags such as `<p>` and you define the extension through the `is` attribute as in `<p is="word-count">`.

For both kinds, you implement the behavior in a class and register it with the browser.

WARNING Customised built-ins custom elements are currently not supported in all browsers, so most developers build autonomous custom elements. Safari has no plans to support it in the future. We will cover customized elements in Chapter 9.

A custom element interacts with the page through lifecycle callbacks, attributes, and events. When the browser parses the document and encounters your tag, it calls the element's constructor (which must call `super()`), then calls `connectedCallback()` when the element is inserted into the DOM. If the component is removed or moved, other callbacks fire. By implementing these hooks, you can create rich interactive elements that feel like native HTML controls.

Not every custom element can fit into the pattern of a web component, but every web component uses a custom element.

7.6.1 Creation

Creating a custom element starts by defining a class that inherits from `HTMLElement` for autonomous elements. In the constructor, you set up the initial state and call `super()`. You can initialize async tasks or internal properties. You should not examine attributes from your HTML node or child nodes here, as they may not yet be available, as a custom element has a lifecycle. When the constructor is executed, the engine hasn't started parsing your element yet.

```
class MyCustomElement extends HTMLElement { #1

  constructor() {
    super(); #2
    // set up initial state
    // do not use attributes or child nodes
  }

}
```

#1 Autonomous element

#2 A constructor is not mandatory, but if present, it must call `super()` first

To access your HTML attributes, nodes, and set your content, use the `connectedCallback()` method. Here you perform DOM manipulations, inject HTML into your element, attach a shadow root, and register event listeners. Because we need to set our context within the component itself, we use `this` to reference our element. Because it's a subclass of `HTMLElement`, all the DOM APIs we've seen in Chapter 6 are exposed on our object.

```
class BitcoinSymbol extends HTMLElement {

  connectedCallback() { #1
    this.innerHTML = "&#8383;"; #2
  }

}
```

#1 `connectedCallback` is defined in the superclass.

#2 We change our HTML content with a Unicode character.

If we save the contents of `BitcoinSymbol` in a JavaScript file, module, or `<script>` tag injected into an HTML document, the class will be ready to use. However, it's not registered in the browser, so it can't be used as a custom element.

7.6.2 Rendering content

There are several strategies to render a custom element's content:

- Imperative DOM creation. Create DOM nodes programmatically in the constructor or `connectedCallback()` using `document.createElement()` and append them to the shadow root. This approach gives you fine control and avoids parsing a template.
- String-based `innerHTML`. Set `innerHTML` to a string containing your markup and styles. This technique is the most straightforward, but it may be less efficient because it reparses the string on each instantiation. Be cautious when using user-supplied content to avoid XSS (Cross-Site Scripting).
- HTML templates. Define a `<template>` element either in your HTML document or in the component module. Clone its `content` and append it to the shadow root. This declarative approach keeps your markup readable and separates it from your JavaScript. Chapter 8 covers templates and slots in detail.
- External HTML file. You can load an external HTML file and inject it into your DOM using `innerHTML` or DOM APIs. We will cover these techniques in later chapters.

WARNING When using `innerHTML`, never insert unsanitized user input directly. Code like `this.shadowRoot.innerHTML = userInput` creates a cross-site scripting (XSS) vulnerability that allows attackers to inject malicious scripts. For text-only content, use `textContent` instead, which safely escapes HTML. If you must render user-supplied HTML, sanitize it first using a library such as DOMPurify (<https://dompurify.com>). This applies equally to shadow DOM and page DOM contexts.

7.6.3 Registration

To make the element available to the page, we must call `customElements.define(name, constructor, [options])`. The `name` of your element must include a hyphen and not be previously registered. The constructor is the class or constructor function that defines the component's contents and behavior.

The optional `options` object has a single property `extends` used to register a customised built-in element. We will cover this technique in Chapter 9.

NOTE The W3C and WHATWG, organizations behind the evolution of HTML and related markups such as SVG and MathML, have agreed not to use hyphens in future specifications for element names to avoid conflicts with custom elements.

For example, to register our Bitcoin symbol, we need to execute

```
customElements.define('symbol-bitcoin', BitcoinSymbol);
```

Now, we are ready to use it in our HTML contents as `<symbol-bitcoin></symbol-bitcoin>`.

WARNING Custom elements can't be used as self-closing HTML tags, as in `<symbol-bitcoin />`, so you must provide the explicit closing tag.

It's also available for use with JavaScript using the DOM API, for example:

```
const bitcoin = document.createElement('symbol-bitcoin');
document.getElementById("container").appendChild(bitcoin);
```

You can also use the `new` keyword and the class name instead of the element tag name. While you can send arguments to the constructor using this technique, it's not recommended, as those arguments will always be undefined when instantiated from the HTML. It's safer to stay with attributes and properties.

```
const bitcoin = new BitcoinSymbol();
document.getElementById("container").appendChild(bitcoin);
```

No matter if you use HTML or JavaScript to create your element, the browser will create a new instance of our `BitcoinSymbol` class, execute its constructor if available, and when connected to a DOM (for example, when we use it in the `appendChild` call), it will execute the `connectedCallback` method automatically.

Have in mind that if you don't register your custom element and use it in an HTML document, the browser doesn't trigger any error, and it will just render an empty `HTMLElement` instead, which will be available in the DOM without any content or reference to your class.

NOTE In Chapter 9, we will get into the details of the component's lifecycle. Still, we can now say that if you instantiate a custom element in JavaScript, it won't execute `connectedCallback` until you effectively attach it to a real DOM, as in using `appendChild` or `insertBefore`.

Register custom elements as soon as their definition is loaded. Because `customElements.define()` can only be called once per name, wrap the call in a module so that it executes at import time. Avoid registering elements conditionally; this can lead to subtle race conditions if some code tries to create the element before it is defined. If your application injects markup containing undefined custom tags, call `customElements.upgrade(container)` after registration to upgrade the existing instances.

NAMING YOUR WEB COMPONENT

There are a couple of rules about the name for your web components when you register them:

- It must start with a lower-case ASCII letter.
- It must be expressed using lower case letters with any Unicode letter (including accents, or Chinese characters), numbers 0-9, and the symbols: `:`, `_`, `.`, and `-`.
- It must not contain spaces.
- It must contain a hyphen.
- It must not be `annotation-xml`, `color-profile`, `font-face`, `font-face-src`, `font-face-uri`, `font-face-format`, `font-face-name`, or `missing-glyph` (already used in markup languages as SVG or MathML).

The one-hyphen rule prevents clashes with future HTML standard elements and is enforced by the browser. If you attempt to define an element without a hyphen, `customElements.define()` will throw an exception. The hyphen can appear anywhere in the name. Still, most developers use a prefix to group related components (for example, `wc-button`, `manning-book`, or `app-modal`), or a type (for example, `symbol-bitcoin` or `symbol-yen`).

- Valid names: `pro-10`, `app-ui:button`, `app:format.date`, `manning-vanilla-book_chapter7`, `my-кнопка`, `button-español`
- Invalid names: `10-pro`, `country flag`, `app:button`, `format.date`, `APP-footer`, `font-face`

7.6.4 Simplifying the syntax

For some cases, it's also possible to reduce the syntax and create literal classes directly when defining the custom element, as in:

```

customElements.define('symbol-bitcoin', class extends HTMLElement {
  connectedCallback() {
    this.innerHTML = "&#8383;";
  }
});

```

Suppose you don't like working with classes for your web components. We need to remember that a class is just a constructor function with additional features, and it can be written in a different format. Therefore, if you like the idea of writing functions as in modern React or the Android framework Compose, you can also do it with a simple helper.

```

function defineWebComponent(tag, setup) {
  if (customElements.get(tag)) return; #1

  class FunctionalElement extends HTMLElement {
    #setupRan = false;
    connectedCallback() {
      if (this.#setupRan) return; #2
      this.#setupRan = true;
      setup(this); #3
    }
  }

  customElements.define(tag, FunctionalElement);
}

defineWebComponent("symbol-bitcoin", e => e.innerHTML="&#8383;"); #4
defineWebComponent("symbol-euro", e => e.innerHTML="&euro;"); #4
defineWebComponent("symbol-yen", e => e.innerHTML="&yen;"); #4

```

#1 Avoid double definition

#2 Run once per instance (avoid duplicate listeners on re-connect)

#3 Executes the function received as an argument

#4 Now we can create simple custom elements with functions

The previous example can be extended to support more abilities, and you can easily create, with little work. This micro-library helps you to get a simpler syntax for your web component generation and registration, as in:

```

registerWebComponent('symbol-bitcoin', {
  constructor() {
    // Write here your constructor code
  }
  onConnected() {
    // Write here your connectedCallback code
  }
  template = `
    <p>&#8383;</p>
  `
  styleUrl = "symbol-bitcoin.css"
});

```

The previous sample is just one demo of something you can achieve; it's not actual API syntax. We will discuss creating micro-libraries in Chapter 12.

7.6.5 The Custom Element Registry

The `CustomElementRegistry` is available through `customElements` in the `window` or `globalThis` object in a browser, and it exposes several helper methods apart from `define` that we've already covered:

- `customElements.get(name)` returns the constructor of a defined element or undefined if it has not been registered.
- `customElements.whenDefined(name)` returns a promise that resolves when the element has been registered. This method is helpful if you need to defer work until a component is available.
- `customElements.upgrade(root)` upgrades all custom elements in the subtree rooted at `root` that have been registered after the elements were created. This method is rarely needed but can be helpful when dynamically injecting markup before registration, known as lazy loading. We will cover this technique in Chapter 9.

7.7 Our first component

The following example demonstrates an autonomous custom element that wraps a `<button>` with a counter. Each time the button is clicked, it increments an internal count and updates the text.

Listing 7.2 A CounterButton web component

```

class CounterButton extends HTMLElement {
  #count; #1

  constructor() {
    super();
    this.#count = 0; #2
  }
  connectedCallback() {
    if (this.querySelector('button')) return; #3
    const button = document.createElement('button');
    button.textContent = 'Clicks: 0';
    button.addEventListener('click', () => {
      this.#count++; #4
      button.textContent = `Clicks: ${this.#count}`;
    });
    this.appendChild(button);
  }
}

customElements.define('counter-button', CounterButton);

```

#1 A private field for the counter

#2 We initialize the field in the constructor

#3 Prevents duplicate render

#4 We add 1 to the count on every button's click

You can use the component in HTML:

```

<script type="module" src="counter-button.js"></script>
<counter-button></counter-button>

```

Let's see how we can use Listing 7.2 to create multiple instances of the same element.

```

<script type="module" src="counter-button.js"></script>

<ul>
  <li><counter-button></counter-button></li>
  <li><counter-button></counter-button></li>
  <li><counter-button></counter-button></li>
</ul>

```

If you click the button, it increments the counter and renders its own text with the new value. The main benefit of using web components is not just splitting your app's content into smaller chunks but also reusing components many times, each as a different instance. For example, we can create a list of 3 counter buttons, each with its own internal button and counter variable, since they work independently.

NOTE The `connectedCallback` method fires every time the element is moved in the DOM (e.g., drag-and-drop operations or strict framework re-renders, not just when it's added to the document. We will cover more about the life cycle of web components in Chapter 9.

The final rendering and DOM tree for the previous example is shown in Figure 7.1. You can see that our `<counter-button>` appears in the Elements browser in DevTools as any other built-in element, with its own tree (such as a single `<button>`).

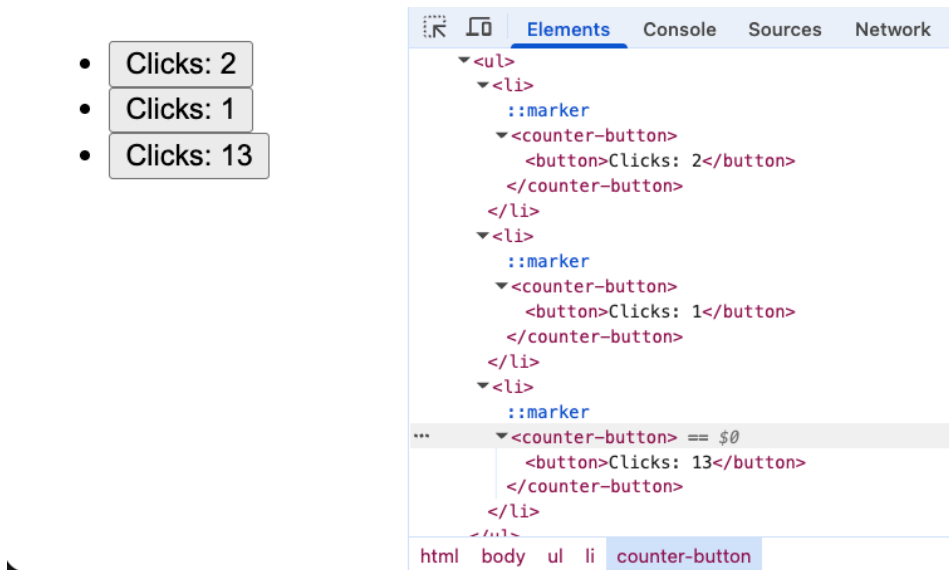


Figure 7.1 You can insert as many elements of your custom element, each with its own properties, attributes, and references in the DOM tree. Each one will encapsulate its own state.

Custom elements participate in the CSS cascade like any other element. You can style a custom tag using its name, and it will apply styles to the element, sometimes known as the host element:


```
counter-button {
  display: inline-block;
}
```

NOTE By default, hosts are defined as inline elements in the browser. You can override that default behavior in the container or within the component.

The `:defined` pseudo-class matches any element that has been registered with `customElements.define()`. This pseudo-class can help apply fallback styles or hide uninitialised elements, and it is handy when loading web components asynchronously. Hence, the `customElements.define` function will be called after the UI is initialized:

For example, you can hide all elements that were not defined with:

```
:not(:defined) {
  display: none
}
```

Or you can create a loading message while the elements are not defined, as in:

```
custom-element-tag:not(:defined)::before { #1
  content: "Loading...";
  align-content: center;
  text-align: center;
  font-size: 0.5rem;
  margin-right: 5px;
  border-radius: 2rem;
  opacity: 1;
}
```

#1 defines a message before any custom-element-tag while is not defined

Most of the CSS work with web components happens inside the component itself first, and then through the exchange of values and some styling between the host and the component instance.

7.8 Shadow DOM

Shadow DOM is a web standard that lets you encapsulate a component's internal structure, styles, and behavior, creating a hidden, isolated DOM tree attached to an element. It provides real encapsulation. Without it, custom elements would be fragile because any CSS or JavaScript on the page could accidentally break their internal implementation by applying a style or using the DOM API to traverse them.

By attaching a shadow root to your element, you create a boundary that hides internal elements and styles from the outside world. Doing so ensures that your component behaves consistently wherever it is used, even when the page defines conflicting styles or when the DOM is traversed via JavaScript. It also prevents your styles from leaking into the rest of the document.

With a shadow DOM:

- Styles don't leak in or out.
- IDs don't collide.
- Events are retargeted so the outside world interacts with the host element, not the internals.
- You have control of communication crossing the shadow boundary.

NOTE The shadow name comes from the idea that this DOM tree exists in the shadows: it's hidden from the main document. It's there, it renders, but it's not directly accessible through regular DOM queries from the outside, from what is now called **page DOM**. So `document.querySelector` or a CSS selector can't reach shadow DOM content.

In Figure [7.2](#), we can see the components of a shadow DOM in the page.

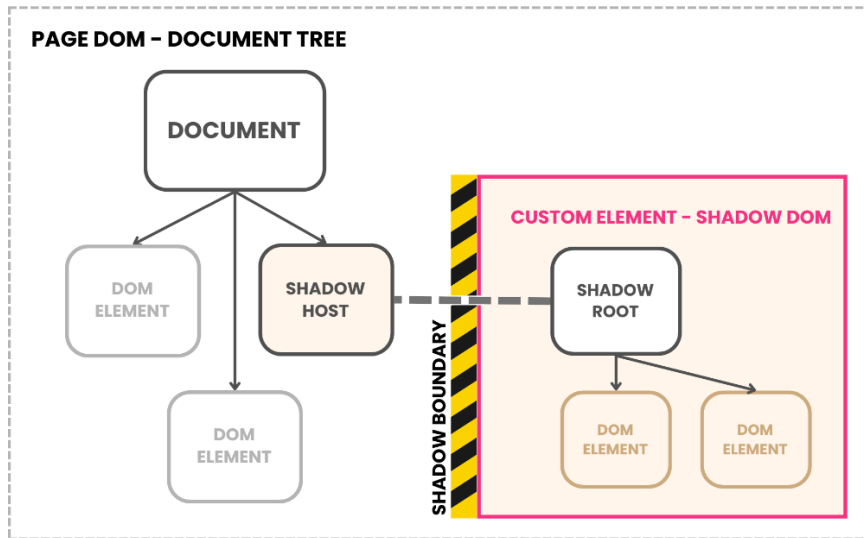


Figure 7.2 A document can include any DOM element, and when it encounters a shadow DOM, it creates a shadow host reference. The shadow host is called the shadow root from the shadow DOM's perspective when crossing the shadow boundary.

7.8.1 Creation

We create a shadow root by calling the element's method `attachShadow()` in the constructor.

```
class ShadowElement extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: 'closed' });
  }
}
```

Calling `attachShadow` returns a `ShadowRoot` that you can treat like a `DocumentFragment`, or the root of the DOM that you will be using within the component code for content manipulation. We can access it through the `shadowRoot` property of the element using `this.shadowRoot`:

```
class ShadowElement extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: 'open' });
  }

  connectedCallback() {
    this.shadowRoot.innerHTML = `Hello!`;
  }
}
```

After a Shadow DOM was attached to the component, use `this.shadowRoot` for internal rendering, and use `this` for the host's public API, such as attributes, properties, events, etc.

`AttachShadow` accepts an object as an argument that can specify values for these properties:

- *mode*: can be `open`, which exposes the shadow root via the `shadowRoot` property, or `closed`, which hides it so that `element.shadowRoot` is `null`.
- *delegatesFocus* boolean option ensuring that focus moves into the shadow tree when the host is focused.

WARNING After we set a shadow DOM in our component, we should never use DOM APIs (such as `innerHTML` or `appendChild`) over the element host through `this`, but through the `shadowRoot` property.

Choosing open or closed mode determines whether outside code (from the parent) can access the shadow tree of our component, allowing it to peek inside the shadow DOM. An open shadow exposes the `shadowRoot` property, allowing us to query internal elements and, for example, forward events, work with the DOM API, or perform imperative actions. A closed shadow returns `null` for `element.shadowRoot`, signalling that the internals are private and should not be touched. Open shadows are helpful for debugging and composition, while closed shadows provide stronger encapsulation. Choose the mode based on how much you want to expose the internals of your component to the outside.

In Figure [7.3](#), we can see the three DOM types we have available for a web component: the page DOM, a closed shadow DOM, and an open shadow DOM.

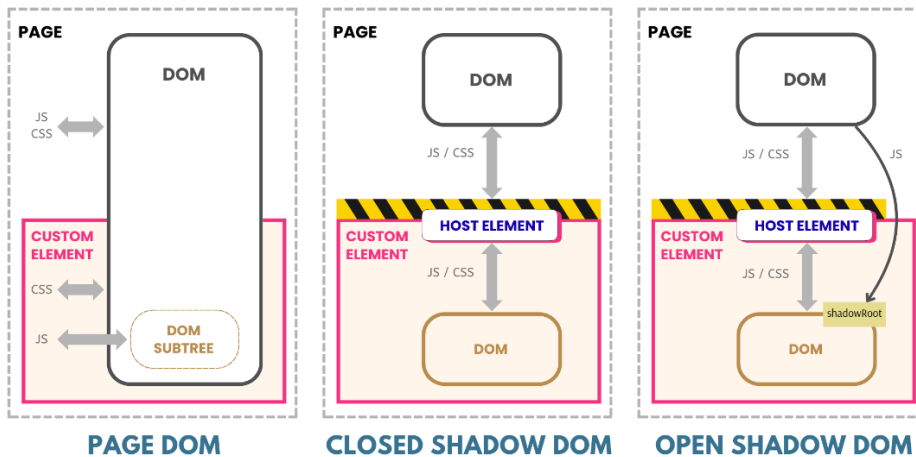


Figure 7.3 When creating a custom element, we need to decide if we want to use the page DOM, a closed shadow DOM, or an open shadow DOM. That will determine the visibility of JavaScript and CSS within the element and in the global scope.

Have in mind that closed shadows are a hint and do not provide strict security; browser extensions can still pierce the boundary. A shadow DOM can also be defined declaratively using the `<template>` element and the `shadowrootmode` attribute. Declarative shadow roots and templates are covered in depth in Chapter 8.

7.8.2 Styles with the page DOM

If a custom element does not use a shadow DOM root, it uses the page DOM and inherits the page's styles, which can lead to unintended consequences. Consider the code in Listing 7.3:

Listing 7.3 A web component using the page DOM

```

<style>
button { #1
  background-color: black; #1
  color: white; #1
} #1
</style>

<script>
class MyComponent extends HTMLElement {
  connectedCallback() {
    this.innerHTML = `
      <style>
        h1 { #2
          border: 1px solid gray; #2
        } #2
      </style>
      <button>Click me</button> #3
    `;
  }
}
customElements.define('my-component', MyComponent);
</script>

<h1>CSS with components</h1> #4
<my-component></my-component> #4

```

#1 styles declared at the page DOM scope

#2 styles declared at the component DOM scope

#3 component DOM scope

#4 page DOM scope

In this example, the page CSS makes every button white over black. And the style declaration within the component states that every `<h1>` should have a border. If we pay attention to the markup, the page markup has no `<button>` and the component markup has no `<h1>`.

CSS with components

Click me

Figure 7.4 When you use the page DOM in your web component, all the CSS declared globally and within the component will apply to both the global document and the contents of the component.

Because the component does not use a shadow root, the page style cascades into the internal `<button>` from our `<my-component>`, and the internal style is also applied to the page `<h1>`, as we can see in Figure 7.4. Therefore, without a shadow root, all the markup and style declarations are treated as part of the same group.

7.8.3 Styles with Shadow DOM

Let's apply a shadow DOM to `MyComponent` by attaching it and changing where we define `innerHTML`, as shown in Listing 7.4.

Listing 7.4 A web component using shadow DOM

```

<style>
button {
  background-color: black;
  color: white;
}
</style>

<script>
class MyComponent extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: 'open' }); #1
  }
  connectedCallback() {
    this.shadowRoot.innerHTML = ` #2
      <style>
        h1 {
          border: 1px solid gray;
        }
      </style>
      <button>Click me</button>
    `;
  }
}
customElements.define('my-component', MyComponent);
</script>

<h1>CSS with components</h1>
<my-component></my-component>

```

#1 We attach a shadow DOM to the component

#2 We now work with the shadowRoot DOM tree

With that change, the sample isolated style declarations from the global scope and from within the component, leading to the results shown in Figure 7.5. As there is no `<button>` in the global DOM, the global style is ignored, as well as the component style declaration, which is ignored because the shadow DOM in the component has no `<h1>` elements.

CSS with components



Click me

Figure 7.5 With a shadow DOM in a component, you won't have undesired effects; global CSS declarations will only affect global markup, and CSS declared within a component affects only the markup inside it.

As we will see later in this chapter, we can connect styles between the parent container and the component with a shadow DOM by declaring explicit interfaces.

The `:host` pseudo-class represents the host element on a shadow DOM, and it's initially useful to change the default `display` nature of the custom component, so we can specify if we want it to be `inline`, `block`, `inline-block`, or `flex`, as in:

```
:host {  
  display: inline-block;  
}
```

7.8.4 Styling web components

Styling a web component involves choosing where to define rules and how to expose them to consumers:

- Component styles in the shadow root. Create a `<style>` element and append it to the shadow root or write it as part of the `innerHTML` value. The rules defined there are scoped to the shadow tree. We will see this simple usage in the samples in this chapter.
- External style sheets. Append a `<link rel="stylesheet">` element inside the shadow root. Be aware that external styles may cause a flash of unstyled content because the browser must fetch the file before it can apply it. Consider deferring the markup until the stylesheet is ready.
- Constructible style sheets. Create a `CSSStyleSheet` object, call `replaceSync()` or `replace()` to set its content, and assign it to `shadowRoot.adoptedStyleSheets`. This approach allows you to share a single sheet across multiple components and update it dynamically.

- Global styles (no shadow DOM). If your component does not use a shadow root, it inherits the page's styles, which can be convenient for simple structural components but can also lead to conflicts. Avoid defining overly generic selectors.

NOTE Some build tools support CSS Modules, which scope class names at build time. This technique is a compile-time feature and not part of any web components specification or a Vanilla Web standard practice.

Unless it's an elementary element, encapsulate styles in a shadow root to prevent page CSS from leaking in and your styles from leaking out. Without a shadow root, custom elements are fragile because any CSS on the page could accidentally break their internal implementation.

Adopted Stylesheets is a modern way to apply CSS to a shadow DOM. Instead of writing a `<style>` tag inside every single component instance (which forces the browser to reparse the CSS every time), you create a shared stylesheet object in memory and tell the component to "adopt" it. You can even share the same stylesheet across different web components, which is helpful for component libraries.

NOTE If you have 1,000 components, the browser parses the CSS once, and all 1,000 components reference that single parsed stylesheet.

The syntax involves assigning an array of stylesheets to the shadow DOM object, as in:

```
shadow.adoptedStyleSheets = [globalResetStyles, buttonStyles,  
mySpecificThemeStyles];
```

Each stylesheet in the array can be defined as a new `CSSStyleSheet` object with its own content, as in:

```
const buttonStyles = new CSSStyleSheet();

// Add CSS rules to it (replaceSync is synchronous and easy for strings)
buttonStyles.replaceSync(`
  button {
    background-color: red;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 5px;
    font-size: 16px;
    cursor: pointer;
  }
  button:hover {
    background-color: darkred;
  }
`);
```

We will see advanced techniques for performance and more on adopted stylesheets in later chapters.

7.9 Element parameterization

A custom element's API surface often revolves around attributes, properties, and visual styles. There are many ways for web components to receive data from the parent to parametrize the component (with styling, behavior, or data) or to expose data to the parent.

- *Attributes* are visible in the HTML markup, and we read and write attributes using `getAttribute()` and `setAttribute()`, and their values are always strings. We typically use them for simple values that can be represented as strings (including numeric values) and that represent behavior or custom data.
- *Properties* in JavaScript are defined on the element's instance, and they can have values of any type, not just strings, including complex structures. They can be part of an API interface that you expose to the web component's consumer. We typically use them for complex values (such as objects, images, or arrays).
- *CSS custom properties(variables)* that the container can define in CSS for customizing styling of the component, including color, font declarations, and animations. Using CSS, let the consumer use media queries and other techniques to define the values for our component; for example, it can provide colors based on the user's dark or light theme.

- *Slots* are placeholders that a web component's consumer can define; they are inserted into `<slot>` elements within a custom element's template HTML. We'll cover slots in detail in Chapter 8.

Allowing a component to be personalized through attributes or properties is key to reusable components, as they can react differently to different values. When talking about visual styling, we can use attributes as in `color='red'`, but the preferred way to define them is through CSS, following the HTML pattern. We will cover how to use CSS to parametrize your component later in this chapter.

For example, if we have a `<star-rating>` element on our page, we can set and read its attributes in HTML, as in `<star-rating value="3">`. Using JavaScript and the DOM API, you can access both properties and attributes:

```
const widget = document.querySelector('star-rating');
widget.setAttribute('value', '3'); #1
const color = widget.getAttribute('selected'); #2
widget.value = 3; #3
const selected = widget.selected; #4
```

#1 We set an attribute as a string

#2 We get an attribute as a string

#3 We set a property as a string

#4 We get a property of any type

Don't access the DOM from the element's constructor, as it may not be initialized. It is safe to use it from when `connectedCallback` is called.

Every custom element will include all the global attributes currently defined in the HTML spec, such as `accesskey`, `dir`, `hidden`, `lang`, `style`, `title`, or `tabindex`. It's up to us to use their values in our content.

WARNING Attributes and properties are not in sync by default; if you want to expose a value in both and keep them synced, you have to do so in your code.

7.9.1 data-* vs standard attributes

The dataset API, as seen in Chapter 6, provides convenient access to values when using `data` attributes, automatically converting `data-attribute-name` to `this.dataset.attributeName` in JavaScript. However, `data-` attributes are best reserved for application-specific information that your component passes through or stores, rather than for core configuration. Using them for your component's primary API can make markup verbose and feel inconsistent with how native elements work.

Standard attributes mirror the conventions of built-in elements, where `<input type="text">` uses a direct attribute rather than a `data-` type. This approach produces cleaner markup, simpler CSS selectors, and establishes your component as a first-class citizen in the DOM. To access attribute values on custom elements, we have to use `getAttribute` and `setAttribute`. When designing your API, use standard attributes for configuration that affects behavior or presentation (for example, `variant` or `size`) and reserve `data` attributes for metadata that is not part of the component's core contract (for example, `data-product-id` or `data-index`).

7.9.2 Boolean attributes

Since HTML5, there has been a pattern for implementing boolean attributes: the presence of the attribute (regardless of its value) indicates true, and its absence indicates false. We can implement the same behavior in our custom element by copying the simple algorithm.

For example, we may have a `readonly` in a `star-rating` component, so `<rating-widget readonly>` means that the read-only attribute is set to `true`. In this case, we can use `element.hasAttribute('readonly')` in JavaScript to decide the current boolean value. If we use `getAttribute('readonly')` on the previous example, we will get an empty string as the value.

WARNING Have in mind that `disabled="false"` in markup is still truthy in the context of the boolean attribute design pattern.

7.9.3 Update the component while observing attributes

One crucial part of implementing web components with attributes is updating the contents when the attribute or property changes after initial rendering.

For example, you may have `<app-product data-product-id='23'>` that triggers an API call to get the product 23 details and render them on `connectedCallback`. But what happens if the component's consumer changes the `data-product-id` value later? We should update the component's details to honor the new product ID.

To detect when an attribute value changes, there is a three-step process that we have to implement:

- Define the list of observed attributes in the custom element
- Implement the method `attributeChangedCallback` that will be fired when any observed attribute value changes
- Update the component's content if needed

To define the list of observed attributes, we must add an array of strings in a static field in our class with the name `observedAttributes`, as in:

```
class MyCustomElement extends HTMLElement {
  static observedAttributes = ["animated", "type", "data-product-id"];

  // rest of the custom element code
}
```

In the previous example, all the attributes we use in your element can be accessed after `connectedCallback`, but we will only be notified when the value of one of those attributes changes: `animated`, `type`, and `data-product-id`.

Once we set the list of observed attributes, the next step is to implement a method that is automatically executed whenever the value of any of those attributes changes. The method to implement is `attributeChangedCallback`, and it receives three arguments: the name of the attribute as a string, the old value, and the new value.

```
class MyCustomElement extends HTMLElement {
  static observedAttributes = ["animated", "type", "data-product-id"];

  attributeChangedCallback(name, oldValue, newValue) {
    console.log(`The attribute '${name}' has changed
      from '${oldValue}' to '${newValue}'`);
  }
}
```

Based on the `name` value, you can trigger different update mechanisms in your component, such as updating the user interface or re-fetching data from a server or database. A typical design pattern is to implement component rendering in a single method, such as `render`, and then call that method from `connectedCallback` and `attributeChangedCallback`.

```

class MyCustomElement extends HTMLElement {
  static observedAttributes = ["animated", "type", "data-product-id"];

  connectedCallback() {
    this.render(); #1
  }

  attributeChangedCallback(name, oldValue, newValue) {
    this.render(); #2
  }

  render() {
    // fetches data and renders the component
    // using any technique, such as innerHTML or the DOM API
  }
}

```

#1 Render when the component is injected into a DOM

#2 Render on attribute value change; check if the current changed attribute should trigger an update

In the following chapters, we will analyze data-binding techniques that automatically trigger UI updates when data changes.

WARNING If we implement `attributeChangedCallback` for one or more attributes that we don't define in the list of observed attributes, the logic will never execute.

7.9.4 Update the component observing properties

We need to remember that attributes and properties work in different contexts. If you are offering a property-based API to the consumer of your web component, you should also trigger updates in your component when a property's value changes.

For example, we can create getters and setters like the following sample:

```

get value() { return this.getAttribute('value'); }
set value(val) { this.setAttribute('value', val); }

```

In Chapter 11, we will focus on data binding techniques and see how to listen for property changes to update the user interface. We can start by noting that, because it's a property, we can implement a setter function.

7.10 Creating a rating component

A typical UI pattern in modern web apps is to use a series of star symbols to indicate an item's rating. For example, a 2/5 rating may be rendered as two yellow stars, or two yellow and three grayed stars. Let's implement the second pattern using Listing 7.5.

Flight rating:



Hotel rating:



Restaurant rating:



```

<!DOCTYPE html>
<html lang="en">
  <head> ... </head>
  <body> == $0
    <p>
      "Flight rating: "
      <stars-rating value="4" aria-label="4 of 5 stars">
        >#shadow-root (open)
      </stars-rating>
    </p>
    <p>
      "Hotel rating: "
      <stars-rating value="2" aria-label="2 of 5 stars">
        >#shadow-root (open)
      </stars-rating>
    </p>
    <p>
      "Restaurant rating: "
      <stars-rating value="5" aria-label="5 of 5 stars">
        >#shadow-root (open)
      </stars-rating>
    </p>
  </body>

```

Figure 7.6 A star-based rating component can be parametrized via a value attribute that controls its rendering.

Let's write the code for them.

Listing 7.5 Creating a Star Rating web component

```

class StarRatingComponent extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({mode: "open"});
  }
  connectedCallback() {
    this.shadowRoot.innerHTML = `
      <style>
        ul {
          list-style: none;
          padding: 0;
          display: flex;
        }
        li {
          color: silver;
        }
        li.active {
          color: orangered;
        }
      </style>
      <ul>
        <li>&#9733;</li> #1
        <li>&#9733;</li> #1
        <li>&#9733;</li> #1
        <li>&#9733;</li> #1
        <li>&#9733;</li> #1
      </ul>
    `;

    this.render();
  }
  static observedAttributes = ["value"];

  attributeChangedCallback(name, oldValue, newValue) {
    if (name=="value") { #2
      this.render(); #2
    } #2
  }
  render() {
    const rawValue = this.getAttribute("value"); #3
    const value = Number.parseInt(rawValue ?? "0", 10) || 0; #4
    const stars = this.shadowRoot.querySelectorAll("li");
    this.ariaLabel = `${(value+1)} of 5 stars`; #5
  }
}

```

```

    for (let i=0; i<stars.length; i++) {
        stars[i].className = (i<value) ? "active" : "";
    }
}
}

customElements.define("star-rating", StarRatingComponent);

```

#1 HTML entity for the star symbol

#2 We only re-render if the value attribute has changed

#3 Attributes are always strings, so we must parse them as integers

#4 We cover all scenarios to avoid getting a NaN value

#5 Setting the ariaLabel helps accessibility of our custom component

Now, in any HTML document that has already executed and registered our component, we can use `<star-rating value='3'></star-rating>` and get an inline component rendering five stars of which three are active in a different color. We can see the sample in action in Figure 7.6, along with the corresponding document DOM and each rating's shadow DOM.

7.11 Working with events

Events are the primary way for a custom element to communicate with its parent. We will use `dispatchEvent()` to fire a `CustomEvent` when something happens within our component, and we want to expose those details to the outside.

In case the event has metadata, we pass it through a `detail` object. We can also set `bubbles: true` if the event should bubble up the DOM tree and `composed: true` if it should cross the shadow boundary.

From the outside of the component, consumers attach listeners using `addEventListener()` on the element, as with any other DOM event.

WARNING Do not use React-style `onValueChanged` attributes on the custom tag, as they are not standard. You may want to implement the classic HTML `onvaluechanged` attribute with JavaScript evaluation, but it's not recommended in a web component.

We can extend our star-rating component and make it editable using a boolean attribute. If it's editable, we can now listen for a click event on a star, update the value, and trigger a `change` event. We implement it in Listing 7.6.

Listing 7.6 Adding a custom event to StarRatingComponent

```

class StarRatingComponent extends HTMLElement {

  connectedCallback() {
    // ...same HTML from previous version

    const editable = this.hasAttribute("editable"); #1
    if (editable) {
      const stars = this.shadowRoot.querySelectorAll("li");
      for (let i=0; i<stars.length; i++) {
        stars[i].style.cursor = "pointer"; #2
        stars[i].role="button"; #3
        stars[i].ariaLabel = `Set rating to ${i+1} stars`; #3
        stars[i].tabIndex = i; #3

        stars[i].addEventListener("click", event => {
          const newValue = i+1; #4
          this.setAttribute("value", newValue); #5
          this.dispatchEvent(new CustomEvent("change", { #6
            detail: { value: newValue } #6
          }))) #6
        });
      }
    }
    this.render();
  }
  // ... same constructor, attributeChangedCallback, observedAttributes
  // and render methods
}

```

#1 We treat `editable` as a boolean

#2 Because our `<li>`s will become interactive, we change the cursor

#3 We also define ARIA attributes for accessibility

#4 Loop index starts at zero and clickable ratings on 1

#5 Remember to work with the attribute with `this` and not `this.shadowRoot`

#6 We dispatch a custom event with any string name and a detail object

Consumers attach listeners to the element using `addEventListener()`. For example:

```

<script type="module" src="star-rating.js"></script>

<p>Flight rating: <star-rating value="4"></star-rating></p> #1
<p>Hotel rating: <star-rating value="2"></star-rating></p> #1
<p>Restaurant rating (editable):
  <star-rating editable id="restaurant-rating"></star-rating> #2
</p>

<script>
document.getElementById("restaurant-rating").addEventListener("change", e => {
  console.log(`New rating: ${e.detail.value}`); #3
})
</script>

```

#1 Read-only stars rating

#2 Editable stars rating with no initial value

#3 We read the value sent by the component in the detail object

Keep in mind that we are not adding `editable` to the list of observed attributes, so once a `<star-rating>` is added to the DOM, it becomes read-only or editable forever, and changing the `editable` attribute won't affect the behavior.

For accessibility reasons, we may also add keyboard selection, e.g., by implementing `keydown` for rating selection.

7.12 Connecting the page DOM and shadow DOM with CSS

There are several simple ways to communicate with the shadow DOM using CSS.

- Read custom properties from the page DOM for theming.
- Expose custom properties for parametrization in the page DOM.
- Expose elements to the page DOM through parts.
- Read state from the page DOM using `:host()`.

In Figure [7.7](#) we see the connection options in a diagram.

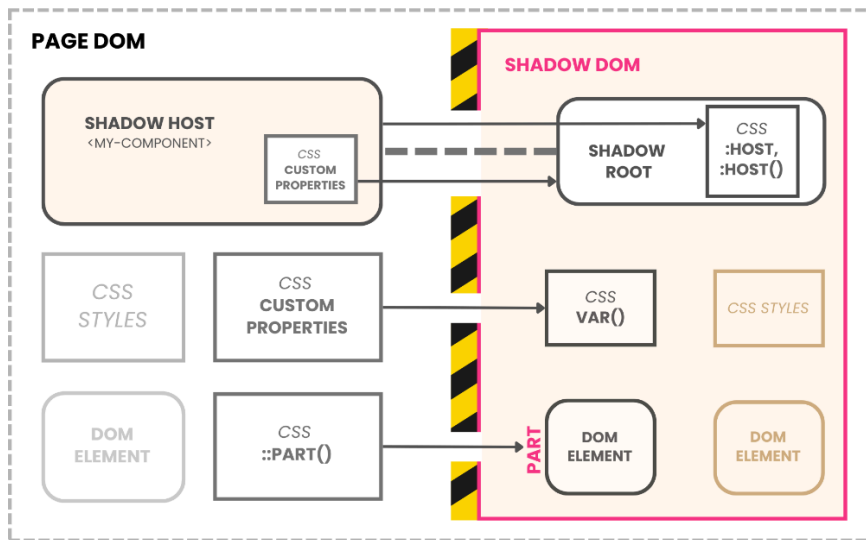


Figure 7.7 A shadow DOM can expose and receive styles from the page DOM using different techniques, such as through custom properties (set in the page DOM or in the shadow root element), through the definition of parts, or using the `:host()` pseudo-class to read shadow host state properties.

7.12.1 Read custom properties for theming

While not all CSS declarations in the page DOM pass through the shadow DOM barrier, CSS custom properties and variables do. Therefore, a web component in a shadow DOM can read custom properties that were set in the page DOM anywhere in the tree.

For example, the main HTML document can define a custom property for a default font declaration as in:

```
:root {
  --theme-font-family: 'Arial';
  --theme-text-size: 13.5px;
}
```

Any web component can read these values without explicitly mapping or defining them when using the shadow DOM. For example, in our `star-rating` component, we can change the star styles as follows:

```
li {
  font-size: var(--theme-text-size, 16px); #1
}
```

#1 We set the font size as 16px or what was defined in a theme variable

It's a good idea to offer a default value if the theme token was not set, as it's 16px in the previous example.

WARNING While theming with global CSS custom properties is simple, it also adds too much cohesion between the component and the project, and it may not help make a web component reusable across many projects, so use it with care.

7.12.2 Expose custom properties for parametrization

While styles defined in the page do not affect the shadow tree, the contents of our component, when using shadow DOM custom properties (variables), do. With that in mind, we can define custom properties on both sides of the shadow barrier and use them inside the component.

For example, in our `star-rating` component, we can define tokens for a custom property from the global page DOM styles, as in:

```
/* this goes in the global CSS file (page DOM) */
star-rating {
  --stars-inactive-color: gray;
  --stars-active-color: green;
}
```

You can use any CSS selector to target the custom elements, not just by element: you can use classes, IDs, or any combination to set different tokens, such as colors, to other instances of our web component.

These tokens will be available in the shadow DOM for use with the `var()` CSS function in any style. It's a good idea to set default initial values for these custom properties within the shadow DOM. To do so, we will use within the shadow DOM the `:host` pseudo-class that represents the host element on a shadow DOM, for example:

```
/* this goes in the custom element CSS declaration (shadow DOM) */
:host {
  --stars-inactive-color: silver;
  --stars-active-color: orangered;
}
```

We should document these CSS parameterization options in the web component's README file or documentation.

Flight rating: ★★★★★

Hotel rating: ★★

Restaurant rating (editable): ★★★★★

Figure 7.8 Using CSS custom properties, you can make your web component stylizable from the consumer's CSS declaration

Therefore, we can make the colors of our stars rating parametrizable by updating our code in Listing 7.7, starting with our web component's styles:

Listing 7.7 Implementing variable color definition for StarRatingComponent

```
/* this goes in the custom element CSS declaration (shadow DOM) */
:host {
  display: inline-block;
  --stars-inactive-color: silver;
  --stars-active-color: orangered;
}
ul {
  list-style: none;
  padding: 0;
  display: flex;
}
li {
  color: var(--stars-inactive-color);
}
li.active {
  color: var(--stars-active-color);
}
```

Then we can use any CSS color, including functions, in the page DOM that uses that web component, for all or some instances of the web component. In this case, we will update the stars' colors for read-only star ratings, making the inactive stars transparent, and for editable star ratings, we use a different set of colors, as seen in Figure 7.8:

```
/* this goes in main HTML style (page DOM) */
star-rating {
  --stars-inactive-color: transparent;
}

star-rating[editable] {
  --stars-inactive-color: white;
  --stars-active-color: purple;
  background: silver;
  border: 1px solid gray;
}
```

When both the shadow DOM and the page DOM are writing tokens for the same CSS custom property, the page DOM has higher priority; the Shadow DOM defines default values that the implementer of the web component can override from the page DOM.

7.12.3 Expose elements to the page DOM

A *part* is a way from a shadow DOM to expose to the consumer one or more elements in the inner DOM to the outside for styling. We use the `part` attribute and define a string name. Consumers can then style these parts via the `::part()` pseudo-element without accessing the entire shadow tree. For example, `::part(header)` in the page's CSS matches the element inside the component with `part="header"`.

NOTE If your component uses nested components and you want to forward `part` names from your inner web components, you can use the `exportparts` attribute on the host element. Check MDN documentation for further reference.

The `part` attribute works very similarly to the `class` attribute in HTML: more than one element can participate in the same part, and one element can be part of several parts at the same time.



Figure 7.9 Using the `::part` CSS pseudo-element, we can target DOM elements from within the shadow DOM that the component's developer exposed.

Let's see a simple sample from our `star-rating` component: we can expose the `<ul>` as the part named `list`, and each `<li>` as the part called `star`. Like with CSS custom properties, we should document which parts will be exposed to the consumer. In Figure 7.9 we can see parts in action.

In our web component's HTML, as seen in Listing 7.8, we can update the `<ul>` with:

Listing 7.8 Defining parts in the `StarRatingComponent` markup

```
<ul part="list"> #1
  <li part="star">&#9733;</li> #2
  <li part="star">&#9733;</li> #2
  <li part="star">&#9733;</li> #2
  <li part="star">&#9733;</li> #2
  <li part="star">&#9733;</li> #2
</ul>
```

#1 From the consumer we will have to use `::part(list)` to target this element

#2 We can have multiple elements as members of the same part name

Now, from the consumer, our CSS can target these parts even without knowing the element that is behind it:

```
star-rating[editable]::part(list)::before { #1
  content: 'Editable'; #1
  position: absolute; #1
  color: gray; #1
  font-size: xx-small; #1
}
star-rating::part(list) { #2
  border: 1px solid gray; #2
  border-radius: 25px; #2
  padding: 10px; #2
}
star-rating::part(star) { #3
  margin: 5px; #3
  rotate: 60deg; #3
} #3
```

#1 It adds a visible mark to editable ratings using `part` and `before`

#2 It targets the list of stars

#3 It targets each star in the component

7.12.4 Read state from the page DOM

If the page DOM defines state information through attributes, such as `class`, `disabled`, or a custom attribute like `theme` or `data-type`, we can use the CSS pseudo-class `:host(selector)` to target different states within the shadow DOM.

WARNING When using `:host`, you target your shadow root or host element, no matter what. When using `:host(selector)`, you target the host element only if it matches the selector inside the parentheses.

For example, we can change the web component's opacity if the `disabled` attribute is present by using:

```
/* it goes in the shadow DOM css declaration */
:host([disabled]) {
  opacity: 0.5;
  cursor: not-allowed;
}
```

Or we can change colors based on a class or attribute that should be defined in the page DOM, as in:

```
/* it goes in the shadow DOM css declaration */
:host(.blue-theme), :host([theme=blue]) {
  background-color: blue;
  color: white;
}
```

That will change the color palette of a web component if the consumer uses `<component-tag class='blue-theme'>` or `<component-tag theme='blue'>`.

7.13 Summary

- The component design pattern structures a user interface as small, reusable, self-contained units that bundle markup, behavior, and style, making large Vanilla Web applications easier to reason about, test, and maintain over time.
- Web Components are not a single API but a composition of standards: Custom Elements, shadow DOM, templates, and related specs—that together allow you to define new HTML tags that behave like native elements.
- Custom Elements provide the entry point: classes extending `HTMLElement` are registered with the browser, participate in the DOM lifecycle, and expose a public contract through attributes, properties, methods, CSS custom properties, parts, and events.
- Choose a descriptive name for your custom element containing a hyphen. Many developers use a prefix such as `app-` or `wc-` to avoid clashes with future native elements.
- Shadow DOM introduces real encapsulation by creating a hidden subtree with isolated styles and structure, preventing global CSS or scripts from the page DOM from breaking a component while still allowing controlled communication across the boundary.
- Decide whether the component needs a shadow root. If so, choose `open` or `closed` mode. An open shadow exposes a `shadowRoot` property, allowing consumers to query internal nodes; in closed mode, it is hidden.
- Components can render content imperatively, via `innerHTML`, by using the DOM API or by using templates; choosing the right strategy affects readability, performance, and security, especially when instances are created at scale.
- Attributes and properties serve different roles: attributes define declarative string values in HTML, properties expose typed APIs in JavaScript, and keeping them in sync is an explicit design responsibility of the component author.

- Events are the primary communication channel from component to consumer; `CustomEvent` with bubbling and composition lets components integrate naturally with the DOM event model, even across shadow boundaries.
- Styling Web Components requires deliberate decisions: page DOM components inherit page styles. In contrast, shadow DOM components rely on scoped styles, CSS custom properties, `:host`, and `::part` to balance encapsulation with theming and customization.
- Choose how to style the component. Inline styles in the shadow root are simplest. Constructible style sheets allow styles to be shared between instances. External style sheets may cause a flash of unstyled content.
- By building on browser standards instead of frameworks, Web Components deliver long-term stability, interoperability across stacks, and a reusable UI layer that can outlive tools and trends.

8 Working with Templates and Slots

This chapter covers

- Using `<template>` elements for components
- Working with external HTML files
- Rendering dynamic data in your templates
- Projecting content to components using `<slot>`

So far, we've been loading HTML content inline in our web components' JavaScript using `innerHTML` or the DOM APIs. But as we grow our application and the complexity of our web components increases, this solution may no longer be ideal. In this chapter, we will cover several techniques we can use to improve performance, developer experience, and the organization of our web component's markup, starting with the Template Content HTML element.

8.1 The Template Content HTML Element

The `<template>` element provides a declarative way to define chunks of markup that are not immediately rendered when the browser parses them. The content is parsed but not executed, which means no images or stylesheets referenced in the markup will be loaded, and no scripts will be executed until we actually use them. To use it, we will clone it and attach the clone to the DOM.

Anything placed inside a `<template>` lives in a separate `DocumentFragment` and is ignored during initial page rendering. A *Document Fragment* is a lightweight, "off-screen" DOM container containing nodes that are not part of the page's live DOM.

You can think of it as an inert blueprint: the parser still checks the markup for validity, but the contents are never attached to the DOM until you explicitly do so with JavaScript. It makes templates ideal for storing repeated structures: table rows, cards, tooltips, component markup, or any other reusable snippet, without having to wrap them in hidden elements or embed them in script strings.

Practically speaking, a `<template>` helps you separate structure from logic. Repeated HTML can live in the document or in a separate HTML file, and your JavaScript can clone it as needed. Because the browser validates the markup ahead of time and does not attempt to execute scripts inside the template, you avoid security pitfalls associated with evaluating arbitrary HTML strings.

While Template Content elements are not required in a Web Component architecture and are entirely independent, they are an everyday companion for Custom Elements and shadow DOM.

WARNING The `<template>` is also used in a technique known as *DSD (Declarative Shadow DOM)* when a `shadowrootmode` attribute is present in the element. In this case, the `<template>` element has an entirely different behavior than the one covered in this chapter. DSD is useful for Server-Side rendering web components, and it's covered in full detail in Chapter 9.

8.1.1 Usage

A `<template>` element can be created declaratively in an HTML document or programmatically with JavaScript, as with any other HTML element.

Using a `<template>` begins by declaring it somewhere in your page (often just before the closing `</body>` element). It is common to give the template an `id` so it can be located later.

NOTE The contents of a `<template>` element are not initially part of the rendered DOM, so using containers (such as `<section>` or `<div>`) for templates is pointless. That's why we typically define templates directly as `<body>` children.

The example below defines a simple table row template that we can use to fill a table with data.

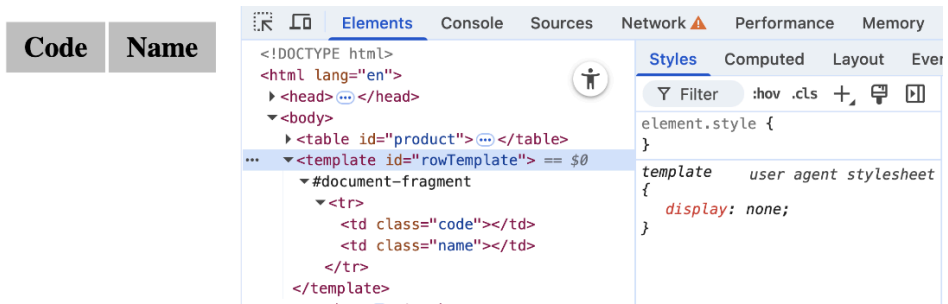


Figure 8.1 The `<template>` element is inert and hidden when rendering the page; to use it you have to clone it and inject that clone into the DOM

Notice that nothing inside the `<template>` is visible on the page until we clone and insert it, as seen in Figure 8.1 when executing Listing 8.1:

Listing 8.1 Using the template HTML element to render multiple rows

```
<table id="product">
  <thead>
    <tr><th>Code</th><th>Name</th></tr>
  </thead>
  <tbody></tbody> #1
</table>

<template id="rowTemplate"> #2
  <tr> #2
    <td class="code"></td> #2
    <td class="name"></td> #2
  </tr> #2
</template> #2
```

#1 Rows will be added here later by JavaScript

#2 This template is parsed, but not rendered by the browser

To use this template, we need to retrieve it by its ID using JavaScript and access the `content` property to obtain the underlying `DocumentFragment`. We should not insert that content into the DOM; instead, create a clone of it and insert it.

Code	Name
1235646565	Vanilla Web Book
0384928528	Web Components Book
9182310003	Modern JavaScript Book

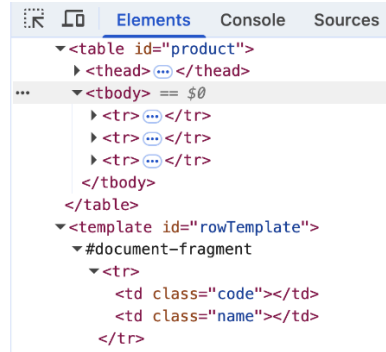


Figure 8.2 Three clones of the template are now inserted in the `<tbody>` so instances of the template become visible.

The following script demonstrates how to populate the previous table with three rows as illustrated in Figure 8.2:

```
const tbody = document.querySelector("#product tbody"); #1
const tpl = document.querySelector("#rowTemplate"); #2

function addProduct(code, name) {
  const clone = document.importNode(tpl.content, true); #3
  const cells = clone.querySelectorAll("td"); #4
  cells[0].textContent = code; #4
  cells[1].textContent = name; #4
  tbody.appendChild(clone); #4
}

addProduct("1235646565", "Vanilla Web Book");
addProduct("0384928528", "Web Components Book");
addProduct("9182310003", "Modern JavaScript Book");
```

#1 Reference to the table

#2 Reference to the template

#3 We clone the template

#4 We always use the clone, not the template

Because the template lives outside of the rendered DOM, you can safely modify the cloned nodes without affecting other instances cloned from the same template.

8.1.2 The Need for Cloning Templates

The `content` property of a template returns a `DocumentFragment`, a lightweight container for a tree of nodes. When you append a `DocumentFragment` to the DOM, its children are transferred: meaning they are moved, not copied. If you were to insert the same fragment multiple times without cloning it, only the first insertion would succeed; subsequent insertions would have no effect because the fragment's children have been consumed and the content is empty.

Template cloning is significantly faster than repeatedly parsing `innerHTML`, especially for components instantiated many times.

To reuse a template, therefore, you must clone its content before insertion. There are two primary methods to do this:

- *Clone node*: available through `template.content.cloneNode(true)`, it clones the nodes in the context of the template's owner document.
- *Import node*: available through `document.importNode(template.content, true)`, it imports the nodes into the document's context, upgrading any custom elements according to the document's registry.

When you get the `<template>` from the same DOM or HTML file you use to render the web component's instance, cloning or importing the node will have the same effect. When your template may originate from another document, you should import the node as a best practice, especially if the template uses custom elements that you registered in the main document.

Cloning preserves the template's inertness. Scripts within the template are not executed when it is cloned, and the template's IDs do not conflict with those elsewhere in the document.

WARNING Do not attach data or event listeners directly to the object returned by `template.content`. When you clone the template, the container itself is not copied: only its children; so any properties or event listeners you add to that element would be lost.

8.1.3 Advantages

Using the `<template>` element confers several benefits:

- Separation of concerns: Markup is kept in HTML, not buried inside JavaScript strings; it also means designers can edit templates without touching code.
- Reduced duplication: One template can be cloned as many times as necessary, avoiding repeated DOM construction code.

- Safety and performance: Templates are parsed by the browser, which helps catch syntax errors early. Because template content is inert until used, it does not trigger network requests (for example, images inside a template are not fetched), nor are scripts executed.

While a `<template>` element doesn't need a custom element or web component, it's primarily used in conjunction with them.

WARNING Avoid using templates solely to hide content. While the browser does not render a template's content, assistive technologies may still read it, and search engines may index it. Use `hidden` or CSS to hide content for accessibility reasons.

8.2 Using Templates in Web Components

Web Components often need to present a chunk of static markup and encapsulated styles, as in a design system button, a modal dialog, a toast notification, or a reusable card component.

To set it up, there are different patterns:

- Using JavaScript and the DOM API programmatically, as seen in previous chapters.
- Defining a `<template>` in your HTML document and cloning it inside your custom element.
- Defining the markup in an external HTML file that you load using JavaScript.

8.2.1 Defining a Template in the HTML document

The following example defines a `product-card` component that displays a product image, title, and price.

The markup and styles live in a template declared in the main document with the rest of the content, as seen in Listing 8.2.

Listing 8.2 Using a <template> in the main DOM for a web component's content

```

<template id="product-card-template"> #1
  <style>
    :host { #2
      display: block;
      border: 1px solid #333;
      padding: 1rem; margin: 1rem;
    }
    h2 { margin: 0 0 0.5rem 0; font-size: 1.25rem; }
    .price { color: #b12704; font-weight: bold; }
  </style>
  <img class="product-image" />
  <h2 class="title"></h2>
  <p class="price"></p>
</template>

<script>
class ProductCard extends HTMLElement {
  static get observedAttributes() { return ["title", "price"]; }
  constructor() {
    super();
    this.attachShadow({ mode: "open" });
    const template = document.getElementById("product-card-template"); #3
    const clone = document.importNode(template.content, true); #4
    this.shadowRoot.appendChild(clone); #5
  }
  attributeChangedCallback(name, oldValue, newValue) { #6
    switch (name) {
      case "title":
        this.shadowRoot.querySelector(".title").textContent = newValue;
        break;
      case "price":
        this.shadowRoot.querySelector(".price").textContent = newValue;
        break;
    }
  }
}
customElements.define("product-card", ProductCard);
</script>

```

- #1 We define the `<template>` element in the main DOM
- #2 We use shadow DOM, so it targets the `<product-card>` element
- #3 We get the template
- #4 We clone the template
- #5 We inject the clone into the shadow DOM
- #6 We update the data when the attribute changes

Once defined, the component can be used anywhere in your page:

```
<product-card image="/img/camera.png" title="Digital Camera" price="$199">
</product-card>
<product-card image="/img/headphones.png" title="Wireless Headphones"
price="$59"></product-card>
```

Using a `<template>` element in the main HTML document for web components is a good solution for server-rendered apps.

Updating the contents of a `<template>` element after it is cloned does not update the clones already in the DOM.

8.2.2 Using external HTML files

If you don't want to store the template for your web component in the main document, you can create external HTML files and load them via the Fetch API and, optionally, cache the results. This is particularly useful on large applications when you have dozens or hundreds of web components. For example, suppose you have a `product-card.html` file containing the contents of the `<template>` element. You could load it once when your application starts and store the parsed template in a static variable so it's not fetched for every instance of our component, as seen in Listing 8.3.

Listing 8.3 Loading the template from external HTML documents

```

class ProductCard extends HTMLElement {
  static template = null; #1

  static async loadTemplate() {
    if (!this.template) {
      const response = await fetch('./product-car.html'); #2
      if (response.status !== 200) { #2
        throw new Error(`Server code returned: #2
          ${response.status}`); #2
      } #2
      const text = await response.text(); #2
      this.template = document.createElement('template'); #3
      this.template.innerHTML = text; #3
    }
    return this.template;
  }

  async connectedCallback() {
    this.attachShadow({ mode: "open" });
    await this.constructor.loadTemplate(); #4
    // The template is ready to use
    const content = this.constructor.template.content;
    const clone = document.importNode(content, true); #5
    this.shadowRoot.appendChild(clone);
    this.attributeChangedCallback("title", null, #6
      this.getAttribute("title")); #6
    this.attributeChangedCallback("price", null, #6
      this.getAttribute("price")); #6
  }

  static get observedAttributes() { return ["title", "price"]; }

  attributeChangedCallback(name, oldValue, newValue) {
    // We only render UI if the template is loaded
    if (this.constructor.template==null) return;
    switch (name) {
      case "title":
        this.shadowRoot.querySelector(".title").textContent = newValue;
        break;
      case "price":
        this.shadowRoot.querySelector(".price").textContent = newValue;
        break;
    }
  }
}

```

```

    }
  }
}

```

#1 Storage for template's content

#2 Fetch the content from the network

#3 Create the <template> to parse the content once

#4 Wait for the template's content

#5 It comes from an external document, so we use import node

#6 We have to force update the UI

This approach caches the template after the first fetch, avoiding repeated network requests when you have several instances of the same web component.

There are two problems with the current implementation that we have: what are we rendering while the template is being downloaded (in case it wasn't ready from a previous instance), and what happens if there is any error while downloading the template, as a network connection error or the file doesn't exist.

As of now, we render nothing in both situations (loading and error), which may create an undesired visual flash effect on the page.

NOTE Even if you move the template download function call to the constructor, we can't guarantee it will be executed before `connectedCallback`, and there is no way to make the DOM rendering engine wait for the file to download. Therefore, the browser will render an empty element until we make a change in the DOM.

To improve the user experience in these situations, we can render alternative content, such as a loading message, an animation, or any other content while the template is being downloaded, or when an error occurs.

For example, we can update our `connectedCallback` function with this code:

```

async connectedCallback() {
  this.attachShadow({ mode: "open" });
  if (this.constructor.template==null) {
    // The template is not loaded
    const alternative = document.createElement("div"); #1
    alternative.textContent = "Loading..."; #1
    this.shadowRoot.appendChild(alternative); #1
    try {
      await this.constructor.loadTemplate();
      this.shadowRoot.removeChild(alternative); #2
    } catch(e) {
      console.error(e);
      alternative.textContent = "Error loading the component"; #3
      return;
    }
  }
  // The template is ready to use
  const clone = document.importNode(this.constructor.template.content, true);
  this.shadowRoot.appendChild(clone);
  this.attributeChangedCallback("title", null, this.getAttribute("title"));
  this.attributeChangedCallback("price", null, this.getAttribute("price"));
}

```

#1 Alternative content for our component while it's loading

#2 On success, remove the alternative content

#3 On failure, change the alternative content

You could also use the Cache Storage API to cache the response and serve it offline, or HTML Resource Hints (<https://web.dev/learn/performance/resource-hints>) to help the browser prioritize downloading the template HTML file before it knows it will be needed.

To avoid duplicated code, you can create a base class with the loading behavior and then make every component that loads the HTML from an external file inherit from it, as seen in Listing 8.4

Listing 8.4 Creating a base class that loads external templates from HTML files

```
// external-template-element.js
export class ExternalTemplateElement extends HTMLElement {
  static template = null;
  static templateUrl = null; #1
  static loadingContent = ''; #2
  static errorContent = 'Error loading component'; #2

  static async loadTemplate() {
    if (!this.template) {
      if (!this.templateUrl) {
        throw new Error(`${this.name} must define a static templateUrl`);
      }
      const response = await fetch(this.templateUrl);
      if (response.status !== 200) {
        throw new Error(`Failed to load template: ${response.status}`);
      }
      const text = await response.text();
      this.template = document.createElement('template');
      this.template.innerHTML = text;
    }
    return this.template;
  }

  async connectedCallback() {
    if (!this.shadowRoot) {
      this.attachShadow({ mode: 'open' });
    }
    if (!this.constructor.template) {
      const placeholder = document.createElement('div');
      placeholder.className = 'component-placeholder';
      placeholder.textContent = this.constructor.loadingContent;
      this.shadowRoot.appendChild(placeholder);
      try {
        await this.constructor.loadTemplate();
        this.shadowRoot.removeChild(placeholder);
      } catch (error) {
        console.error(`${this.constructor.name}:`, error);
        placeholder.textContent = this.constructor.errorContent;
        placeholder.className = 'component-error';
        this.templateError(error);
        return;
      }
    }
  }
}
```



```

    }
    const clone = document.importNode(this.constructor.template.content, true);
    this.shadowRoot.appendChild(clone);
    this.templateReady();
  }

  templateReady() {} #3
  templateError() {} #3
}

```

#1 Subclasses must override this property with the file name

#2 Subclasses can override these messages

#3 Event hooks for subclasses to override

Then, we can create a new web component using our base class, as in the following sample:

```

// user-card.js
import { ExternalTemplateElement } from "../external-template-element.js";

class UserCard extends ExternalTemplateElement { #1
  static templateUrl = './user-card.html'; #2
  static loadingContent = '...'; #3

  templateReady() { #4
    // TODO: implement custom rendering content
  }
}
customElements.define("user-card", UserCard);

```

#1 We inherit the element from our base class

#2 We must define the URL of the template

#3 Optionally, we can set other arguments

#4 Optionally, we can override the hook methods

Remember to call `super.connectedCallback()` if you override the subclass `connectedCallback`. Optionally, you can implement `templateError` to do something when the template couldn't be loaded in this instance.

You can customize the `ExternalTemplateElement` base class for your needs, such as creating animated loading placeholders instead of a loading text, or waiting 500ms before rendering a loading message to avoid flickering content.

8.2.3 Loading external CSS stylesheets

We covered how CSS works with web components in Chapter 7, but when working with `<template>` elements or loading templates from external HTML files, we need to pay attention to a new effect if the template loads external stylesheets using `<link rel="stylesheet">` or through `@import`: the FOUT (Flash of Unstyled Text).

Flash of Unstyled Text is an undesired effect that renders HTML on the screen without an external stylesheet, leaving it unstyled until the stylesheet is loaded. It happens because a `<template>` is inert until it's cloned and injected into the DOM, so the browser doesn't download or parse the external stylesheets you may link to until then.

To reduce this effect, you can:

- Use the `CSSStyleSheet` constructor with adopted stylesheets seen in Chapter 7.
- Use `<style>` in the HTML instead of a link to an external stylesheet.
- Preload the stylesheets you will need using JavaScript or Resource Hints in the main DOM.
- Extend the loading state of your web component until all the stylesheets are loaded.

If you want to use the last technique, you will have to detect if all the external `link` stylesheets are loaded; for that purpose, you can use the following helper function:

```
function waitForShadowStyles(shadowRoot) {
  const links = [...shadowRoot.querySelectorAll('link[rel="stylesheet"][href]')];

  return Promise.all(
    links.map(link => {
      // Already loaded (common when cached)
      // link.sheet is set only after the stylesheet is available.
      if (link.sheet) return Promise.resolve();

      return new Promise((resolve, reject) => {
        const cleanup = () => {
          link.removeEventListener('load', onLoad);
          link.removeEventListener('error', onError);
        };
        const onLoad = () => { cleanup(); resolve(); };
        const onError = () => { cleanup(); reject(new Error(`Failed to load CSS:
${link.href}`)); };

        link.addEventListener('load', onLoad, { once: true });
        link.addEventListener('error', onError, { once: true });
      });
    })
  );
}
```

Then, in your web component, you can use something like:

```

class MyCard extends HTMLElement {
  #stylesReady;

  constructor() {
    super();
    this.attachShadow({ mode: 'open' });
    // ... load from a `` or from external files
    this.#stylesReady = waitForShadowStyles(root);
  }

  async connectedCallback() {
    try {
      await this.#stylesReady; #1
      // here is safe to measure layout, run animations, etc.
    } catch (e) {
      console.error(e);
    }
  }
}

customElements.define('my-card', MyCard);

```

#1 We wait until all the external stylesheets from the template are loaded

8.3 Rendering Dynamic Data in Templates

Static templates are great when every instance of a component shares the same structure. Many web components, however, needs to inject dynamic data or loop over collections, as in a book details component or a list of products. When we were defining the HTML content of our custom element with JavaScript, we were using template strings and variable replacements, as in:

```

this.rootElement.innerHTML = `

${price}</p>`;


```

But if the content comes from a template content element, we can't use this technique, since the content is already written and parsed in the template. One way to achieve it without introducing a complete framework is to use the DOM API to look for specific placeholders in the `textContent` property of DOM elements, or specific values for certain attributes. For example, your template could do something like: `<p data-fill="lastName"></p>` or `<p>{{ lastName }}</p>`, and then you browse the DOM looking for `data-fill` attributes or the pattern `{{ property }}` and inject data (such as the `lastName` value) from an object.

WARNING You could also use a simple replacement function on the `innerHTML` to replace dynamic data in the template, but that won't leverage the pre-parsed nature of the `<template>` element.

Another way to handle dynamic templates is to clone a `<template>` multiple times and fill in variables imperatively. For example, suppose you have an `item-template` defined in HTML. You can loop over your data array, clone the fragment for each item, and update the text nodes accordingly.

Consider a `<cart-summary>` component that displays a list of items in a shopping cart. Each item has a name, quantity, and price. This technique avoids string parsing and leverages the DOM directly, and you can see it in Listing 8.5:

Listing 8.5 Using nested web components to render dynamic data as multiple elements

```
<template id="item-template">
  <li class="cart-item">
    <span class="item-name"></span>
    <span class="item-qty"></span>
    <span class="item-price"></span>
  </li>
</template>
```

The web component should look like:

```

class CartSummary extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({mode:'open'});
    const tpl = document.getElementById('item-template');
    this.itemTemplate = tpl.content;
  }
  set items(data) {
    const list = document.createElement('ul');
    data.forEach(item => {
      const clone = document.importNode(this.itemTemplate, true);
      clone.querySelector('.item-name').textContent = item.name;
      clone.querySelector('.item-qty').textContent = `x${item.quantity}`;
      clone.querySelector('.item-price').textContent = item.price;
      list.appendChild(clone);
    });
    this.shadowRoot.replaceChildren(list);
  }
}
customElements.define('cart-summary', CartSummary);

```

After defining the component, you can use it in HTML via the `<cart-summary>` element, and access its API from JavaScript, as shown in the following example:

```

<cart-summary id="summary"></cart-summary>

<script>
  const summary = document.getElementById('summary');
  summary.items = [
    { name: 'Camera', quantity: 1, price: '$199' },
    { name: 'Headphones', quantity: 2, price: '$59' }
  ];
</script>

```

8.3.1 Creating a Custom Template Engine

In more complex scenarios, you should do more than simple string replacement, property by property. Template engines abstract away the details of string concatenation and provide syntax for variables, loops, and conditionals. The general pattern is to supply a template (often defined in HTML or as a string) and a data object to a rendering function. The engine parses the template, replaces placeholders with data, executes loops or conditionals, and returns HTML. You can then insert the resulting markup into your component's shadow DOM.

We can take Listing 8.5 and go one step further, making the pattern replacement automatic by finding the double curly braces pattern with a regular expression and matching `{{ name }}` with the `name` property of the object we are rendering, as seen in Listing 8.6.

Listing 8.6 Updating the Template for pattern matching replacement.

```
<template id="item-template">
  <li class="cart-item">
    <span class="item-name">{{ name }}</span> #1
    <span class="item-qty">x{{ quantity }}</span> #1
    <span class="item-price">{{ price }}</span> #1
  </li>
</template>
```

#1 We add the `{{ property-name }}` pattern in the template

We need to use a helper `interpolate` function that can be shared for all web components using the DOM API `TreeWalker` for efficiently traversing nodes:

```
function interpolate(template, data) {
  const clone = document.importNode(template, true);

  const walker = document.createTreeWalker(clone, #1
    NodeFilter.SHOW_TEXT, null); #1

  let node;
  while (node = walker.nextNode()) {
    if (node.textContent.includes('{{')) { #2
      node.textContent = node.textContent.replace( #2
        /\{\{s*(\w+)\s*\}\}/g, #2
        (match, key) => data[key] ?? ''); #2
    } #2
  }
  return clone;
}
```

#1 We only care about text nodes

#2 If there is a `{{` we replace the pattern `{{ key }}` with the value using a regular expression

We need to replace the `forEach` function, and now we have a simple interpolation system.

```
data.forEach(item => {
  const clone = interpolate(this.itemTemplate, item);
  list.appendChild(clone);
});
```

Just in case you want to go even further and add support for attribute value replacement (such as `<input value="{{phoneNumber}}>`) or nested properties (such as `<p>{{address.city}}</p>`), you can upgrade the `interpolate` function:

```
function interpolate(template, data) {
  const clone = document.importNode(template, true);

  const getValue = (obj, path) => {
    return path.split('.').reduce((acc, key) => acc?.[key], obj) ?? '';
  };

  const pattern = /\{\{s*([\w.]+)s*\}\}/g;

  const walker = document.createTreeWalker(clone,
    NodeFilter.SHOW_TEXT, null);

  let node; #1
  while (node = walker.nextNode()) { #1
    if (node.textContent.includes('{{')) { #1
      node.textContent = node.textContent.replace(pattern, #1
        (match, path) => getValue(data, path)); #1
    } #1
  } #1

  clone.querySelectorAll('*').forEach(el => { #2
    for (const attr of el.attributes) { #2
      if (attr.value.includes('{{')) { #2
        attr.value = attr.value.replace(pattern, #2
          (match, path) => getValue(data, path)); #2
      } #2
    } #2
  }); #2

  return clone;
}
```

#1 Processes text nodes

#2 Processes attributes

8.3.2 Using Community Template Engines

Libraries add dependencies and complexity. For most web components, the simple `interpolate` function we built earlier is sufficient. Consider a community template engine only when you need:

- Complex conditionals/loops that would be awkward manually
- Shared templates across components (partials)
- Build-time template compilation
- Team familiarity with a specific engine

Popular engines such as lit-html, Mustache, Handlebars, doT, and Nunjucks follow this pattern and can be used in vanilla JavaScript without a complete framework.

WARNING When using community templates with Web Components, keep in mind that engines typically produce plain HTML; you still need to attach the rendered markup to your component's shadow root or light DOM using `innerHTML` so they won't take advantage of the `<template>` element's markup parsing performance.

Several mature template engines can help you generate HTML for your components. Let's see what templates look like with two examples on Mustache and lit-html:

Mustache (<https://github.com/janl/mustache.js>): A logic-less templating system that works across many languages. It expands template tags using values from an object. Because it is logic-less, templates remain predictable and straightforward. Size: ~9KB minified.

```
import Mustache from 'mustache';

const template = `
  <div class="product">
    <h2>{{title}}</h2>
    <p>{{description}}</p>
    {{#inStock}}<span class="badge">In Stock</span>{{/inStock}}
    {{^inStock}}<span class="badge sold-out">Sold Out</span>{{/inStock}}
  </div>
`;

const view = {
  title: 'USB-C Camera',
  description: 'A great camera for streaming',
  inStock: true
};

this.shadowRoot.innerHTML = Mustache.render(template, view);
```

lit-html (<https://lit.dev/docs/libraries/standalone-templates>): A library from Google that uses tagged template literals to render and update HTML efficiently. Unlike string-based engines, lit-html creates an actual DOM and only updates the parts that changed. Size: ~5KB minified.

```

import { html, render } from 'lit-html';

class ProductCard extends HTMLElement {
  connectedCallback() {
    this.attachShadow({ mode: 'open' });
    this._items = [];
    this.render();
  }

  set items(value) {
    this._items = value;
    this.render();
  }

  render() {
    render(html`
      <style>:host { display: block; }</style>
      <ul>
        ${this._items.map(item => html`
          <li>
            <span>${item.name}</span>
            <span>${item.price}</span>
            <button @click=${() => this.addToCart(item)}>Add</button>
          </li>
        `)}
      </ul>
    `, this.shadowRoot);
  }

  addToCart(item) {
    this.dispatchEvent(new CustomEvent('add-to-cart', {
      detail: item,
      bubbles: true
    }));
  }
}

```

When selecting an engine, consider its size, features, and rendering approach. String-based engines (Mustache, Handlebars, doT, EJS, Nunjucks) are simpler to understand but re-parse HTML on every render. Tagged template literal libraries (lit-html, uhtml) maintain references to dynamic parts and update only what changed, offering better performance for frequently-updating components. We will cover more about live data binding in Chapter 11.

Many string-based engines support precompilation at build time, converting templates to JavaScript functions. It eliminates runtime parsing overhead and reduces bundle size by shipping only the compiled functions rather than the whole engine.

In the context of Web Components, you typically render string-based templates into the shadow root via `innerHTML`:

```
this.shadowRoot.innerHTML = Mustache.render(template, data);
```

For tagged template literal libraries, you call their render function with the shadow root as the target:

```
render(html`...`, this.shadowRoot);
```

Keep in mind there are no parentheses when using tagged template literals (as in `html`).

JSX IN VANILLA WEB COMPONENTS

JSX is a syntax extension that lets you write HTML-like markup directly in JavaScript. React popularized it, but it is not tied to it; JSX compiles to function calls, and you can configure which function gets called. With Babel, TypeScript, or modern bundlers like Vite and esbuild, you can set a "pragma" that transforms `<div>` into `h('div')` instead of `React.createElement('div')`.

Libraries like `jsx-dom` (<https://github.com/proteriax/jsx-dom>), `NanoJSX` (<https://nanojsx.io>), or even a custom 20-line `h` function can then create fundamental DOM elements from that JSX. The tradeoff is that JSX requires a build step; you cannot run it directly in the browser, so it suits projects that already have a compilation pipeline, and it's typically out of the Vanilla Web package.

Let's do a quick comparison of each community template engine for web components in Table 8.1:

Table 8.1 Comparison of template engines available for web components

Engine	Size	Approach	Best for
Mustache https://github.com/janl/mustache.js	~9KB	Logic-less string templates	Simple interpolation, multi-language projects
Handlebars https://handlebarsjs.com	~77KB / ~4KB precompiled	String templates with helpers	Complex templates, build-time compilation
doT.js https://github.com/olado/doT	~4KB	Compiled string templates	Performance-critical, many instances
EJS https://ejs.co	~10KB	Embedded JavaScript strings	Developers who prefer plain JS in templates
Nunjucks https://mozilla.github.io/nunjucks	~45KB	Feature-rich string templates	Complex apps, template inheritance, SSR
lit-html https://lit.dev/docs/libraries/standalone-templates	~5KB	Tagged template literals, DOM diffing	Reactive components, frequent updates
uhtml https://github.com/WebReflection/uhtml	~2.5KB	Tagged template literals, DOM diffing	Minimal footprint, similar to lit-html

8.4 Using Slots

Slots are placeholders inside a component's shadow DOM where external content can be projected in. They allow component authors to define customization points while keeping the rest of the markup encapsulated. Think of slots as "holes" in your component's template that consumers from the main page fill with their own content.

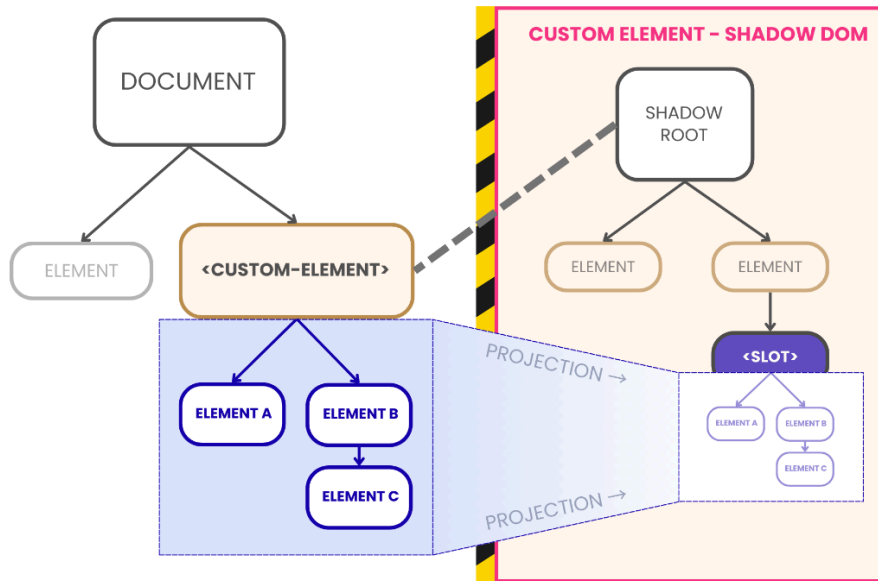


Figure 8.3 This diagram shows how elements from the main DOM can be projected into the shadow DOM of a custom element using the `<slot>` element. When you use a `<slot>` in a shadow DOM, the children of the custom element in the main DOM are projected into it as slotted content.

The `<slot>` element in the web component marks where projected content will appear. It does not render anything on its own; it simply reserves a location. When you use that component and place child elements inside it, those children get distributed to matching slots in the shadow DOM as seen in Figure 8.3. This mechanism is called *content projection* or *light DOM distribution*.

As with any customization on web components, each instance of the same web component can provide different content for the same slots. The content for slots can be from simple strings to complete trees of HTML elements.

`<slot>` is just an HTML element that can only exist in a shadow DOM, and it can be used in `<template>` or directly injected with other techniques into the shadow DOM.

The simplest example we can create is a web component that follows the decorator design pattern: it just decorates whatever content you provide. Let's think about it first from the consumer point of view, creating a highlight-card component that you use like this, and it just puts the content in a yellow box with some styles:

```

<highlight-card>Warning: you ran out of credit</highlight-card>

<highlight-card>
  <section> #1
    <h3>Error</h3> #1
    <p>We couldn't load the data</p> #1
  </section> #1
</highlight-card>

```

#1 Slotted content

The highlight card component's template uses `<slot></slot>` anywhere in its shadow DOM, and it will project the `Warning: you ran out of credit` message or the `<section>` tree on each instance.

```

<template id="highlight-card-template">
  <style>
    :host {
      background: yellow;
      display: block;
      margin: 1em;
      padding: 1em;
    }
    section {
      border: 1px solid red;
      padding: 1em;
    }
  </style>
  <section>
    <slot></slot> #1
  </section>
</template>

```

#1 Anything inside the slot will be treated as default content

In the previous example, we used the simplest slot: it's just a default slot that projects whatever you put within the custom element in the `<slot>`.

There are two kinds of slots:

- *Unnamed slot (default)*: A `<slot>` without a `name` attribute. Any child nodes of the custom element that do not have a `slot` attribute are assigned here. A shadow root can have only one unnamed slot; if you define multiple unnamed slots, only the first one receives content, and the rest are ignored.

- *Named slot:* A `<slot>` with a `name` attribute. Only children with a matching `slot="name"` attribute are assigned to it. You can have as many named slots as you need, but each name should be unique within a shadow root.

In the same shadow DOM, you can have one unnamed slot and several named slots coexisting.

8.4.1 Fallback Content

If a slot receives no content from the page DOM, it displays its fallback content—whatever you place between the opening and closing `<slot>` tags:

```
<slot>No message provided.</slot>
```

Fallback content is only visible when nothing is projected into the slot. As soon as a consumer provides matching content, the fallback disappears entirely.

8.4.2 Full example

The `product-details` component in Listing 8.7 uses slots to allow consumers to inject a title, description, and action button.

Listing 8.7 Using slots in a web component

```

<template id="details-template">
  <style>
    :host { display: block; border: 1px solid #ddd; padding: 1rem; border-radius:
8px; }
    h3 { margin-top: 0; }
    .description { color: #555; }
    .actions { margin-top: 1rem; padding-top: 1rem; border-top: 1px solid #eee; }
  </style>
  <h3>
    <slot name="title">Untitled Product</slot> #1
  </h3>
  <p class="description">
    <slot></slot> #2
  </p>
  <div class="actions">
    <slot name="action"><button disabled>Out of stock</button></slot> #3
  </div>
</template>

<script>
class ProductDetails extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: 'open' });
    const template = document.getElementById('details-template');
    const clone = document.importNode(template.content, true);
    this.shadowRoot.appendChild(clone);
  }
}
customElements.define('product-details', ProductDetails);
</script>

```

#1 Named slot for the product title with fallback text

#2 Default (unnamed) slot for the description

#3 Named slot for the action button with a disabled fallback

Now we can use the component as in:

```

<product-details>
  <span slot="title">USB-C Camera</span> #1
  High-resolution 12-megapixel camera with optical zoom and low-light enhancement.
#2
  <button slot="action">Add to Cart</button> #3
</product-details>

```

#1 Projects into the title slot

#2 Projects into the default slot (no slot attribute needed)

#3 Projects into the action slot

As seen in Figure 8.4 when rendered, the component displays "USB-C Camera" as the heading, the description text in the paragraph, and the "Add to Cart" button in the actions area. If you omit the action button, the fallback "Out of stock" button appears instead. The order in which the elements are used in `<product-details>` is irrelevant.

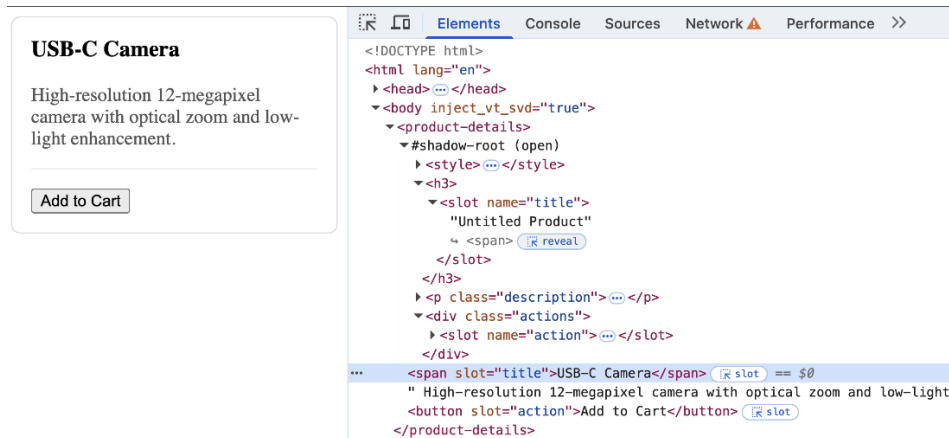


Figure 8.4 Using named slots lets you project different content on different parts of your web component's template.

When inspecting the elements in most browsers' developer tools, you can see a flag when a slot projection is in place. In Chromium-based browsers, you see an "slot" button that points to the `<slot>` element where it is currently projected, and a "reveal" button in the `<slot>` element that points to the markup that is being projected in that instance, as you can see in Figure 8.4.

8.4.3 The Flattened Tree

When the browser renders a component with slots, it creates a *flattened tree* that combines the shadow DOM with the projected light DOM. It is purely a rendering concept: the actual DOM structure remains unchanged.

Consider this component usage:

```
<product-details>
  <span slot="title">Camera</span>
  A great camera.
</product-details>
```

In figure 8.5 you can see on the left how the DOM tree remains, and on the right the flattened tree (what gets rendered).

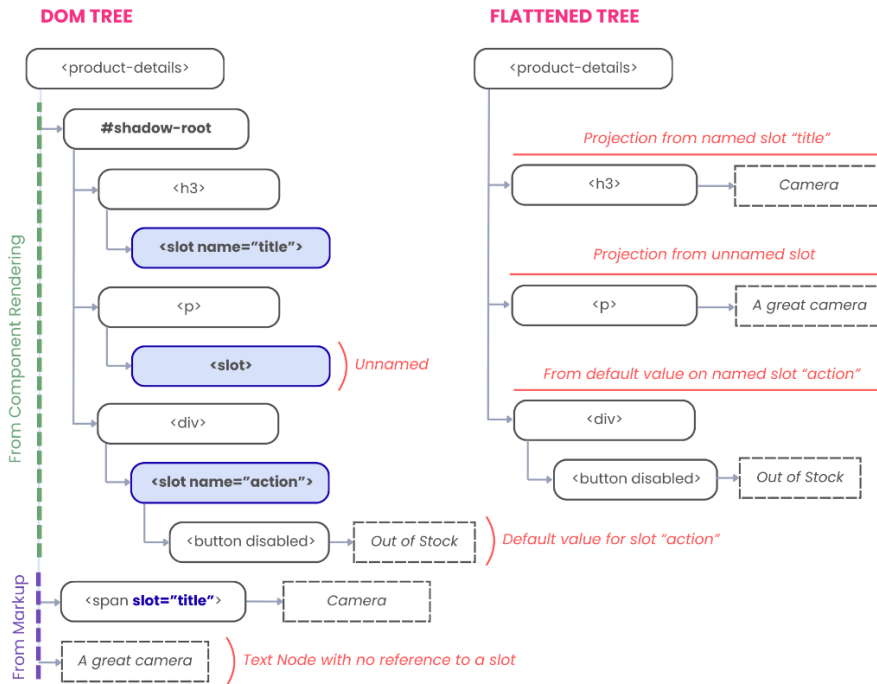


Figure 8.5 The flattened tree combines the shadow DOM with the projected light DOM from the main document and it helps us understand how slots are being rendered, for both named and unnamed slots, and for default or projected content.

The page DOM children remain in their original location; they are not moved into the shadow DOM. The slot mechanism projects them visually without relocating them in the DOM tree.

8.4.4 Multiple Elements in One Slot

A slot can receive multiple elements. All children with a matching `slot` attribute (or no attribute for the default slot) are projected in order:

```
<product-details>
  <span slot="title">Camera Bundle</span>
  <p>Includes camera body and two lenses.</p> #1
  <p>Perfect for beginners and professionals alike.</p> #1
  <ul> #1
    <li>12MP sensor</li> #1
    <li>4K video</li> #1
    <li>Weather-sealed</li> #1
  </ul> #1
</product-details>
```

#1 Slotted content for the default slot

In the previous sample, all three elements (two paragraphs and the list) are placed in the default slot and rendered in sequence.

8.5 Styling Slotted Content

One of the most important things to understand about slots is where styling comes from. Slotted content is styled by the document where it's defined, not by the shadow DOM where it's projected.

```
<!-- Main document styles apply to slotted content -->
<style>
  .highlight { color: red; font-weight: bold; }
</style>

<product-details>
  <span slot="title" class="highlight">Special Offer!</span>
  This product is on sale.
</product-details>
```

The `.highlight` class from the main document applies to the slotted span, even though it renders inside the shadow DOM.

WARNING We need to remember that slotted content is projected into the shadow DOM, but it doesn't live there.

8.5.1 The `::slotted()` pseudo-Element

The shadow DOM can style slotted content using the `::slotted()` pseudoelement, but with a significant limitation: it only selects direct children, not their descendants.

```
/* Inside shadow DOM */

::slotted(*) { #1
  margin: 0; #1
  padding: 0.5rem; #1
} #1

::slotted(span) { #2
  font-weight: bold; #2
} #2

::slotted([slot="title"]) { #3
  font-size: 1.5rem; #3
} #3

/* */
::slotted(p span) { #4
  color: red; #4
} #4

::slotted(*) span { #4
  color: red; #4
} #4
```

#1 Targets the slotted elements

#2 Targets only `<span>` slotted elements

#3 Targets the slotted element for the slot named title

#4 These selectors are invalid: cannot select descendants

This limitation is intentional. It preserves encapsulation by preventing the component from accessing consumer-provided content too deeply.

8.5.2 Styling Strategies

Given the limitations of `::slotted()`, here are practical approaches for styling:

Style the slot's container:

```
/* Shadow DOM styles */
.title-wrapper {
  font-size: 1.5rem;
  font-weight: bold;
  color: #333;
}
```

```
<!-- Shadow DOM template -->
<div class="title-wrapper">
  <slot name="title"></slot>
</div>
```

Use CSS custom properties as a styling bridge:

```
/* Shadow DOM */
:host {
  --title-color: #333;
  --title-size: 1.5rem;
}

.title-wrapper {
  color: var(--title-color);
  font-size: var(--title-size);
}
```

```
/* Consumer document */
product-details {
  --title-color: navy;
  --title-size: 2rem;
}
```

Document expectations for slotted content:

Sometimes the most straightforward approach is to document that consumers should apply certain classes or styles to their slotted content:

```
<!-- Consumer follows component's documented conventions -->
<product-details>
  <h2 slot="title" class="product-title">Camera</h2>
</product-details>
```

8.6 Slots API

The Slots API provides methods and events to interact with slot assignment and monitor changes programmatically.

8.6.1 The slotchange Event

The `slotchange` event fires on a `<slot>` element whenever the direct nodes assigned to it change: when elements are added, removed, or when their `slot` attribute changes. It won't be fired when children of the direct nodes are added, removed, or changed.

8.6.2 assignedNodes and assignedElements

Every `<slot>` element has two methods for retrieving its projected content:

- `assignedNodes(options)`: Returns all assigned nodes, including text nodes and comments.
- `assignedElements(options)`: Returns only assigned element nodes, filtering out text and comments.

Both methods accept an options object with a `flatten` property:

```
const slot = this.shadowRoot.querySelector('slot[name="items"]');
const elements = slot.assignedElements(); #1
const nodes = slot.assignedNodes(); #2
const flattened = slot.assignedNodes({ flatten: true }); #3
```

#1 Get only element nodes assigned to this slot

#2 Get all nodes, including text nodes

#3 Flatten nested slots (get the actual rendered content)

The `flatten: true` option is practical when you have nested components with slots inside slots. Instead of returning the nested `<slot>` element, it returns what that nested slot ultimately renders.

8.6.3 Practical Example: Tab Component

Listing 8.8 shows a tab container component that uses the Slots API to create tab buttons from slotted panels dynamically:

Listing 8.8 The TabContainer declaration and registration

```

class TabContainer extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({ mode: 'open' });
    this.shadowRoot.innerHTML = `
      <style>
        .tabs { display: flex; border-bottom: 2px solid #ddd; }
        .tab-button {
          padding: 0.75rem 1rem; border: none; background: none;
          cursor: pointer; border-bottom: 2px solid transparent;
          margin-bottom: -2px;
        }
        .tab-button.active { border-bottom-color: #007bff; color: #007bff; }
        .tab-button:hover { background: #f5f5f5; }
        .panels { padding: 1rem; }
        ::slotted(*) { display: none; }
        ::slotted(.active) { display: block; }
      </style>
      <div class="tabs"></div>
      <div class="panels"> <slot></slot> </div>
    `;
  }

  connectedCallback() {
    const slot = this.shadowRoot.querySelector('slot');
    slot.addEventListener('slotchange', () => this.buildTabs());
  }

  buildTabs() {
    const slot = this.shadowRoot.querySelector('slot');
    const panels = slot.assignedElements();
    const tabBar = this.shadowRoot.querySelector('.tabs');

    tabBar.innerHTML = '';

    panels.forEach((panel, index) => {
      const button = document.createElement('button');
      button.className = 'tab-button';
      button.textContent = panel.getAttribute('data-tab-title') || `Tab ${index +
1}`;
      button.addEventListener('click', () => this.activateTab(index));
      tabBar.appendChild(button);
    });
  }
}

```



```

    });

    if (panels.length > 0) {
      this.activateTab(0);
    }
  }

  activateTab(index) {
    const slot = this.shadowRoot.querySelector('slot');
    const panels = slot.assignedElements();
    const buttons = this.shadowRoot.querySelectorAll('.tab-button');

    panels.forEach((panel, i) => {
      panel.classList.toggle('active', i === index);
    });

    buttons.forEach((button, i) => {
      button.classList.toggle('active', i === index);
    });
  }
}

customElements.define('tab-container', TabContainer);

```

Now that we have defined and registered the tab container component, we are ready to use it by projecting the children of `<tab-container>` into the component's slot, as seen in Listing 8.9.

Listing 8.9 The TabContainer component usage example

```
<tab-container>
  <div data-tab-title="Overview">
    <h3>Product Overview</h3>
    <p>This camera features a 12MP sensor...</p>
  </div>
  <div data-tab-title="Specifications">
    <h3>Technical Specs</h3>
    <ul>
      <li>Sensor: 12MP APS-C</li>
      <li>Video: 4K 30fps</li>
    </ul>
  </div>
  <div data-tab-title="Reviews">
    <h3>Customer Reviews</h3>
    <p>4.5/5 from 234 reviews</p>
  </div>
</tab-container>
```

The component reads the `data-tab-title` attribute from each slotted panel to create the tab buttons, and uses `slotchange` to rebuild the tabs if panels are dynamically added or removed, as seen in [8.6](#).

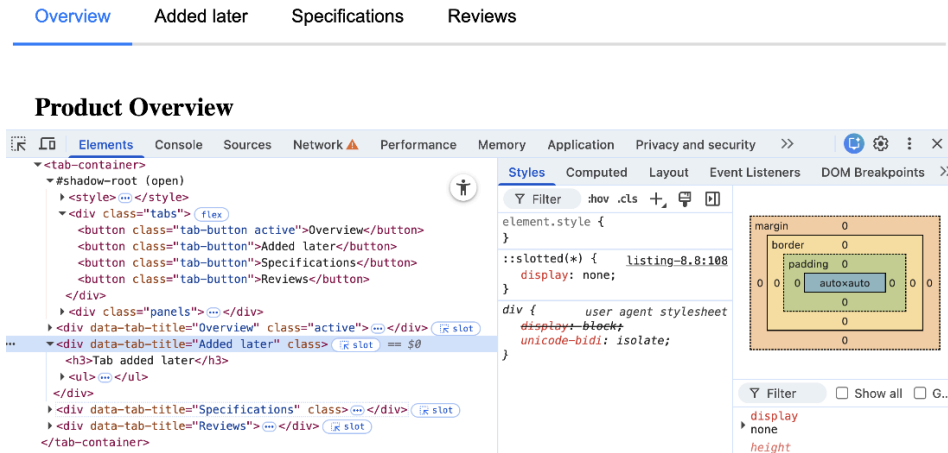


Figure 8.6 This tab container updates automatically the tab lists if a new element appears, is moved or is removed as a child of the web component, thanks to the `slotchange` event handler.

8.6.4 Finding a Slot's Assigned Slot

Elements from the main document have an `assignedSlot` property that returns the slot they're projected into:

```
const span = document.querySelector('span[slot="title"]');
console.log(span.assignedSlot); #1
```

#1 The `<slot name="title">` element in the shadow DOM

Finding the assigned slot is useful when you need to traverse from light DOM content back to its projection point.

8.7 Nested Slots

Slots can be nested when components contain other components. The inner component's slots can receive content passed through the outer component's slots.

```
<!-- outer-component template -->
<div class="wrapper">
  <inner-component>
    <slot name="content" slot="inner-content"></slot>
  </inner-component>
</div>
```

```
<!-- inner-component template -->
<div class="inner-wrapper">
  <slot name="inner-content"></slot>
</div>
```

```
<!-- Usage -->
<outer-component>
  <p slot="content">This text passes through both components</p>
</outer-component>
```

The paragraph is assigned to outer-component's "content" slot, which itself is slotted into inner-component's "inner-content" slot.

This pattern is powerful but can become confusing. Document slot hierarchies clearly and avoid deeply nested slot chains when possible.

8.8 Slot Best Practices

Name slots semantically. Use descriptive names like `header`, `footer`, `title`, `actions` rather than generic names like `slot1`, `slot2`. The slot name is part of your component's public API.

Provide meaningful fallback content. Fallbacks should make the component usable even when consumers don't fill every slot. A disabled button or placeholder text is better than an empty gap.

Document available slots. Consumers need to know what slots exist and what content they expect. Consider adding a README or JSDoc comments listing each slot.

Keep slot hierarchies shallow. Deeply nested slots are hard to reason about. If you find yourself nesting more than two levels, consider restructuring your components.

Use slots for customization, not core content. Slots are best for optional customization points. If content is required for the component to function, consider using attributes or properties instead.

```
<product-card title="Camera" price="$199"></product-card> #1

<product-card title="Camera" price="$199"> #2
   #2
  <div slot="description"> #2
    <p>Professional-grade camera...</p> #2
    <ul>...</ul> #2
  </div> #2
</product-card> #2
```

#1 Prefer attributes for required data

#2 Use slots for optional rich content

8.9 Summary

- The `<template>` element lets you define inert HTML fragments in the document: they are not rendered and do not affect layout until you instantiate them.
- Using a template means working with its `.content`, a `DocumentFragment` that you can clone before inserting in the DOM: each instance of your component needs its own nodes, so you do not accidentally move elements around the DOM.
- Static templates pair well with Custom Elements: keep one shared `template` per class, clone per instance, and attach it to the light DOM or the shadow DOM based on your encapsulation goals.
- There are multiple ways to ship templates: inline in the main document, fetched as an HTML file, or retrieved from Cache Storage to avoid repeated network work.

- When loading external HTML files for your templates, we need to add loading and error states to our web components.
- Loading CSS stylesheets with `<link>` elements in the template HTML may lead to Flash Of Unstyled Text (FOUT) effects, so use `adoptedStyles` instead, insert the CSS in `<style>` elements, or monitor the stylesheet loading process.
- Dynamic templates with data: you can populate a cloned fragment by querying nodes and setting `textContent` and attributes, or by doing controlled placeholder replacement when you fully trust the input.
- Template engines are an optional layer on top of templates: they turn data into a DOM by combining cached fragments with binding rules for text, attributes, and repeated sections.
- A small custom engine can cover most needs: map data into a cloned fragment, update only what changed, and avoid `innerHTML` for untrusted content to reduce XSS risk.
- The `<slot>` element enables composition: the component owns structure and behavior, while consumers provide children that will be projected; named slots support layouts like title, media, and actions without leaking internal markup.
- There is only one default unnamed slot and multiple named slots per shadow DOM. Slotted content appears when consuming the web component as child elements; if `slot=""` is present, it is treated as content for a named slot projection.
- Slots are also reactive: `slotchange` and `assignedNodes` let a component respond when the consumer adds, removes, or reorders slotted content.
- Slotted content is primarily styled from the page DOM that owns the content; the shadow DOM can style direct descendants using the `::slotted(selector) pseudoelement`.

9 *Advanced web components*

This chapter covers

- Adding `ElementInternals` for forms and accessibility
- Hooking into the full custom element lifecycle
- Managing focus, keyboard interaction, and custom states
- Rendering components with Declarative Shadow DOM
- Testing and deploying web components
- Integrating web components across stacks

We have already built web components with custom elements, templates, and shadow DOM. The next step is to cover advanced APIs and patterns that make components feel more like native controls, easier to integrate, and simpler to ship in real projects.

9.1 Element internals

The `'ElementInternals'` interface gives custom elements access to capabilities that plain `HTMLElement` subclasses do not get automatically. The most important ones are form participation, constraint validation, and default accessibility semantics.

Without `ElementInternals`, a custom control can still render and respond to events, but it will not behave like a native form field by default. It will not automatically submit values, integrate with form validation in the same way, or expose advanced ARIA defaults as cleanly.

To use the API, call `attachInternals()` once inside the constructor and keep the returned object for later use:

```
class CustomControl extends HTMLElement {
  constructor() {
    super();
    this.internals = this.attachInternals();
  }
}
```

The returned `ElementInternals` object becomes the bridge between your component and browser features such as form submission, labels, validation, and accessibility metadata.

9.1.1 Form participation

The most powerful feature of `ElementInternals` is form participation. A *form-associated custom element* can:

- submit a value with its parent form
- participate in constraint validation
- react to form resets
- react to disabled state changes
- appear in `form.elements`
- associate with `<label>` elements

To enable form participation, add `static formAssociated = true` to the class and call `attachInternals()` in the constructor.

For example, we can make a text field component participate in a form, validate a required value, and react to changes in the reset and disabled states. We will start by defining the HTML template in a variable as in the following code:

```
const template = `
  <style>
    :host { display: inline-block; }
    input {
      font: inherit;
      padding: 0.5rem;
      border: 1px solid #ccc;
    }
    input:focus {
      outline: 2px solid blue;
    }
  </style>
  <input type="text">
`;
```

After the HTML is ready, we can create a basic class with its constructor and a value property as we can see in the following listing.

Listing 9.1 A form-associated custom element

```
class FancyInput extends HTMLElement {
  static formAssociated = true;
  static observedAttributes = ["value", "required", "disabled"];

  #value = "";
  #input;

  constructor() {
    super();
    this.internals = this.attachInternals();
    this.attachShadow({ mode: "open" });

    this.shadowRoot.innerHTML = template;

    this.#input = this.shadowRoot.querySelector("input");
    this.#input.addEventListener("input", () => {
      this.value = this.#input.value;
    });
  }

  get value() {
    return this.#value;
  }

  set value(v) {
    this.#value = String(v ?? "");
    this.#input.value = this.#value;
    this.internals.setFormValue(this.#value);
    this.validate();
  }
}

customElements.define("fancy-input", FancyInput);
```

After that we can add the methods in the class to make it work, as in:


```

connectedCallback() {
  this.#syncFromAttributes();
  this.validate();
}

attributeChangedCallback() {
  this.#syncFromAttributes();
  this.validate();
}

formResetCallback() {
  this.value = this.getAttribute("value") || "";
}

formDisabledCallback(disabled) {
  this.#input.disabled = disabled;
}

validate() {
  if (this.hasAttribute("required") && !this.#value) {
    this.internals.setValidity(
      { valueMissing: true },
      "This field is required",
      this.#input
    );
  } else {
    this.internals.setValidity({});
  }
}

#syncFromAttributes() {
  if (this.hasAttribute("value")
    && this.#value !== this.getAttribute("value")) {
    this.#value = this.getAttribute("value");
    this.#input.value = this.#value;
    this.internals.setFormValue(this.#value);
  }
  this.#input.disabled = this.hasAttribute("disabled");
}

```

Once the component is defined, you can use it in a form just like a native input:

```

<form id="signup">
  <label for="username">Username</label>
  <fancy-input id="username" name="username" required></fancy-input>
  <button type="submit">Sign Up</button>
</form>

<script>
  document.getElementById("signup").addEventListener("submit", (e) => {
    e.preventDefault();
    const data = new FormData(e.target);
    console.log("Username:", data.get("username"));
  });
</script>

```

The important detail is that `setFormValue()` makes the component participate in `FormData` and form submission without extra glue code in the page.

9.1.2 Validation methods

`ElementInternals` exposes validation methods that mirror the behavior of native form controls:

- *setValidity(flags, message, anchor)* sets the validity state.
- *checkValidity()* returns `true` or `false` and dispatches `invalid` when appropriate.
- *reportValidity()* does the same check and asks the browser to show its validation UI when possible.

The `flags` object can include familiar keys such as `valueMissing`, `typeMismatch`, `patternMismatch`, `tooLong`, `tooShort`, `rangeUnderflow`, `rangeOverflow`, `stepMismatch`, `badInput`, or `customError`.

The following snippet shows the most common pattern when using validation methods:

```

this.internals.setValidity( #1
  { customError: true }, "Username already taken", #1
  this.shadowRoot.querySelector("input") #1
); #1

if (!this.internals.checkValidity()) { #2
  console.log("Validation failed:", this.internals.validationMessage); #2
} #2

this.internals.reportValidity(); #3

```

#1 Mark as invalid with a custom error

#2 Check validity programmatically

#3 Ask the browser to report the error

Native-like validation is one of the clearest reasons to use form-associated custom elements instead of building a loose widget that only looks like a field.

9.1.3 Accessing form and labels

`ElementInternals` also exposes read-only properties that let a component inspect its form context. The most useful ones are:

- `form`: the associated `HTMLFormElement`, or `null`
- `labels`: a `NodeList` of associated labels
- `willValidate`: whether the control participates in constraint validation

A component can inspect them in `connectedCallback()` or in response to user actions:

```

connectedCallback() {
  console.log("Form:", this.internals.form);
  console.log("Labels:", this.internals.labels);
  console.log("Will validate:", this.internals.willValidate);
}

```

Those properties are especially useful when a component needs to adapt behavior based on whether it is inside a form or associated with one through the `form` attribute.

9.2 Web components lifecycle

A custom element goes through several phases during its lifetime. Understanding when each callback runs helps you avoid duplicated listeners, stale DOM references, and awkward initialization bugs.

In Chapter 7, we introduced `connectedCallback()` and `attributeChangedCallback()`. Here we complete the picture, shown in [9.1](#), and then cover the form-associated callbacks that become available when `formAssociated` is enabled—relevant only to specific kinds of web components.

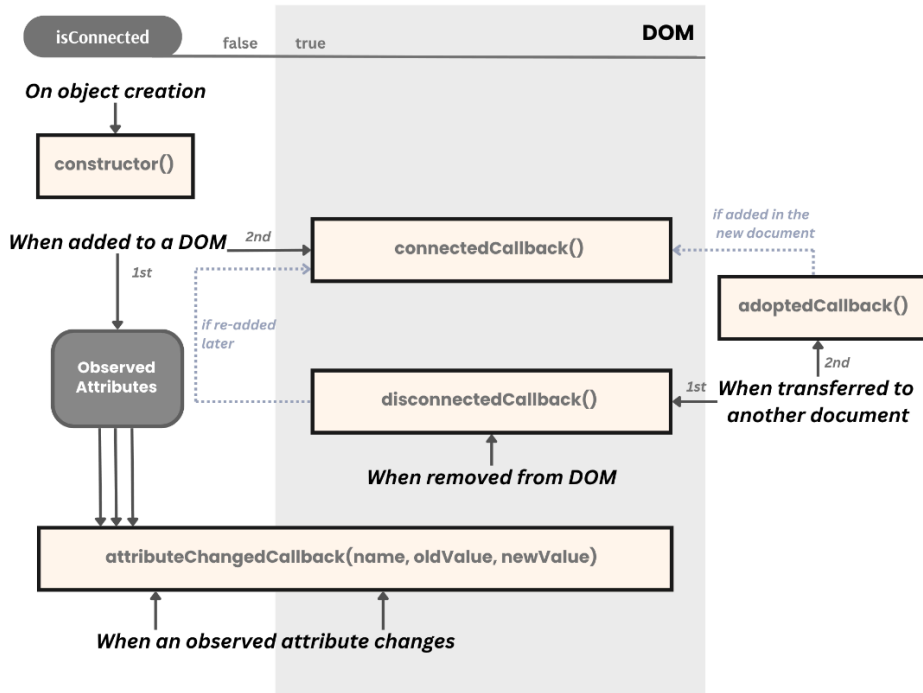


Figure 9.1 The basic lifecycle execution of a custom element, from object creation through being added to, removed from, or transferred to another document. The `attributeChangedCallback` can execute whether `isConnected` is **true** or **false**.

9.2.1 Basic lifecycle flow

The core lifecycle callbacks of every custom element in a web component are:

1. `'constructor()'`: runs when the element is created or upgraded. Use it for internal state, attaching a shadow root, and binding handlers.
2. `'connectedCallback()'`: runs each time the element is inserted into a document.
3. `'attributeChangedCallback(name, oldValue, newValue)'`: runs when an observed attribute changes.
4. `'disconnectedCallback()'`: runs each time the element is removed from a document.

5. ``adoptedCallback()`: runs when the element moves to a new document, for example, through ``document.adoptNode()`.

The `isConnected` property is also helpful. It tells you whether the element is currently connected to a document, which is useful when an attribute change triggers rendering logic before the element has been inserted.

The following listing logs the basic callbacks of a custom element in one place.

Listing 9.2 A custom element showing the main lifecycle callbacks

```

class LifecycleDemo extends HTMLElement {
  static observedAttributes = ["status"];

  constructor() {
    super();
    console.log("1. constructor");
    this.attachShadow({ mode: "open" });
  }

  connectedCallback() {
    console.log("2. connectedCallback");
    this.render();
  }

  attributeChangedCallback(name, oldValue, newValue) {
    console.log(`3. attributeChangedCallback: ${name} changed`);
    if (this.isConnected) {
      this.render();
    }
  }

  disconnectedCallback() {
    console.log("4. disconnectedCallback");
  }

  adoptedCallback() {
    console.log("5. adoptedCallback");
  }

  render() {
    this.shadowRoot.innerHTML =
      `<p>Status: ${this.getAttribute("status") || "none"}</p>`;
  }
}

customElements.define("lifecycle-demo", LifecycleDemo);

```

Attribute changes can happen before `connectedCallback()` if the parser encounters attributes during upgrade. Guarding DOM work with `isConnected` or an internally initialized flag keeps the code predictable.

WARNING `attributeChangedCallback()` can run before `connectedCallback()` when an element is upgraded with attributes already present in the markup. Guard DOM reads and writes accordingly.

9.2.2 The `adoptedCallback()` in practice

The `adoptedCallback()` lifecycle method runs when the element moves to a new document and it's rare, but it matters when an element moves between documents, such as when working with iframes or documents created with `DOMParser` or `document.implementation.createHTMLDocument()`.

The following example adopts an element into an iframe document:

```
const iframe = document.querySelector("iframe");
const element = document.querySelector("my-component");

const adopted = iframe.contentDocument.adoptNode(element);
iframe.contentDocument.body.appendChild(adopted);
```

When adoption happens, `adoptedCallback()` runs. If you later insert the adopted node into the target document, `connectedCallback()` runs there as well.

Most components do not need `adoptedCallback()`, but the callback is useful if you cache references to `document`, `window`, or other document-scoped objects that must be refreshed after adoption.

9.2.3 Form-associated lifecycle

Form-associated custom elements add four more callbacks:

1. ``formAssociatedCallback(form)``: called when the element becomes associated with a form or loses that association.
2. ``formDisabledCallback(disabled)``: called when the disabled state changes, including through an ancestor `<fieldset>`.
3. ``formResetCallback()``: called when the form resets.
4. ``formStateRestoreCallback(state, mode)``: called when the browser restores user state or applies form-fill data.

The next code shows the structure of a minimal form-associated component with all four callbacks.

```

class CustomInput extends HTMLElement {
  static formAssociated = true;

  constructor() {
    super();
    this.internals = this.attachInternals();
  }

  formAssociatedCallback(form) {
    console.log("Associated with form:", form?.id || "none");
  }

  formDisabledCallback(disabled) {
    console.log("Disabled state changed:", disabled);
    this.style.opacity = disabled ? "0.5" : "1";
  }

  formResetCallback() {
    console.log("Form was reset");
    this.value = this.getAttribute("value") || "";
  }

  formStateRestoreCallback(state, mode) {
    console.log(`State restored (${mode}):`, state);
    this.value = state;
  }
}

customElements.define("custom-input", CustomInput);

```

Those callbacks are especially useful when the component's internal state is richer than a single string and needs to stay aligned with form behavior.

9.2.4 Restoring form state after navigation

`formStateRestoreCallback(state, mode)` deserves special attention because the second argument of `setFormValue()` lets you store a restoration state that differs from the submitted value.

That distinction is useful when the value submitted to the server is normalized, but the value restored for the user should preserve a richer structure.

The following code sample models a date-range picker that submits a compact string value, while storing a richer `FormData` state for restoration.


```

class DateRangePicker extends HTMLElement {
  static formAssociated = true;

  constructor() {
    super();
    this.internals = this.attachInternals();
    this.attachShadow({ mode: "open" });
  }

  set value({ start, end }) {
    const submittedValue = `${start}/${end}`;

    const state = new FormData();
    state.append("start", start);
    state.append("end", end);

    this.internals.setFormValue(submittedValue, state);
    this.render(start, end);
  }

  // Other methods (connectedCallback, event handling) omitted

  formStateRestoreCallback(state, mode) {
    if (mode === "restore" && state instanceof FormData) {
      this.render(state.get("start"), state.get("end"));
      return;
    }

    if (mode === "autocomplete" && typeof state === "string") {
      const [start, end] = state.split("/");
      this.render(start, end);
    }
  }

  render(start, end) {
    this.shadowRoot.innerHTML = `<p>${start} - ${end}</p>`;
  }
}

customElements.define("date-range-picker", DateRangePicker);

```

The first argument to `setFormValue()` is the submission value. The optional second argument is the restoration state that the browser may later pass back to `formStateRestoreCallback()`.

That difference matters more than it seems. Submission and restoration often look the same in simple controls, but richer widgets usually benefit from separating them.

9.2.5 Cleanup in `disconnectedCallback()`

`disconnectedCallback()` is where a component should release resources it owns. Timers, pending fetches, observers, and global listeners all belong here.

The following listing uses one `AbortController` for both a global listener and a fetch, making teardown simple and explicit.

```

class TimerComponent extends HTMLElement {
  #intervalId = null;
  #abortController = null;

  connectedCallback() {
    this.#abortController = new AbortController();
    this.#intervalId = setInterval(() => this.tick(), 1000);
    window.addEventListener("resize", this.handleResize, {
      signal: this.#abortController.signal
    });
    this.fetchData();
  }

  disconnectedCallback() {
    if (this.#intervalId !== null) {
      clearInterval(this.#intervalId);
      this.#intervalId = null;
    }
    if (this.#abortController) {
      this.#abortController.abort();
      this.#abortController = null;
    }
  }

  async fetchData() {
    try {
      const response = await fetch("/api/data", {
        signal: this.#abortController.signal
      });
      // Process response...
    } catch (e) {
      // Handle error...
    }
  }

  handleResize = () => { /* Handle resize */ };
  tick() { /* Update component */ }
}

```

Using `AbortController` like this avoids the classic bug where half the cleanup logic lives in one branch, and the other half gets forgotten.

9.2.6 Waiting for element registration

Sometimes, application code needs to use a custom element before its module has finished loading. The `customElements.whenDefined()` method solves that coordination problem by returning a promise that resolves when the given tag name is registered.

```
customElements.whenDefined("fancy-button").then(() => {
  const button = document.querySelector("fancy-button");
  button.label = "Ready";
});

await customElements.whenDefined("fancy-button");
document.querySelector("fancy-button").label = "Ready";
```

When you only need a synchronous existence check, `customElements.get("fancy-button")` is also useful.

9.2.7 Styling undefined elements with `:defined`

JavaScript has `whenDefined()`. CSS has `:defined` and `:not(:defined)`. Those selectors are handy when you want to hide or soften unupgraded elements until the browser has registered them, which is particularly useful when you render custom HTML elements from the server and the JavaScript defining those elements will be downloaded and parsed later.

```
fancy-button:not(:defined) {
  opacity: 0.3;
  pointer-events: none;
}

fancy-button:defined {
  opacity: 1;
}
```

Use that pattern carefully. Hiding a component entirely can be fine for decorative UI, but it may be the wrong choice if the component contains critical content that should remain visible before an upgrade.

9.3 Advanced accessibility for web components

Web components can be accessible by default, but they do not inherit native semantics automatically just because they look like buttons, toggles, or form controls. The browser sees a custom element as a generic host unless you provide more information.

The best strategy is still the boring one: use a native element when you can. When you cannot, `ElementInternals` provides a clean way to define default ARIA semantics for the host element.

9.3.1 ARIA via `ElementInternals`

`ElementInternals` exposes ARIA reflection properties such as `role`, `ariaLabel`, `ariaDescription`, `ariaChecked`, `ariaDisabled`, and many more. Setting them on `ElementInternals` is usually better than setting ARIA attributes directly on the page because it keeps the defaults within the component.

The following component behaves like a button and uses `ElementInternals` to define its default semantics for accessibility. We start by defining a template variable with the HTML contents:

```
const template = `
  <style>
    :host {
      display: inline-block;
      cursor: pointer;
    }
    :host([disabled]) {
      opacity: 0.5;
      cursor: default;
    }
  </style>
  <slot>Click me</slot>
`;
```

After we define the template, we can create the class that defines the Element, the `role` through internals, some accessibility properties, such as `ariaDisabled` and basic key management.

```

class CustomButton extends HTMLElement {
  #disabled = false;

  constructor() {
    super();
    this.internals = this.attachInternals();
    this.internals.role = "button";

    this.attachShadow({ mode: "open" });
    this.shadowRoot.innerHTML = template;

    this.tabIndex = 0;
    this.addEventListener("click", this.handleClick);
    this.addEventListener("keydown", this.handleKeydown);
  }

  get disabled() {
    return this.#disabled;
  }

  set disabled(value) {
    this.#disabled = Boolean(value);
    this.internals.ariaDisabled = this.#disabled ? "true" : "false";
    this.toggleAttribute("disabled", this.#disabled);
    this.tabIndex = this.#disabled ? -1 : 0;
  }

  handleClick = () => {
    if (this.#disabled) return;
    this.dispatchEvent(new Event("activate", { bubbles: true }));
  };

  handleKeydown = (e) => {
    if (this.#disabled) return;

    if (e.key === "Enter" || e.key === " ") {
      e.preventDefault();
      this.handleClick();
    }
  };
}

customElements.define("custom-button", CustomButton);

```

Notice the difference between semantics and behavior. Setting `ariaDisabled` communicates the state to assistive technology, but the component still has to prevent activation in code.

9.3.2 Common ARIA properties

`ElementInternals` exposes a large set of ARIA reflection properties. The following ones are among the most common in custom widgets.

Table 9.1 Common ARIA reflection properties exposed by `ElementInternals`

Property	Purpose
<code>role</code>	Defines what kind of widget the element represents
<code>ariaLabel</code>	Provides an accessible name when visible text is not enough
<code>ariaDescription</code>	Provides extra descriptive text
<code>ariaDisabled</code>	Marks the element as disabled but still perceivable
<code>ariaChecked</code>	Communicates checked state in switches, checkboxes, and radios
<code>ariaPressed</code>	Communicates pressed state in toggle buttons
<code>ariaExpanded</code>	Communicates whether a collapsible region is open
<code>ariaHidden</code>	Hides the element from assistive technologies
<code>ariaLive</code>	Creates a live region that announces updates

You can find the full list in the MDN ARIA reference at <https://developer.mozilla.org/docs/Web/Accessibility/ARIA/Reference/Attributes>. In practice, you should only set the properties that truly describe the widget you are building.

9.3.3 Building an accessible toggle

A toggle switch is a good example because it combines accessibility semantics, keyboard interaction, and optional form participation.

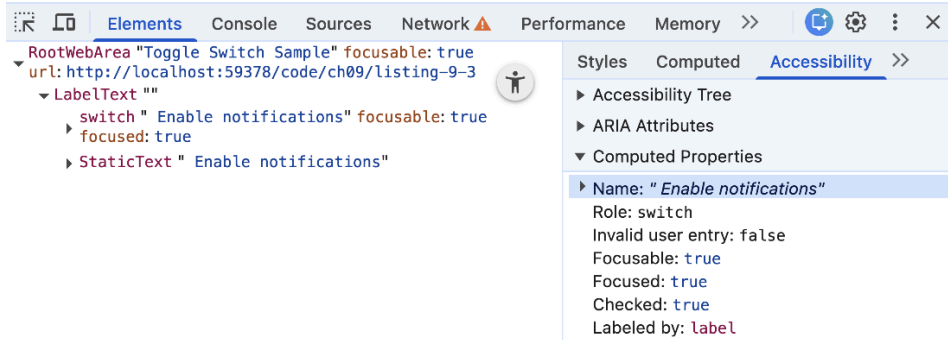
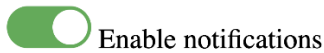


Figure 9.2 A toggle switch HTML element that can customize ARIA roles and attributes, such as checked and focused, that we can see in the ARIA explorer of our web component

Listing 9.3, shown in Figure 9.2, implements a switch that:

- uses `role="switch"`
- exposes `checked` and `disabled`
- reacts to click, Enter, and Space
- participates in forms
- uses the public `checked` attribute as the styling hook, instead of abusing `aria-checked` for CSS

NOTE For simplicity when listing code samples in this book we may be using HTML templates in the JavaScript file, but remember we can move that template to the HTML main DOM or to an external HTML file.

Let's start with the HTML template that for simplicity we define in the JavaScript.


```

const toggleTemplate = `
  <style>
    :host {
      display: inline-flex; align-items: center; gap: 0.5rem;
      cursor: pointer; user-select: none;
    }
    :host([disabled]) {
      opacity: 0.5; cursor: default;
    }
    .track {
      width: 3rem; height: 1.5rem;
      background: #ccc; border-radius: 1rem;
      position: relative; transition: background 0.2s;
    }
    :host([checked]) .track {
      background: #4caf50;
    }
    .thumb {
      width: 1.25rem; height: 1.25rem; background: white;
      border-radius: 50%; position: absolute; top: 0.125rem;
      left: 0.125rem; transition: transform 0.2s;
    }
    :host([checked]) .thumb {
      transform: translateX(1.5rem);
    }
  </style>

  <div class="track" aria-hidden="true">
    <div class="thumb"></div>
  </div>
  <slot></slot>
`
;

```

Now it's time to create the `ToggleSwitch` class with the switch behavior and ARIA custom properties management.

Listing 9.3 An accessible toggle switch

```

class ToggleSwitch extends HTMLElement {
  static formAssociated = true;
  static observedAttributes = ["checked", "disabled"];

  constructor() {
    super();
    this.internals = this.attachInternals();
    this.internals.role = "switch";

    this.attachShadow({ mode: "open" });
    this.shadowRoot.innerHTML = toggleTemplate;

    this.addEventListener("click", this.#onClick);
    this.addEventListener("keydown", this.#onKeyDown);
  }

  connectedCallback() {
    if (!this.hasAttribute("tabindex")) {
      this.tabIndex = this.disabled ? -1 : 0;
    }
    this.#updateState();
  }

  attributeChangedCallback() { this.#updateState(); }

  get checked() { return this.hasAttribute("checked"); }
  set checked(value) { this.toggleAttribute("checked", Boolean(value)); }

  get disabled() { return this.hasAttribute("disabled"); }
  set disabled(value) { this.toggleAttribute("disabled", Boolean(value)); }

  #onClick = () => {
    if (this.disabled) return;
    this.checked = !this.checked;
    this.dispatchEvent(new Event("change", { bubbles: true }));
  };

  #updateState() {
    this.internals.ariaChecked = this.checked ? "true" : "false";
    this.internals.ariaDisabled = this.disabled ? "true" : "false";
    this.internals.setFormValue(this.checked ? "on" : null);
    this.tabIndex = this.disabled ? -1 : 0;
  }
}

```

```

    }

    #onKeyDown = (e) => {
      if (this.disabled) return;
      if (e.key === "Enter" || e.key === " ") {
        e.preventDefault();
        this.#onClick();
      }
    };
  }
}

customElements.define("toggle-switch", ToggleSwitch);

```

Consumers can use it as a straightforward markup:

```

<label>
  <toggle-switch name="notifications"></toggle-switch>
  Enable notifications
</label>

```

The example uses a public `checked` attribute for both API and styling. Later in the chapter, `CustomStateSet` shows another option for internal-only states that should not become part of the public attribute contract.

9.4 Focus and keyboard management

Accessibility is not only about ARIA. Focus and keyboard behavior usually make or break a component.

A custom element that contains interactive controls has to answer a few questions clearly:

- Where does focus go first?
- How does keyboard interaction work?
- How does focus leave the component?
- Should focus return somewhere when the component closes?

9.4.1 Delegating focus into shadow DOM

When a component attaches a shadow root, the `delegatesFocus` option can help direct focus to the first focusable descendant when the host itself is focused.

```

constructor() {
  super();
  this.attachShadow({ mode: "open", delegatesFocus: true });
}

```

That option is especially useful for field-like components where the visible focus target lives inside the shadow tree.

`delegatesFocus` is helpful, but it is not a full focus-management strategy. Dialogs, menus, tab lists, and composite widgets still need explicit keyboard behavior.

9.4.2 Managing focus programmatically

Components that open and close often need to move focus intentionally. A modal dialog is the classic example: save the previously focused element, move focus into the dialog when it opens, and restore focus when it closes.

```

class ModalDialog extends HTMLElement {
  #previousActiveElement;

  open() {
    this.#previousActiveElement = document.activeElement;
    this.hidden = false;
    this.shadowRoot.querySelector("[autofocus]")?.focus();
  }

  close() {
    this.hidden = true;
    this.#previousActiveElement?.focus();
  }
}

```

Native `<dialog>` is still the best choice when it fits the problem. A custom dialog component should only recreate that behavior when the native dialog is not enough for the specific design or integration you need.

9.4.3 Keyboard interaction patterns

Keyboard support should follow established widget conventions, not invent new rules. The WAI-ARIA Authoring Practices Guide (<https://www.w3.org/WAI/ARIA/apg>) is still the best reference for those patterns.

Common expectations include:

- buttons activate with Enter or Space
- checkboxes and switches toggle with Space

- radio groups use arrow keys for navigation
- tabs use arrow keys to move between tab buttons
- menus use arrow keys and Escape
- dialogs trap focus and close with Escape

Listen to `keydown` or `keyup` events with `addEventListener()`. Avoid deprecated `keypress` patterns for modern component work.

```
this.addEventListener("keydown", (e) => {
  if (e.key === "Escape") {
    this.close();
  }
});
```

A component that looks polished but ignores keyboard patterns is not actually finished. It is just hiding its bugs behind CSS.

9.5 Custom state set

The `CustomStateSet` API allows autonomous custom elements to expose internal states to CSS through the `:state()` pseudo-class.

Native controls already have state selectors such as `:checked`, `:disabled`, or `:invalid`. `CustomStateSet` brings the same idea to your own components without forcing you to mirror every internal detail as attributes or classes.

Good state names tend to be short and descriptive, such as `"loading"`, `"expanded"`, `"selected"`, or `"error"`.

9.5.1 The states property

You access the state set through `ElementInternals.states`, which exposes set-like methods such as `add()`, `delete()`, and `has()`.

```

class MyElement extends HTMLElement {
  constructor() {
    super();
    this.internals = this.attachInternals();
  }

  connectedCallback() {
    this.internals.states.add("loading");
    this.internals.states.delete("loading");

    if (this.internals.states.has("ready")) {
      // ...
    }
  }
}

customElements.define("my-element", MyElement);

```

Use host attributes when the state is part of the public API. Use custom states when the state is mostly internal, and you only want CSS to react to it.

WARNING Custom states are not mutually exclusive. Multiple states can be active simultaneously. If you need exclusive states, you must implement the logic to remove conflicting states when a new one is added.

9.5.2 The `:state()` pseudo-class

Once a state is present in the set, CSS can select it with `:state()`.

```

my-element:state(loading) {
  opacity: 0.5;
  pointer-events: none;
}

my-element:state(error) {
  border-color: red;
}

my-element:state(loading):state(first-load) {
  background: linear-gradient(90deg, #eee, #ddd);
}

```

Because `:state()` targets the custom element itself, it composes cleanly with other selectors such as `::part()`:

```
my-accordion:state(expanded)::part(header) {
  font-weight: bold;
}

my-accordion:state(expanded)::part(icon) {
  transform: rotate(180deg);
}
```

That pattern is the correct way around: the state belongs to the host, and the part styling follows from it.

9.5.3 Building a stateful component

The following listing creates a button that exposes `loading`, `success`, and `error` as custom states that we can see in [9.3](#).

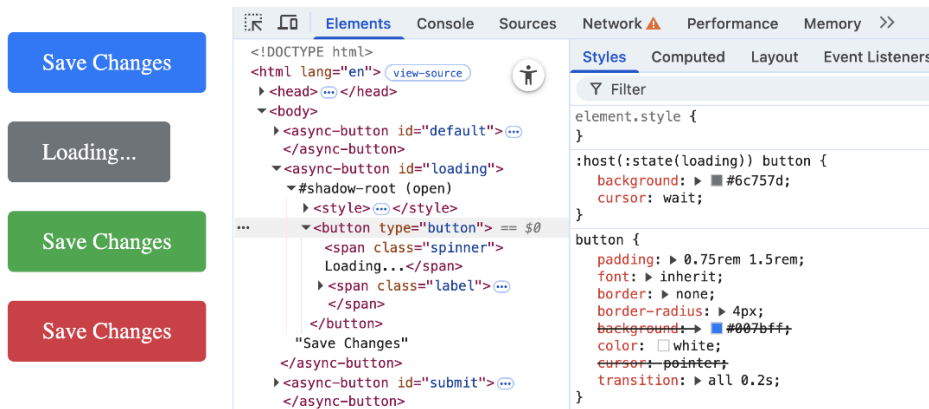


Figure 9.3 Four instances of the `async-button` component rendered in each of its states: default, loading, success, and error. The DevTools panel shows the styles applied to the loading instance.

Let's start with the HTML template:

```
const asyncButtonTemplate = `
  <style>
    button {
      padding: 0.75rem 1.5rem; font: inherit;
      border: none; border-radius: 4px;
      background: #007bff; color: white;
      cursor: pointer; transition: all 0.2s;
    }

    :host { display: inline-block; }

    :host(:state(loading)) button { background: #6c757d; cursor: wait; } #1
    :host(:state(success)) button { background: #28a745; } #1
    :host(:state(error)) button { background: #dc3545; } #1

    :host(:state(loading)) .label { display: none;} #2

    :host(:state(loading)) .spinner { display: inline; } #3
    :host(:not(:state(loading))) .spinner { display: none; } #3

  </style>

  <button type="button">
    <span class="spinner">Loading...</span>
    <span class="label"><slot>Submit</slot></span>
  </button>
`;
```

#1 Styles for the button in different states

#2 Styles for the label in loading state

#3 Styles for the spinner span in different states

After creating the template, we are ready to create the class component

Listing 9.4 A button component with internal CSS states

```

class AsyncButton extends HTMLElement {
  constructor() {
    super();
    this.internals = this.attachInternals();
    this.attachShadow({ mode: "open" });
    this.shadowRoot.innerHTML = asyncButtonTemplate;

    this.shadowRoot.querySelector("button").addEventListener("click", () => {
      this.dispatchEvent(new Event("action", { bubbles: true }));
    });
  }

  setLoading() {
    this.clearStates();
    this.internals.states.add("loading");
  }

  setSuccess() {
    this.clearStates();
    this.internals.states.add("success");
  }

  setError() {
    this.clearStates();
    this.internals.states.add("error");
  }

  reset() {
    this.clearStates();
  }

  clearStates() {
    this.internals.states.delete("loading");
    this.internals.states.delete("success");
    this.internals.states.delete("error");
  }
}

customElements.define("async-button", AsyncButton);

```

Now, let's see how a consumer can then drive those internal states from application logic with the following example that calls a `saveData` function. In this sample, `saveData` always resolve the promise after 3 seconds, which is enough to see how our button changes the state to loading and success. We can also see that we define the text of the button as `Save Changes` that replaces the `<slot>` default `Submit` value set in the component's template.

```
<async-button id="submit">Save Changes</async-button>

<script>
  const button = document.getElementById("submit");

  button.addEventListener("action", async () => {
    button.setLoading();
    try {
      await saveData();
      button.setSuccess();
    } catch (e) {
      button.setError();
    }
  });

  // It waits 3 seconds and resolve the promise
  function saveData() {
    return new Promise(
      (resolve, reject) => setTimeout(()=>resolve(), 3000)
    );
  }
</script>
```

`CustomStateSet` shines when styling should react to internal state, but you do not want to publish that state as part of the host element's visible markup.

9.6 Declarative Shadow DOM

Declarative Shadow DOM (DSD) lets the browser create a shadow root directly from HTML during parsing, without a JavaScript `attachShadow()` call.

You declare it with a `<template>` element that carries a `shadowrootmode` attribute. During parsing, the browser attaches a real shadow root to the host and moves the template contents into it.

That makes DSD especially useful for server-rendered or static HTML, where you want shadow content to be available before component JavaScript loads.

9.6.1 Basic syntax

The following example renders a product card with a declarative shadow root:

```
<product-card>
  <template shadowrootmode="open"> #1
    <link rel="stylesheet" href="product-card.css">

    <div class="row">
      <p class="title"><slot name="title">Untitled</slot></p>
      <p class="price"><slot name="price">$0</slot></p>
    </div>

    <p class="muted"><slot name="sku"></slot></p>
    <p><slot name="actions"></slot></p>
  </template>

  <span slot="title">USB-C Camera</span>
  <span slot="price">$199</span>
  <span slot="sku">SKU: CAM-UC-01</span>
  <button slot="actions" type="button">Add to cart</button>
</product-card>
```

#1 We define a template with an open shadow root

When the browser parses that markup, the shadow root already exists before the component class upgrades. The initial render does not have to wait for JavaScript.

WARNING Older Chrome-only examples used a `shadowroot` attribute. Use `shadowrootmode` instead. That is the current standardized syntax.

9.6.2 Hydrating a declarative shadow root

The hydration code needs one small but important adjustment: a component should check whether a shadow root already exists before creating one.

Calling `attachShadow()` on an element that already has a declarative shadow root only works when the requested mode matches the existing mode. In that case, the existing root is cleared and returned. If the modes do not match, the browser throws an exception.

Because clearing the root would destroy the server-rendered content, the safest pattern is to reuse an existing root and only call `attachShadow()` as a fallback.

```

class ProductCard extends HTMLElement {
  #root;

  constructor() {
    super();
    const internals = this.attachInternals();
    this.#root = internals.shadowRoot;

    if (!this.#root) {
      this.#root = this.attachShadow({ mode: "open" });
      this.#root.innerHTML = `
        <style>
          :host {
            display: block; border: 1px solid #ddd;
            padding: 12px; border-radius: 10px;
          }
          .title { font: 600 16px/1.2 system-ui; margin: 0 0 6px; }
          .price { font: 600 14px/1.2 system-ui; }
          .row {
            display: flex; justify-content: space-between;
            gap: 12px; align-items: baseline;
          }
        </style>

        <div class="row">
          <p class="title"><slot name="title">Untitled</slot></p>
          <p class="price"><slot name="price">$0</slot></p>
        </div>
        <p><slot name="actions"></slot></p>
      `;
    }

    this.#root.addEventListener("click", (e) => {
      const button = e.target.closest("button");
      if (!button) return;

      this.dispatchEvent(new CustomEvent("add-to-cart", {
        bubbles: true,
        composed: true
      }));
    });
  }
}

```

```
customElements.define("product-card", ProductCard);
```

`attachInternals().shadowRoot` works for both open and closed declarative shadow roots. `this.shadowRoot` only works for open ones.

That detail matters if you want the hydration pattern to survive a later change from open to closed mode.

9.6.3 Using CSS with DSD before hydration

A declarative shadow root can render before the custom element is registered, so `:defined` still matters. One useful pattern is to keep the component visible while soft-disabling interactivity until hydration finishes.

```
product-card:not(:defined) {
  opacity: 0.85;
}

product-card:not(:defined) button {
  pointer-events: none;
}
```

Hiding everything with `display: none` can throw away the whole benefit of server-rendering the shadow tree in the first place.

9.6.4 DSD configuration attributes

Declarative shadow roots also support attributes that mirror options normally passed to `attachShadow()`.

`SHADOWROOTDELEGATESFOCUS`

When present, `shadowrootdelegatesfocus` enables focus delegation:

```

<checkout-field>
  <template shadowrootmode="open" shadowrootdelegatesfocus>
    <style>
      :host { display: block; }
      :host(:focus-within) { outline: 2px solid blue; }
    </style>
    <label>
      <span><slot name="label">Email</slot></span>
      <input type="email" autocomplete="email">
    </label>
  </template>

  <span slot="label">Email address</span>
</checkout-field>

```

That is useful for field-like components where clicking the host should move focus to the first focusable element in the shadow tree.

SHADOWROOTCLONABLE

When present, `shadowrootclonable` makes the shadow root clonable so that `cloneNode(true)` or `importNode()` can include it.

```

<product-card id="template-card">
  <template shadowrootmode="open" shadowrootclonable>
    <style>
      :host { display: block; border: 1px solid #ddd; padding: 12px; }
    </style>
    <p><slot name="title"></slot></p>
  </template>

  <span slot="title">Template Product</span>
</product-card>

```

```

const original = document.querySelector("#template-card");
const clone = original.cloneNode(true);

console.log(clone.shadowRoot !== null);
document.body.append(clone);

```

That pattern is mostly useful for tooling, duplication of rendered fragments, or advanced client-side composition.

SHADOWROOTSERIALIZABLE

When present, `shadowrootserializable` allows the shadow root to be included when serializing with `getHTML()` and the appropriate option:

```
const html = element.getHTML({ serializableShadowRoots: true });
```

That feature is mainly relevant for tooling, testing, and server-rendering pipelines. Most application components will never need it directly.

9.6.5 Resource loading in declarative shadow roots

A declarative shadow root is real DOM, not inert template content. External stylesheets and images inside them start behaving like normal resources as soon as the browser parses them.

That is different from a regular inert `<template>`, where resources stay dormant until cloning.

```
<my-component>
  <template shadowrootmode="open">
    <link rel="stylesheet" href="component.css">
    
    <slot></slot>
  </template>

  <p>Content here</p>
</my-component>
```

That behavior is one reason DSD is useful for SSR. The shadow tree can render and start loading its own resources before the component class hydrates.

NOTE Inline and external stylesheets both work in declarative shadow roots. Identical external stylesheets referenced by multiple declarative shadow roots are loaded and parsed efficiently by the browser.

9.6.6 When to use Declarative Shadow DOM

Declarative Shadow DOM is most valuable when components are rendered on the server or generated as static HTML:

- server-side rendering with immediate styled output
- progressive enhancement where behavior arrives later

- static site generation that still wants real shadow DOM
- content that should render before component modules finish loading

DSD is less compelling when a component exists only on the client and is created by JavaScript anyway. In that case, imperative `attachShadow()` is usually enough.

Use the tool that matches the rendering model. DSD is powerful, but it is not a badge of modernity you need to pin on every component.

9.7 Testing web components

Web components are ordinary DOM elements and JavaScript classes, which means you can test them with many tools. The important part is the environment: component tests are more trustworthy in a real browser because features such as `customElements`, shadow DOM, slot distribution, and form association are part of the platform.

A useful mental model is to split tests into three layers:

- unit tests for methods and state transitions
- integration tests for DOM behavior and events
- visual regression tests for rendered appearance

9.7.1 A simple browser-based test page

The most direct way to understand component testing is still a simple HTML page that loads the component, runs assertions, and reports failures in the console.

The following listing shows a minimal test harness for a `my-counter` component that you can execute with `runTests()`.

Listing 9.5 A minimal browser test page for a custom element

```

import "./my-counter.js";

const tests = []; let passed = 0; let failed = 0;

function test(name, fn) { tests.push({ name, fn }); }

function assert(condition, message) {
  if (!condition) throw new Error(message || "Assertion failed");
}

async function runTests() {
  for (const { name, fn } of tests) {
    try {
      await fn();
      passed++; console.log(`PASS: ${name}`);
    } catch (e) {
      failed++; console.error(`FAIL: ${name}`, e.message);
    }
  }
  console.log(`\n${passed} passed, ${failed} failed`);
}

test("counter starts at zero", () => {
  const el = document.createElement("my-counter");
  document.body.appendChild(el);
  assert(el.value === 0, "Initial value should be 0");
  el.remove();
});

test("increment increases value", () => {
  const el = document.createElement("my-counter");
  document.body.appendChild(el);
  el.increment();
  assert(el.value === 1, "Value should be 1 after increment");
  el.remove();
});

test("click increments the counter", () => {
  const el = document.createElement("my-counter");
  document.body.appendChild(el);
  el.shadowRoot.querySelector("button").click();
  assert(el.value === 1, "Value should be 1 after click");
});

```

```
el.remove();
});
```

That style of testing is crude, but it teaches the right lesson: component tests are just browser code exercising browser behavior.

A manual HTML page is fine for exploration. Still, real projects usually move to an automated browser runner such as *Web Test Runner* or *Playwright*, or to a browser-backed setup integrated into CI.

Web Test Runner is built around real browsers and supports headless runs, watch mode, and CI-friendly output. That makes it a strong fit for platform features such as custom elements and shadow DOM.

9.7.2 Testing shadow DOM and slots

Shadow DOM changes how you query internal nodes, but it does not make the component untestable. You query through `shadowRoot` in open-shadow components.

```
// inside a test callback
const button = element.shadowRoot.querySelector("button");
button.click();
```

Slots also have their own API surface. Instead of checking rendered text mindlessly, it is often more robust to inspect assigned nodes or assigned elements:

```
// inside a test callback
element.innerHTML = `<span slot="title">My Title</span>`;

const slot = element.shadowRoot.querySelector('slot[name="title"]');
const assigned = slot.assignedElements();

if (assigned[0].textContent !== "My Title") {
  throw new Error("Named slot projection failed");
}
```

The same principle applies to default slots. Prefer verifying the actual slot assignment instead of guessing from outer HTML.

9.7.3 Testing form-associated components

Form-associated custom elements deserve explicit tests for submission and validation, because that is the whole reason for using the extra API surface.

```
// inside a test callback
const form = document.createElement("form");
const field = document.createElement("fancy-input");

field.setAttribute("name", "myfield");
form.appendChild(field);
document.body.appendChild(form);

field.value = "test-value";

const formData = new FormData(form);

if (formData.get("myfield") !== "test-value") {
  throw new Error("Form-associated value was not submitted");
}

form.remove();
```

Validation tests can also call `checkValidity()` or `reportValidity()` and verify the component's `validity` state or validation message.

9.7.4 Visual regression testing

Logic tests do not catch spacing regressions, missing outlines, or theme breakage. Screenshot-based testing is useful for components with multiple states, especially interactive ones.

Capture stable states such as:

- default
- hover or focus
- disabled
- error
- success or loading
- light and dark themes

Keep visual regression tests focused. They are useful, but they also tend to make noise if the state setup is sloppy.

9.8 Library integration

One of the strongest features of web components is that they remain HTML elements. That makes them usable inside React, Angular, Vue, and other libraries, even when the surrounding application has no interest in how the component is implemented internally.

The integration details differ a little across frameworks, but the basic rules stay the same:

- import or register the custom element before using the tag
- use attributes for simple string configuration
- use properties for complex values
- use native or custom events to communicate back out

9.8.1 Using web components in frameworks

React, Angular, and Vue can all render custom element tags directly.

React 19 has much better custom-element support than earlier React versions. It can assign properties on the client when it recognizes them on the element instance, and it supports listening to custom events with `on...` props that preserve event casing.

```
import "./fancy-button.js";

export default function App() {
  return (
    <fancy-button onActivate={(e) => console.log("activated", e.detail)}>
      Click me
    </fancy-button>
  );
}
```

If you need to pass a complex object into a custom element in React, a ref is still the safest route because properties are richer than attributes.

Angular can use custom elements in templates once the relevant compilation context allows them. In standalone components, `schemas: [CUSTOM_ELEMENTS_SCHEMA]` is available on the component decorator. In NgModule-based applications, the schema belongs in the module.

```
import { Component, CUSTOM_ELEMENTS_SCHEMA } from "@angular/core";
import "./fancy-button.js";

@Component({
  standalone: true,
  selector: "app-root",
  template: `<fancy-button (click)="handleClick()">Click me</fancy-button>`,
  schemas: [CUSTOM_ELEMENTS_SCHEMA]
})
export class AppComponent {
  handleClick() {
    console.log("clicked");
  }
}
```

Vue can consume custom elements directly, and the compiler can be configured to treat dashed tags as custom elements.

```
import { defineConfig } from "vite";
import vue from "@vitejs/plugin-vue";

export default defineConfig({
  plugins: [
    vue({
      template: {
        compilerOptions: {
          isCustomElement: (tag) => tag.includes("-")
        }
      }
    })
  ]
});
```

After that, the element can be used in a template just like any other tag:

```
<script setup>
import "./fancy-button.js";
</script>

<template>
  <fancy-button @click="handleClick">Click me</fancy-button>
</template>
```

Across all three frameworks, keep the rule simple: attributes for simple data, properties for complex data, and custom events for component-to-host communication.

9.8.2 Wrapping React components as web components

The reverse direction is also possible. A framework component can be wrapped inside a custom element so that non-framework pages can use it.

WARNING We are just scratching the surface in this book on how to connect Web Components with a framework or library. Check the official documentation from your favorite tool to get more information.

The next listing wraps a React button inside a custom element host.

```

import React from "react";
import { createRoot } from "react-dom/client";
import { MyReactButton } from "./MyReactButton.jsx";

class ReactButtonWrapper extends HTMLElement {
  static observedAttributes = ["label", "disabled"];

  constructor() {
    super();
    this.attachShadow({ mode: "open" });
    this.root = createRoot(this.shadowRoot);
  }

  connectedCallback() {
    this.render();
  }

  disconnectedCallback() {
    this.root.unmount();
  }

  attributeChangedCallback() {
    this.render();
  }

  render() {
    this.root.render(
      <MyReactButton
        label={this.getAttribute("label") || "Click"}
        disabled={this.hasAttribute("disabled")}
        onClick={() => {
          this.dispatchEvent(new Event("click", { bubbles: true }));
        }}
      />
    );
  }
}

customElements.define("react-button", ReactButtonWrapper);

```

That pattern is good enough for small wrappers. Larger libraries usually benefit from a helper layer that centralizes prop mapping and event forwarding.

9.8.3 Wrapping Angular components as web components

Angular exposes an official bridge for this use case through `@angular/elements`.

The following Angular component becomes a custom element through `createCustomElement()`:

```
// my-widget.component.ts
import { Component, EventEmitter, Input, Output } from "@angular/core";

@Component({
  selector: "internal-widget",
  template: `
    <div class="widget">
      <h3>{{ title }}</h3>
      <button (click)="handleClick()">{{ buttonText }}</button>
    </div>
  `,
  styles: [`.widget { padding: 1rem; border: 1px solid #ccc; }`]
})
export class MyWidgetComponent {
  @Input() title = "Widget";
  @Input() buttonText = "Click";
  @Output() widgetClick = new EventEmitter<void>();

  handleClick() {
    this.widgetClick.emit();
  }
}
```

Register it as a custom element:


```
// app.module.ts
import { Injector, NgModule } from "@angular/core";
import { createCustomElement } from "@angular/elements";
import { MyWidgetComponent } from "../my-widget.component";

@NgModule({
  declarations: [MyWidgetComponent]
})
export class AppModule {
  constructor(injector: Injector) {
    const element = createCustomElement(MyWidgetComponent, { injector });
    customElements.define("my-widget", element);
  }

  ngDoBootstrap() {}
}
```

Angular maps inputs to dash-cased attributes and dispatches outputs as custom events. Avoid reusing the Angular component selector as the custom-element tag name. Angular's official docs call that out as a source of confusing double-instantiation problems.

9.8.4 Wrapping Vue components as web components

Vue 3 exposes `defineCustomElement()`, which returns a native custom-element constructor. Using Vue 3.4+ syntax:

```
// MyButton.ce.vue
<script setup>
import { ref } from "vue";

const props = defineProps({
  label: { type: String, default: "Click" }
});

const emit = defineEmits(["button-click"]);
const count = ref(0);

function handleClick() {
  count.value++;
  emit("button-click", count.value);
}
</script>

<template>
  <button @click="handleClick">
    {{ label }} ({{ count }})
  </button>
</template>

<style scoped>
button {
  padding: 0.5rem 1rem;
  border: none;
  border-radius: 4px;
}
</style>
```

Registering the element is straightforward:

```
// main.js
import { defineCustomElement } from "vue";
import MyButton from "./MyButton.ce.vue";

const MyButtonElement = defineCustomElement(MyButton);
customElements.define("my-button", MyButtonElement);
```

Vue custom elements dispatch emitted events as native `CustomEvent` objects and can inject styles into the shadow root. Vue's docs also note that the custom-element path carries a runtime size cost, which is fine for a component library but often hard to justify for a single tiny element.

9.9 Deploying web components

Building the component is only half the job. Shipping it cleanly matters too.

A good distribution story should answer a few practical questions:

- How does someone load the component in a plain HTML page?
- How does someone consume it from a bundler-based application?
- How stable are the import paths over time?
- Is the element registered as a side effect, or does the consumer call a function?

9.9.1 Shipping a standalone ES module

The simplest distribution model is still a browser-ready ES module that registers the element as a side effect when imported.

```
/* file: /components/fancy-button.js */
class FancyButton extends HTMLElement {
  #initialized = false;

  connectedCallback() {
    if (this.#initialized) return;
    this.#initialized = true;

    if (!this.textContent.trim()) {
      this.textContent = "Click me";
    }

    this.addEventListener("click", () => {
      this.dispatchEvent(new CustomEvent("fancy-click", {
        bubbles: true
      }));
    });
  }
}

customElements.define("fancy-button", FancyButton);
```

A consumer can then use it directly:

```
<script type="module" src="/components/fancy-button.js"></script>
<fancy-button>Buy now</fancy-button>
```

That model is boring, and boring is good here. It keeps adoption friction low.

9.9.2 Publishing as an npm package

Bundler-based applications usually prefer an npm package. In that case, the package metadata should make the public entry points obvious and should preserve side-effect files that register elements.

A minimal `package.json` for a single-component package might look like this:

```
{
  "name": "vanilla-fancy-button",
  "version": "1.0.0",
  "type": "module",
  "main": "dist/fancy-button.cjs",
  "module": "dist/fancy-button.js",
  "exports": {
    ".": {
      "import": "./dist/fancy-button.js",
      "require": "./dist/fancy-button.cjs"
    }
  },
  "sideEffects": [
    "dist/fancy-button.js"
  ]
}
```

The important part is `sideEffects`. If the module exists solely to register the custom element, an aggressive bundler must not tree-shake it.

A consumer can then do the usual import:

```
import "vanilla-fancy-button";

document.querySelector("fancy-button")
  ?.addEventListener("fancy-click", () => {
    console.log("User interacted with the button");
  });
```

If the package exports multiple components, it is usually worth offering both a convenient root entry point and smaller per-component entry points.

9.9.3 Offering a CDN-friendly build

Many developers will want to try a component before installing anything. CDN-friendly ESM builds make that easy.

A browser-ready file published through npm can be loaded from services such as unpkg or jsDelivr:

```
<script
  type="module"
  src="https://unpkg.com/vanilla-fancy-button@1/dist/fancy-button.js">
</script>

<fancy-button>Subscribe</fancy-button>
```

The same file can also be imported from another module by URL:

```
import "https://unpkg.com/vanilla-fancy-button@1/dist/fancy-button.js";
```

Stable public paths matter here. If users paste a CDN URL into a page, that path becomes part of your external contract, whether you like it or not.

9.9.4 Bundling and transpiling

Web components do not require bundling, but bundling can still be useful for optimization, multiple output formats, and backward compatibility.

Typical reasons to bundle include:

- transpiling newer syntax for older targets
- producing both ESM and CommonJS outputs
- minifying and compressing published files
- combining internal modules into fewer files
- generating separate modern and legacy builds

Even when using a bundler, keep public entry points clear and documented. Consumers should not have to reverse-engineer the build system to load one element.

9.9.5 Distribution checklist

A small pre-publish checklist saves a lot of support emails later:

- register the element from a single documented entry module
- use a stable custom-element name with a hyphen
- document at least one plain HTML usage example
- expose clear package entry points in `package.json`
- keep CDN paths stable across versions

- version the package with semantic versioning
- document any public attributes, properties, events, CSS custom properties, and parts

A well-distributed web component should be easy to try, easy to install, and boring to maintain. Anything more theatrical usually means the packaging story is compensating for a weak API.

9.10 Summary

- Using `ElementInternals` gives custom elements native-like form participation, validation, labels, and accessibility defaults, which makes advanced controls behave more like built-in form elements.
- Understanding lifecycle callbacks such as `connectedCallback()`, `attributeChangedCallback()`, `disconnectedCallback()`, `adoptedCallback()`, and the form-associated callbacks helps components initialize, update, restore state, and clean up predictably.
- Applying accessibility semantics through `ElementInternals`, while pairing them with real behavior and solid keyboard support, makes custom widgets such as buttons and switches usable beyond their visual appearance.
- Managing focus intentionally, using patterns like `delegatesFocus`, explicit focus restoration, and established keyboard conventions, keeps interactive components aligned with native user expectations.
- Exposing internal UI state through `CustomStateSet` and `:state()` lets CSS react to component state without leaking every internal detail into public host attributes.
- Rendering components with Declarative Shadow DOM allows server-rendered or static HTML to include real shadow roots before JavaScript loads. At the same time, the hydration code can reuse existing roots instead of recreating them.
- Testing components in real browsers, including shadow DOM, slots, and form-associated behavior, makes component tests much more trustworthy than purely server-side DOM simulation.
- Integrating custom elements into React, Angular, and Vue works because web components stay framework-agnostic HTML elements, while framework components can also be wrapped and exported as custom elements when needed.
- Distributing components as ES modules, npm packages, or CDN-ready builds keeps adoption friction low, as long as entry points, side effects, and public API contracts stay clear and stable.